

Le Caméléon

THÈSE

présentée et soutenue publiquement le 10 décembre 2026

pour l'obtention du

Doctorat de l'Université de Lorraine

(mention informatique)

par

Yaya Sy

Composition du jury

<i>Présidents :</i>	Maxime Amblard	Professeur des universités Université de Lorraine, France
<i>Rapporteurs :</i>	Anthony Larcher	Professeur des universités Le Mans Université, France
	Benoît Crabbé	Professeur des universités Université Paris Cité, France
<i>Examineurs :</i>	Marie Candito	Maîtresse de conférence Université Paris Cité, France
<i>Directeurs de thèse :</i>	Christophe Cerisara	Chargé de recherche CNRS, LORIA, France
	Irina Illina	Maîtresse de conférence, HDR Université de Lorraine, France

Mis en page avec la classe thesul.

Ohana means family.
Family means nobody gets left behind, or forgotten.
— Lilo & Stitch

Dedicated to the loving memory of Rudolf Miede.
1939–2005

ABSTRACT

Scaling up large language models (LLMs) has unlocked many capabilities that do not emerge at smaller scales. However, larger models also incur greater training and inference costs. How can we scale model capacity while limiting its computational cost, particularly at inference time? We show that, unlike sparse LLMs, dense LLM activations exhibit a low-rank structure, suggesting that only a fraction of their representations is relevant during inference. Based on this observation, we study efficiency through (1) model compression and (2) activation sparsity. For model compression, we propose methods that compress LLMs by locally distilling their activations into low-rank weights, which greatly reduces inference and training costs. For activation sparsity, we first study Matryoshka embedding LLMs as a form of structured sparsity and show that they concentrate information in a few coordinates, wasting compute on the remaining ones. We therefore explore adaptive sparsity, where the model learns to allocate the optimal amount of sparsity depending on the input. We propose *ZeroNeuron*, a new method for learning adaptive structured activation sparsity for fast pre-training and inference. The method is based on stretching approaches and has many advantages over TopK-based activation sparsity, such as being differentiable and limiting cross-GPU communication in distributed training.

PUBLICATIONS

This might come in handy for PhD theses: some ideas and figures have appeared previously in the following publications:

This is just an example.

- Isenberg, Tobias, André Miede, and Sheelagh Carpendale (2006). “A Buffer Framework for Supporting Responsive Interaction in Information Visualization Interfaces.” In: *Proceedings of the Fourth International Conference on Creating, Connecting, and Collaborating through Computing (C⁵ 2006)*. IEEE, pp. 262–269. ISBN: 978-0-7695-2563-1.
- Lampe, Ulrich, Markus Kieselmann, André Miede, Sebastian Zöller, and Ralf Steinmetz (2013a). “A Tale of Millis and Nanos: On the Accuracy of Time Measurements in Virtual Machines.” In: *Proceedings of the Second European Conference on Service-Oriented and Cloud Computing (ESOCC 2013)*. Springer, pp. 172–179. ISBN: 978-3-642-40650-8.
- Lampe, Ulrich, Qiong Wu, Ronny Hans, André Miede, and Ralf Steinmetz (2013b). “To Frag Or To Be Fragged – An Empirical Assessment of Latency in Cloud Gaming.” In: *Proceedings of the Third International Conference on Cloud Computing and Services Science (CLOSER 2013)*, pp. 5–12. ISBN: 978-898-8565-52-5.
- Miede, André (2010). “Theses and other Beautiful Documents with classicthesis.” In: *TUGboat – The Communications of the T_EX Users Group* 31.1, pp. 18–20. ISSN: 0896-3207.
- Miede, André, Gökhan Şimşek, Stefan Schulte, Daniel F. Abawi, Julian Eckert, and Ralf Steinmetz (2011). “Revealing Business Relationships – Eavesdropping Cross-organizational Collaboration in the Internet of Services.” In: *Proceedings of the Tenth International Conference Wirtschaftsinformatik (WI 2011)*. Vol. 2, pp. 1083–1092. ISBN: 978-1-4467-9236-0.
- Tsai, Hsin-Yi, Melanie Siebenhaar, André Miede, Yu-Lun Huang, and Ralf Steinmetz (2012). “Threat as a Service? Virtualization’s Impact on Cloud Security.” In: *IEEE IT Professional* 14.1, pp. 32–37. ISSN: 1520-9202.

Attention: This requires a separate run of bibtex for your refsection, e.g., `ClassicThesis1-btx` for this file. You might also use biber as the backend for biblatex. See also <http://tex.stackexchange.com/questions/128196/problem-with-refsection>.

*We have seen that computer programming is an art,
because it applies accumulated knowledge to the world,
because it requires skill and ingenuity, and especially
because it produces objects of beauty.*

— Donald E. Knuth ([Knuth, 1974](#))

ACKNOWLEDGMENTS

Put your acknowledgments here.

Many thanks to everybody who already sent me a postcard!

Regarding the typography and other help, many thanks go to Marco Kuhlmann, Philipp Lehman, Lothar Schlesier, Jim Young, Lorenzo Pantieri and Enrico Gregorio, Jörg Sommer, Joachim Köstler, Daniel Gottschlag, Denis Aydin, Paride Legovini, Steffen Prochnow, Nicolas Repp, Hinrich Harms, Roland Winkler, Jörg Weber, Henri Menke, Claus Lahiri, Clemens Niederberger, Stefano Bragaglia, Jörn Hees, Scott Lowe, Dave Howcroft, José M. Alcaide, David Carlisle, Ulrike Fischer, Hugues de Lassus, Csaba Hajdu, Dave Howcroft, Anonymous, Konrad Höffner, and the whole L^AT_EX-community for support, ideas and some great software.

Regarding L_YX: The L_YX port was initially done by *Nicholas Mariette* in March 2009 and continued by *Ivo Pletikosić* in 2011. Thank you very much for your work and for the contributions to the original style.

CONTENTS

I INTRODUCTION

1	MODELING LANGUAGE	3
1.1	A dialog between Shannon and Chomsky	3
1.2	The Transformer, the best machine language learner?	6
1.2.1	Attention	6
1.2.2	Multilayer Perceptron	8
1.2.3	A Transformer Is Never Large Enough	12
2	BACKGROUND ON EFFICIENT LANGUAGE MODELS	15
2.1	Model compression	16
2.1.1	Small Llama Models, or How to Account for Inference Optimality	18
2.1.2	Unstructured Pruning	19
2.1.3	Structured Pruning	20
2.1.4	The best training objective for compression	22
2.2	Our contribution to model compression	26
2.3	Activation Sparsity	27
2.3.1	Sparse Mixture of Experts (MoE)	28
2.3.2	Sparse Memory Layers	35
2.3.3	Embedding Memory Layers	38
2.3.4	Post-training Activation Sparsity	40
2.3.5	Matryoshka Learning	41
2.4	Our contribution to activation sparsity	43

II COMPRESSION OF TRANSFORMER MODELS

3	LILLAMA	47
3.1	Introduction	47
3.2	Motivation: Low-Rank Activations	49
3.3	Background: Low-Rank Approximation of Transformer Models	50
3.3.1	Low-Rank Approximation from Weights	50
3.3.2	Low-Rank Approximation from Features	51
3.4	Proposed Approach	52
3.4.1	Non-linear Feature Approximation.	52
3.4.2	Beyond Teacher-only Activations.	54
3.4.3	Module-wise Distillation.	54
3.5	Fast and Memory-Efficient Compression	55
3.5.1	Uniform Weights Compression	55
3.5.2	Top Layers First Compression	56
3.5.3	Bottom Layers First Compression	56
3.5.4	Ease of Implementation	57
3.6	Experiments on Transformers	57
3.6.1	Setup	57

3.6.2	Results	58
3.7	Other Architectures and Modalities	59
3.7.1	Mamba Architecture	59
3.7.2	Speech modality: Whisper	61
3.8	Analysis	62
3.8.1	Which Distillation Loss Should Be Used?	63
3.8.2	How Should the Rank Be Chosen?	65
3.8.3	Is SVD initialization necessary?	65
3.8.4	Cost.	65
3.8.5	Examples of generated texts	66
3.8.6	Comparisons.	67
3.8.7	Can this approach work for low-resource and under-represented languages?	67
3.9	Limitations	70
3.9.1	Compatibility with quantization:	70
3.9.2	Compromise between specificity and generality:	71
3.10	Conclusion	71
3.11	Open Questions	71
3.11.1	If the weights are full rank, why still compress them?	71
3.11.2	Can an LLM be pre-trained from scratch with low-rank weights?	71
4	BALDWHISPER	73
4.1	Introduction	73
4.2	BaldWhisper Approach	75
4.2.1	Stage 1: Layer Merging	76
4.2.2	Stage 2: Activation-Aware Embedding Decomposition	77
4.3	Experiments	77
4.3.1	Experimental Setup	77
4.3.2	Results	78
4.4	Analysis	78
4.5	Extreme Head Shearing of LLMs	80
4.6	Conclusion	81
III ACTIVATION SPARSITY OF TRANSFORMER MODELS		
5	CROSS-LINGUAL MRL	85
5.1	Introduction	85
5.2	Building a Wolof Speech Language Model	86
5.2.1	Background and Motivation	87
5.2.2	Wolof-Specific HuBERT Continued Pretraining	88
5.2.3	Supervised ASR Fine-Tuning and Evaluation	88
5.2.4	From a Speech Encoder to a Wolof Speech LLM	90
5.2.5	Implications for Cross-modal Retrieval	92
5.3	Datasets	92

5.3.1	Training Datasets	92
5.3.2	Evaluation Datasets	94
5.4	Modeling	94
5.4.1	Text-only Embeddings	95
5.4.2	Late Modality Fusion	95
5.4.3	Dual Modality	96
5.5	Experiments	96
5.5.1	Setup	96
5.5.2	Document Retrieval	96
5.5.3	Speech Keyword Spotting	97
5.5.4	Generalization to an Unseen Task	98
5.6	Analysis	99
5.6.1	Speech Quality in FLEURS	100
5.6.2	Prompting	100
5.6.3	Rank of Matryoshka Dimensions	101
5.6.4	Deployment Costs	101
5.7	Limitations	102
5.8	Conclusion	102
6	ADAPTIVE NEURAL ACTIVITY TRANSFORMER	103
6.1	Introduction	103
6.2	Contemplating the MLP	104
6.2.1	Transformer MLPs as Key-Value Memories	104
6.2.2	The Effective Dimensionality of the MLP	105
6.3	Learning to Select Key-Value Pairs Adaptively	107
6.3.1	ZeroNeuron Sparse Activation Gating	108
6.3.2	Structured Activation Sparsity of the MLP	109
6.4	Experiments	111
6.4.1	Experimental Setup	111
6.4.2	Results	113
6.5	Generalization to Embedding Language Models	116
6.6	Conclusion	117
IV CONCLUSION AND FUTURE WORK		
7	OVERALL CONCLUSION AND FUTURE WORK	121
7.1	Overall Conclusion	121
7.2	Future Work	122
V APPENDIX		
A	APPENDIX TEST	127
A.1	Appendix Section Test	127
A.2	Another Appendix Section Test	127
BIBLIOGRAPHY 129		

LIST OF FIGURES

Figure 1.1	Decoder-only Transformer architecture.	6
Figure 1.2	Share of model storage occupied by embedding parameters across different model sizes. Data taken from https://blog.squeezebits.com/vocabulary-trimming-methods .	11
Figure 1.3	Effect of reducing the LM Head rank on accuracy and cross-entropy. Figures from (Godey et al., 2024).	12
Figure 1.4	Examples of emergent abilities in large language models. Figure from (Wei et al., 2022). An ability is emergent when its score surpasses the random baseline after remaining near that baseline at smaller scales.	13
Figure 2.1	Fixed-token scaling view of Chinchilla and Llama 3.1 models. Adapted from https://deepmind.google/blog/an-empirical-analysis-of-compute-optimal-large-language-model-training/	19
Figure 2.2	Illustration of unstructured pruning. The architecture is unchanged, but individual weights are removed independently.	19
Figure 2.3	Convergence of Sheared LLaMA after structured pruning and continued pre-training, compared to other models of similar sizes. Plot taken from the original paper (Xia et al., 2024)	22
Figure 2.4	Illustration of structured pruning. A full layer is removed, together with all its incoming and outgoing weights.	23
Figure 2.5	Sequence-level distillation. The teacher first generates a complete answer, and the student is then trained on this synthetic sequence.	23
Figure 2.6	Off-policy distillation. The student and the teacher are evaluated on the same data, and the student distribution is trained to match the teacher distribution.	24
Figure 2.7	On-policy distillation. The student first samples an output, then the teacher evaluates this student-generated output.	26

Figure 2.8	Switch Transformer encoder block, where the dense feed-forward layer is replaced by a sparse Switch FFN layer. Figure adapted from (Fedus et al., 2022). 28
Figure 2.9	Scaling properties of Switch Transformer. Increasing the number of experts improves performance and sample efficiency at equal computational budget. Figure from (Fedus et al., 2022). 29
Figure 2.10	Effect of MoE granularity on loss for fixed model size and number of training tokens. Figure adapted from (Krajewski et al., 2024). 31
Figure 2.11	Expert selection proportions in Mixtral across domains and layers. The dashed line corresponds to uniform routing over the eight experts. Figure adapted from (Jiang et al., 2024). 33
Figure 2.12	Cosine similarity between languages based on expert-choice patterns in NLLB-200. Figure adapted from (NLLB Team et al., 2022). 34
Figure 2.13	LongCat-Flash architecture with zero-computation experts. The router still selects K experts, but some selected experts are identity paths, which makes the effective compute adaptive. Figure from (Meituan LongCat Team et al., 2025). 36
Figure 2.14	Overview of a key-value memory layer. The input is projected into a query, compared to the memory keys, and only the values associated with the selected keys are used for computation. Figure from (Lample et al., 2019). 36
Figure 2.15	Architecture of an n -gram embedding layer. The token embedding is augmented with hashed n -gram embedding branches. Figure from (Liu et al., 2026b). 40
Figure 2.16	Matryoshka Representation Learning trains a representation to be useful at several embedding dimensions. Figure adapted from (Kusupati et al., 2022). 42
Figure 2.17	MatFormer nests several FFN sizes inside the Transformer block and trains the corresponding submodels jointly. Figure from (Devvrit et al., 2024). 42

Figure 3.1	Lillama approach: STEP 1 selects layers to compress for a target compression ratio (e.g., N%) using various strategies (see Section 3.5). STEP 2 compresses and initializes the chosen parameters via SVD. STEP 3 distills the low-rank weights with a small calibration dataset. 47
Figure 3.2	Activations are low-rank. Comparison of the stable rank of weights (*_proj) and layer activations (layer_activations). Each bar is the average stable rank across all layers. 51
Figure 3.3	Accelerating convergence through a joint loss 53
Figure 3.4	Illustration of 20% compression when using the three strategies: bottom, top, and uniform. Green boxes are compressed layers with low-rank matrices while gray boxes are non-compressed ones. $l_1, l_2, \dots, l_{n-1}, l_n$ are the layer indices. 55
Figure 3.5	Example of a PyTorch implementation of our approach using the Teacher-only distillation strategy. 57
Figure 3.6	Higher ranks generally give better results. Mean accuracy over our test benchmarks as a function of the minimum rank k in Algorithm 1 63
Figure 3.7	SVD initialization improves sample efficiency. Convergence when initializing low-rank weights randomly (without SVD) or with SVD. 64
Figure 3.8	The joint loss generally converges better. Convergence of the three losses illustrated in Figure 3.3, evaluated using perplexity on the Wiki-text2 test corpus during distillation. 64
Figure 4.1	Layer merging in the Whisper decoder: each student layer is a weighted average of two consecutive teacher layers, then trained with cross-entropy and knowledge distillation. 74
Figure 4.2	Head shearing of the Whisper embedding through low-rank decomposition. The decomposition is activation-aware because the low-rank weights are subsequently feature-distilled. 76
Figure 4.3	Choice of α and β . Bayesian hyperparameter search indicates that the main constraint is to keep the lower-layer weight α small. 79
Figure 4.4	Activation similarities between all decoder-layer pairs. Consecutive layers tend to be more similar, suggesting that they can be merged. 80

Figure 4.5	Training loss of Qwen2.5-0.5B on the smol-smoltalk corpus, comparing a 75% reduction in embedding parameters with the uncompressed baseline. 81
Figure 5.1	Wolof speech collection and filtering 87
Figure 5.2	Wolof Speech LLM architecture 90
Figure 5.3	Speech data processing pipeline 93
Figure 5.4	Late-fusion and dual architectures. Late fusion projects downsampled HuBERT features into the LLM embedding space and concatenates them with prompt embeddings. The dual architecture pools speech features and uses dimension-specific projections. 95
Figure 5.5	Dimensions needed for an energy ratio 99
Figure 5.6	Dimensions needed for full energy. 99
Figure 5.7	Recall by embedding dimension and number of examples. 100
Figure 5.8	Indexing, storage, query-time, and accuracy trade-offs across dimensions. 101
Figure 6.1	Effective intermediate dimension across models 107
Figure 6.2	Effective dimension and relative expert width 107
Figure 6.3	Training dynamics of the adaptive ZoN gate. 113

LIST OF TABLES

Table 1.1	Average SuperGLUE score for gated MLP variants. 9
Table 1.2	Examples of MLP key-value memory pairs identified in prior work (Geva et al., 2021). For each value vector $\mathbf{v}_i^\ell$, <i>Top prediction</i> is the vocabulary token receiving the highest score when the value is projected through the model’s output embedding. <i>Precision@50</i> is the percentage of the 50 training prefixes that most strongly activate the paired key $\mathbf{k}_i^\ell$ whose ground-truth next token equals that prediction. The final column presents one such matching prefix, with its ground-truth next token shown in blue. 10
Table 2.1	Main efficiency metrics for LLM training and inference. Arrows indicate the optimization direction. 17

Table 2.2	Model sizes prescribed by different scaling laws for a fixed training-data budget of 16.55T tokens. 18
Table 2.3	Vocabulary specialization in the seventh layer of OLMoE-1B-7B. Table adapted from (Muenighoff et al., 2025). 35
Table 2.4	Effect of n -gram vocabulary scaling in Qwen3.8-Flash-Next. Vocabulary scales are measured relative to the base tokenizer vocabulary size. Larger vocabularies add parameters. Table adapted from (Qwen Team, 2026a). 40
Table 3.1	Positioning of our approach within the compression literature. No Custom Kernel indicates that a custom GPU kernel is not required to observe speedup and No Ampere-only indicates that speedup is not restricted to Ampere GPUs. Calib. Data is the total number of tokens in billions used for compression. 48
Table 3.2	Benchmarks and corresponding metrics that were used to evaluate the models. 59
Table 3.3	Zero-shot evaluation results for the base non-compressed models (0%), and the 20%, 25%, and 30% compressed models. 60
Table 3.4	Compressed models save memory and speed up computation. Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and varying the sequence length from 512 to 16384 tokens. All the models were loaded in bf16 and used Flash-Attention2. OOM stands for out of memory. 60
Table 3.5	40% compressed Phi-2 3B competes with models of similar size while being compressed using only 13M tokens. We compress Phi-2 3B to 1.7B (by removing 40% of the parameters) and compare its zero-shot performance to that of other recent language models of similar sizes. 61
Table 3.6	Compressed models recover well when fine-tuned. Zero-shot performance of the base non-compressed models (0%) and the 40% compressed models, without and with fine-tuning (FT). Models were fine-tuned on the whole Slim-Orca dataset (191 million tokens). 61

Table 3.7	Mamba-based language models can be efficiently compressed. Zero-shot evaluation results for the non-compressed (0%) and for the compressed Mamba-based language models (20%). 61
Table 3.8	Compressing Mamba models saves memory and speeds up computation. Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All models were loaded in bf16 and used Flash-Attention2. 62
Table 3.9	The 37% compressed Whisper is faster and 1.46× lighter. The WER is computed on the English test subset of the FLEURS dataset. The speed (tokens/second) corresponds to the transcription of 400 tokens from a dataset containing 2,048 speech examples, using a batch size of 128. A single A100 GPU is used and the model is loaded in FP32, without Flash-Attention. We also give the GPU Memory (VRAM) occupied by each model. 62
Table 3.10	The joint loss generally performs better: Zero-shot performance of the three losses using the same setup for each model, as described in Section 3.6. 63
Table 3.11	Comparing the Wikitext perplexity convergence of the three losses for 2M training tokens. 65
Table 3.12	Compression times for the three compression strategies at 20% parameter reduction on a single A100 GPU. For the bottom-first and top-first compression strategies, we used the same hyperparameters as in Section 3.6. 66
Table 3.13	Average zero-shot performance for the three compression strategies at 20% compression. For the bottom-first and top-first compression strategies, we used the same hyperparameters as in Section 3.6. 67
Table 3.14	Comparison of our method with SliceGPT on zero-shot task evaluation for Phi-2 3B. We give the evaluation scores before compression (0%), both as reported by SliceGPT and as reproduced by us. Both methods use the Alpaca calibration dataset for compression. 67

Table 3.15	Steve Jobs Story by the 20% and 40% compressed Phi-2 3B models, without <i>any</i> fine-tuning. Prompt: "Here the story of Steve Jobs:"	68
Table 3.16	The Civil War Story by the 20% and 40% compressed models, without <i>any</i> fine-tuning. Prompt: "What is the cause of the Civil War? Here is the story."	69
Table 3.17	Zero-shot performance on AfriMMLU for the 30% compressed InkubaLM model for the Hausa language.	70
Table 4.1	Main results. Accuracy and efficiency after layer merging and head shearing (embedding decomposition). Speed is measured on a MacBook Air M1 while decoding 256 tokens for each utterance in a batch of 10. Bambara's less standardized orthography may partly explain the high absolute WER.	78
Table 4.2	Extreme head shearing of Gemma. Scores are percentages (higher is better). We report normalized accuracy for HellaSwag, PIQA, and ARC-Challenge, and accuracy for BoolQ and WinoGrande.	81
Table 5.1	Hours of supervised Wolof speech used for ASR fine-tuning and evaluation.	89
Table 5.2	ASR results after identical supervised fine-tuning. The arrow denotes English pretraining followed by Wolof continued pretraining.	89
Table 5.3	ASR results of the Wolof Speech LLM.	91
Table 5.4	Speech-to-text translation results of the Wolof Speech LLM.	91
Table 5.5	Kallaama retrieval results	97
Table 5.6	Document-retrieval nDCG on Fleurs-Retrieval-Eval.	97
Table 5.7	Keyword-spotting performance across embedding dimensions.	98
Table 5.8	Few-shot speech intent detection at dimension 1024.	98
Table 6.1	We check possible sample size biases for Qwen3-30B-A3B, since MoE layers receive different numbers of tokens. We examine the correlation between the number of routed tokens per expert and the energy ratio $R@0.99$ and find none.	106
Table 6.2	Model architecture and training budgets. Dense-600M uses a narrower MLP; the other architecture settings are shared.	112

Table 6.3	Benchmark accuracy (%) for the dense and sparse models. CORE reports the mean centered score (%). BB denotes BIG-bench. 114
Table 6.4	Evaluation of Dense-1.2B (Dense) and ZoN-1.2B-A600M (Sparse). All entries are percentages; CORE is the mean centered score. %0 measures the fraction of zeros predicted by ZoN MLP gates during inference. BB denotes BIG-bench. 115
Table 6.5	Performance of NV-Embed representations on classification and retrieval tasks. Scores are percentages. k is the number of coordinates retained at inference; dense representations use all d_{model} coordinates. The dense baseline uses NV-Embed with a projector fine-tuned on each task and retains all coordinates. The best score in each column is bold and the second-highest distinct score is underlined. 116
Table A.1	Autem usu id 127

LISTINGS

Listing 2.1	Pseudo-code implementation of a PEER layer forward pass. 39
Listing 6.1	Simplified implementation of the ZoN MLP training forward pass. 109
Listing A.1	A floating example (listings manual) 127

ACRONYMS

Part I

INTRODUCTION

MODELING LANGUAGE

1.1 A DIALOG BETWEEN SHANNON AND CHOMSKY

It is curious that we still don't know where the expression *language model* comes from or when it became established. We mean the *expression* itself, rather than the concept. The concept of a language model is quite old, and one of the earliest formulations closely related to what we now call language modeling can be found in Shannon's work on information theory (Shannon, 1948)¹. Shannon starts from a simple observation: sequences of linguistic symbols are not random, but exhibit regularities characteristic of the language.

"[...] the messages to be transmitted consist of sequences of letters. These sequences, however, are not completely random. In general, they form sentences and have the statistical structure of, say, English."

— Claude E. Shannon, *A Mathematical Theory of Communication*
(1948)

¹ This intuition was already explored by Markov in 1913 (Markov, 2007); Shannon later provided a more fully developed formalization.

If English sentences have a statistical structure, then a statistical model that captures this structure should be able to generate sequences that look like English. But what is this statistical model? How can we estimate it? Later, Shannon gives a more operational view:

"We can think of a discrete source as generating the message, symbol by symbol. It will choose successive symbols according to certain probabilities depending, in general, on preceding choices [...]."

— Claude E. Shannon, *A Mathematical Theory of Communication*
(1948)

Shannon thus describes a statistical model of English but does not call it a *language model*. The story takes a different turn in the early 1970s. In 1974, a team at IBM working on speech recognition described a statistical model closely related to Shannon's formulation and explicitly called it a *language model*. In their probabilistic decomposition of speech recognition, the language model is simply the component that assigns probabilities to possible word sequences—that is, a model of which utterances are more or less likely in a given language. As they put it, "it is the function of the language model (LM) to provide us with $\Pr\{W \mid L\}$," where W denotes a word sequence and L the language. This work is therefore probably one of the earliest uses of the expression *language model* in essentially its modern statistical sense:

a probability distribution over linguistic sequences estimated from language data.

Calling these systems language models could suggest that they are linguistic models of a language. But does a better statistical model of English mean a better model of the language itself? The idea that statistical learning can capture linguistic structure is compatible with usage-based and connectionist approaches. Nativist approaches, however, question whether statistical learning alone is sufficient to explain linguistic knowledge. In *Three Models for the Description of Language*, Chomsky states (Chomsky, 1956, p. 113):

“We find that no finite-state Markov process that produces symbols with transition from state to state can serve as an English grammar. [...] such processes that produce n-order statistical approximations to English do not come closer, with increasing n, to matching the output of an English grammar.”

— Noam Chomsky, *Three Models for the Description of Language*
(1956)

There are two main claims made by Chomsky in this paper. The first claim is structural and concerns the **bounded memory** of finite-state models. For finite-order Markov language models, this limitation manifests as a fixed context length: only a bounded portion of the preceding sequence can determine the next symbol. English, however, permits recursive constructions whose dependencies can require information from an arbitrarily large preceding structure. Modern language models have greatly increased the amount of preceding context they can exploit, making them much better at modeling long-range dependencies. Yet practical models still operate with bounded context and memory, so the distinction highlighted by Chomsky between unbounded linguistic structure and bounded computational memory remains relevant. Second, even if we consider a statistical model with unbounded memory, Chomsky’s criticism goes beyond model capacity. His broader claim is that **language is innate**, biologically programmed in the human brain (Chomsky, 1965) and cannot be learned simply by accumulating statistics over observed linguistic data. The main argument is the *poverty of stimulus*: a child learns their native language rapidly, despite being exposed to limited, noisy, and incomplete examples. According to this account, infants have structural biases that help them learn and generalize linguistic structures from a few examples. Chomsky’s theory holds that human linguistic competence depends on internal constraints that cannot be inferred from experience alone.

It is not straightforward, however, to translate this notion of innateness into the language of machine learning. Does Chomsky’s argument imply that general-purpose learning architectures are inherently incapable of acquiring language from data alone? Or can we instead design computational architectures and general learning objectives whose

inductive biases make the acquisition of linguistic structure easier? ² Maybe we can also reverse the question: can learning from experience alone help uncover the structure latent in language? Even though modern machine learning algorithms are still far from matching the data efficiency of human children, substantial progress has been made by designing learning architectures that optimize the language modeling objective, improving the linguistic competence of these systems.

The language modeling objective, as Shannon describes it, is to learn to predict the next symbol from a fixed number of previous symbols. These symbols can be letters or words, but nowadays they are usually tokens, units that can form a word or a subword. More formally, the probability of picking a token s_i given a sequence of previous tokens $s_{<i} = (s_1, s_2, \dots, s_{i-1})$ is written as $P(s_i | s_{<i})$. Using the chain rule, these individual probabilities can be multiplied together to give the probability of the whole text:

$$P(s) = p(s_1) \times p(s_2|s_1) \times p(s_3|s_1, s_2) \times \dots \times p(s_t|s_{<t}) = \prod_{i=1}^t P(s_i | s_{<i}). \quad (1.1)$$

Each individual probability can be defined from a score over the vocabulary:

$$p(s_i | s_{<i}) = \frac{\text{score}(s_{<i}, s_i)}{\sum_{v \in \mathcal{V}} \text{score}(s_{<i}, v)}. \quad (1.2)$$

Here, $\text{score}_\theta(x, y)$ measures how strongly the context x is associated with the next token y , and $\mathcal{V}$ is the token vocabulary. The function $\text{score}(x, y)$ can be designed very freely but in neural language models, it is often the exponential of the dot product between the context vector of x and the token vector y .

Using maximum likelihood estimation, we can estimate sequence probabilities with a statistical model parameterized by θ and trained on a large corpus $\mathcal{C}$ of texts:

$$\theta^* = \arg \max_{\theta} \sum_{s \in \mathcal{C}} \sum_{i=1}^{|s|} \log P_\theta(s_i | s_{<i}). \quad (1.3)$$

This is the learning objective of language modeling; it remains to design a learning algorithm that can optimize it effectively. While many neural architectures have been used for this purpose, ranging from CNNs to RNNs, one architecture has proven particularly effective: the Transformer. We next present the Transformer through the lens of linguistic and knowledge acquisition, and discuss how scaling this architecture leads to further gains in knowledge and linguistic competence.

² For Chomsky, this question is not really interesting, as reproducing linguistic behavior with an artificial architecture is not, by itself, an explanation of human language acquisition:
<https://www.youtube.com/watch?v=PZQOIuuQiJA>

1.2 THE TRANSFORMER, THE BEST MACHINE LANGUAGE LEARNER?

Earlier work improved neural machine translation by adding attention to an RNN-based encoder–decoder (Bahdanau et al., 2015). The Transformer replaced recurrence with attention (Vaswani et al., 2017). Removing recurrent dependencies allows all token positions within a layer to be processed in parallel during training, making the architecture more parallelizable and easier to scale. Figure 1.1 presents the decoder-only Transformer architecture used for autoregressive language modeling. The model stacks N layers, each containing a self-attention module followed by an MLP, before an output head computes the next-token probability distribution. We next discuss these three components and their roles in acquiring knowledge and generalizing linguistic structures.

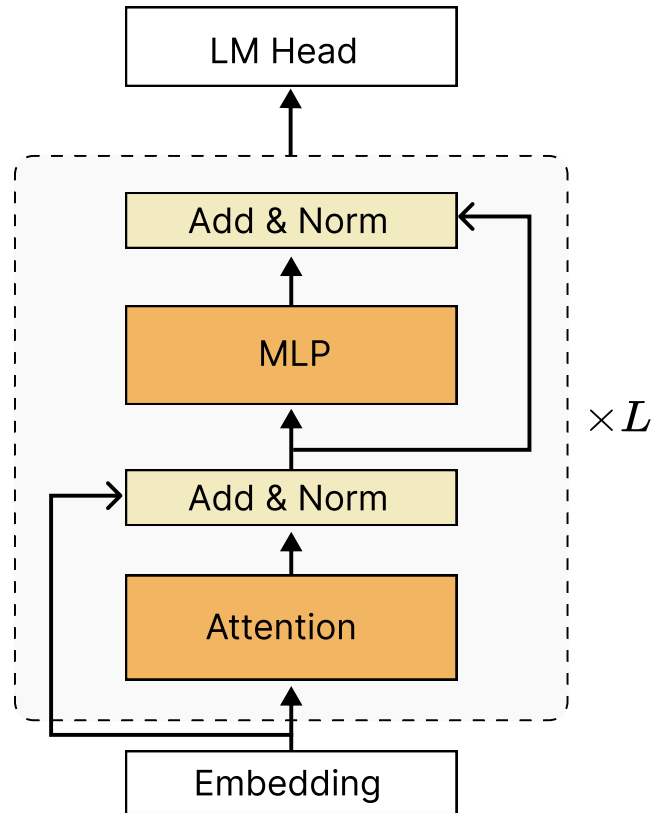


Figure 1.1: Decoder-only Transformer architecture.

1.2.1 Attention

The innovation of the Transformer is to remove the RNN and replace it entirely with **Multi-Head Attention (MHA)**. Given $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$,

the sequence of token embeddings for T positions, MHA projects the sequence into queries, keys and values for H heads:

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times H \times d_{\text{head}}}. \quad (1.4)$$

Equivalently, each head h has its own projections:

$$\mathbf{Q}_h = \mathbf{XW}_h^Q, \quad \mathbf{K}_h = \mathbf{XW}_h^K, \quad \mathbf{V}_h = \mathbf{XW}_h^V, \quad (1.5)$$

where $\mathbf{W}_h^Q \in \mathbb{R}^{d_{\text{model}} \times d_q}$, $\mathbf{W}_h^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $\mathbf{W}_h^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are learned projection matrices for head h . The head output is then computed with scaled dot-product self-attention:

$$\text{head}_h = \text{softmax} \left(\frac{\mathbf{Q}_h \mathbf{K}_h^\top + \mathbf{M}}{\sqrt{d_k}} \right) \mathbf{V}_h. \quad (1.6)$$

For a decoder language model, $\mathbf{M}$ is a causal mask: entries corresponding to future positions are set to $-\infty$, so token i can only attend to tokens $j \leq i$. Multi-head attention runs H such heads in parallel and concatenates their outputs:

$$\text{MHA}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) \mathbf{W}^O, \quad (1.7)$$

with $\mathbf{W}^O \in \mathbb{R}^{Hd_v \times d_{\text{model}}}$. Each head can therefore learn a different projection of the same context.

Having multiple independent heads is beneficial since it has been shown that **different attention heads play linguistically interpretable roles**. Some specialize in positional relations, syntactic dependencies, coreference, or other linguistically interpretable patterns (Clark et al., 2019b; Voita et al., 2019). However, attention heads are also substantially redundant, as many of them can be removed with little or no degradation in model performance (Michel et al., 2019; Voita et al., 2019; Kovaleva et al., 2019).

In standard MHA, as introduced in the original Transformer paper, the projection matrices $\mathbf{W}_h^Q$, $\mathbf{W}_h^K$ and $\mathbf{W}_h^V$ are usually chosen so that all heads together project back to a space of size d_{model} . However, these projections can become a bottleneck during generation. In a naive implementation, at each decoding step, all previous tokens would be projected again to recompute their keys and values. The **KV-Cache** reduces this computation and speeds up generation by caching the keys and values of previous tokens. However, this cache grows linearly with the sequence length, and it is easy to run out of memory when generating very long texts. To reduce the KV-Cache, we need to reduce the dimensionality of keys and values. Among the popular approaches to reduce the KV-Cache, **Grouped-Query Attention (GQA)** shares key-value pairs across multiple query heads. Intuitively, the keys and values have fewer heads than the queries:

$$\mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times h_g \times h_{\text{dim}}}, \quad (1.8)$$

$$\mathbf{Q} \in \mathbb{R}^{T \times h_q \times h_{\text{dim}}}, \quad h_g < h_q. \quad (1.9)$$

The key-value heads are then broadcast to the query heads, and attention is computed as usual:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top) \mathbf{V}. \quad (1.10)$$

Standard MHA corresponds to no sharing, while multi-query attention is the extreme case where all query heads share the same keys and values. GQA is the compromise between the two: it reduces the KV-Cache while keeping several different key-value heads. While GQA reduces KV-cache memory, it does not reduce the attention computation. Nowadays, GQA is often combined with **Sparse Attention**, such as DeepSeek Sparse Attention (DeepSeek-AI, Liu, et al., 2025), to reduce this compute by allowing each query to attend to only a subset of previous tokens.

1.2.2 Multilayer Perceptron

After the attention layer, the contextualized tokens are fed into the multilayer perceptron (MLP). While attention mixes information across tokens, the MLP mixes information across channels: each token is processed independently. In the original Transformer paper, the MLP is defined as:

$$\text{MLP}(x) = f(\mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}}$$

where $f(\cdot)$ is the non-linear activation function, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{inter}}}$ projects the input token $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ into a higher-dimensional space, with $d_{\text{model}} \ll d_{\text{inter}}$. Finally, $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{inter}} \times d_{\text{model}}}$, projects back to the model dimension. Modern LLMs often replace this simple MLP with a gated linear unit variant. Instead of applying only one non-linearity to the expanded representation, the block learns two projections: one projection $\mathbf{V}_{\text{up}}$ produces the values, and the other projection $\mathbf{W}_{\text{up}}$ gates them. Omitting bias terms, these alternatives can be written as:

$$\text{MLP}_{\text{GLU}}(\mathbf{x}) = (\sigma(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.11)$$

$$\text{MLP}_{\text{GEGLU}}(\mathbf{x}) = (\text{GELU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.12)$$

$$\text{MLP}_{\text{Bilinear}}(\mathbf{x}) = (\mathbf{x}\mathbf{W}_{\text{up}} \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.13)$$

$$\text{MLP}_{\text{SiLU}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}, \quad (1.14)$$

$$\text{MLP}_{\text{ReLU}}(\mathbf{x}) = (\text{ReLU}(\mathbf{x}\mathbf{W}_{\text{up}}) \odot \mathbf{x}\mathbf{V}_{\text{up}}) \mathbf{W}_{\text{down}}. \quad (1.15)$$

Here, $\odot$ denotes element-wise multiplication. Table 1.1 gives the performance of each activation on the SuperGLUE benchmark. The SiLU-based variant gives better results, which helps explain why many LLMs, including Mistral-v0.1 and Granite, use it. It is difficult to explain why gated MLPs work better with some activations than others. As the author of the study puts it (Shazeer, 2020):

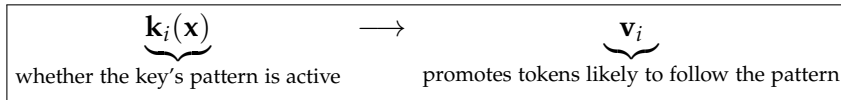
“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.”

Average SuperGLUE score	
MLP _{GLU}	73.95
MLP _{GEGLU}	73.96
MLP _{Bilinear}	73.81
MLP _{SwiGLU}	74.56
MLP _{ReGLU}	73.66

Table 1.1: Average SuperGLUE score for gated MLP variants.

— Noam Shazeer, *GLU Variants Improve Transformer* (2020)

In modern LLM architectures, MLP parameters can account for over 90% of the total model parameters. It has been shown that MLP layers play an important role in storing knowledge acquired during web-scale pretraining. One interpretation treats the MLP as a key–value memory (Geva et al., 2021): each column of $\mathbf{W}_{\text{up}}$ in the MLP represents a key $\mathbf{k}_i$ that detects linguistic patterns seen during training, while each row of $\mathbf{W}_{\text{down}}$ represents the corresponding value $\mathbf{v}_i$, which promotes the tokens likely to follow those patterns. The key $\mathbf{k}_i$ can be strongly or weakly activated depending on the input x . When $\mathbf{k}_i$ is strongly activated, its corresponding value $\mathbf{v}_i$ is assigned a large weight and therefore contributes more strongly to the MLP output. This mechanism can be schematized as follows:



The MLP does not only store lexical knowledge. In some cases, this lexical knowledge coincides with factual knowledge. When a key detecting the prefix *The capital of France is* is strongly associated with a value representing *Paris*, this lexical key–value memory turns out to also encode world knowledge. Hence, more broadly, it is widely believed that MLPs play a central role in storing the knowledge acquired during pretraining (Geva et al., 2021; Dai et al., 2022). This view is also reflected in many knowledge-editing methods, which often directly target MLP weights (Meng et al., 2023a; Meng et al., 2023b; Fang et al., 2025). If we view the MLP as a key–value memory, it is reasonable to expect that each input token requires access to only a subset of its key–value pairs. We will later discuss how this intuition can guide the design of an adaptive sparse memory layer.

1.2.2.1 Language Modeling Head

Well, the tokens are contextualized thanks to the multi-head attention, and each token retrieves the relevant knowledge encoded within the

Value	Top prediction	Precision@50	Matching trigger example
$\mathbf{v}_{222}^{15}$	<i>each</i>	68%	<i>But when bees and wasps resemble each</i>
$\mathbf{v}_{752}^{16}$	<i>played</i>	16%	<i>Her first role was in Vijay Lalwani's psychological thriller Karthik Calling Karthik, where Padukone was cast as the supportive girlfriend of a depressed man (played</i>
$\mathbf{v}_{2601}^{13}$	<i>extratropical</i>	4%	<i>Most of the winter precipitation is the result of synoptic-scale, low-pressure weather systems (large-scale storms such as extratropical</i>
$\mathbf{v}_{881}^{15}$	<i>part</i>	92%	<i>Comet served only briefly with the fleet, owing in large part</i>
$\mathbf{v}_{2070}^{16}$	<i>line</i>	84%	<i>Sailing from Lorient in October 1805 with one ship of the line</i>
$\mathbf{v}_{3186}^{12}$	<i>jail</i>	4%	<i>On May 11, 2011, four days after scoring 6 touchdowns for the Slaughter, Grady was sentenced to twenty days in jail</i>

Table 1.2: Examples of MLP key–value memory pairs identified in prior work (Geva et al., 2021). For each value vector $\mathbf{v}_i^\ell$, *Top prediction* is the vocabulary token receiving the highest score when the value is projected through the model’s output embedding. *Precision@50* is the percentage of the 50 training prefixes that most strongly activate the paired key $\mathbf{k}_i^\ell$ whose ground-truth next token equals that prediction. The final column presents one such matching prefix, with its ground-truth next token shown in blue.

MLP. This process is repeated for L layers and at the end produces a context vector $\mathbf{c}_{<i}$ that is ready for next-token prediction using the language modeling head (LM Head or output embedding):

$$p(s_i | s_{<i}) = \frac{\exp(\mathbf{c}_{<i} \mathbf{W}_i)}{\sum_{v \in \mathcal{V}} \exp(\mathbf{c}_{<i} \mathbf{W}_v)}. \quad (1.16)$$

where $\mathbf{W} \in \mathbb{R}^{d \times |\mathcal{V}|}$ is the LM Head weight matrix. The LM head weight matrix is another important component of large-scale Transformer LLMs. It is closely linked to the tokenizer: the vocabulary size determines the number of parameters in the output projection, while the quality of tokenization affects how easily the model learns and, ultimately, its performance. Recent tokenizers use large vocabularies to capture the diversity of tokens in web-scale data. Hence, the LM

head can account for a large share of the parameters in an LLM, particularly in small models. Figure 1.2 shows the memory occupied by the input/output embedding parameters for different model sizes, including dense and quantized ones. We can see that the embedding and LM Head parameters represent a larger fraction of the model when the Transformer backbone is small³. This is mainly caused by

³ Quantization is usually not applied to the embedding weights.

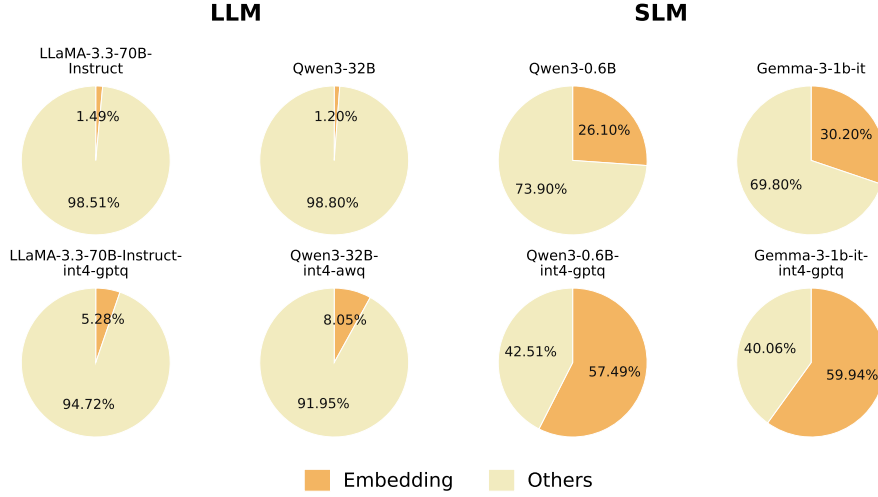


Figure 1.2: Share of model storage occupied by embedding parameters across different model sizes. Data taken from <https://blog.squeezebits.com/vocabulary-trimming-methods>.

the large vocabulary of the models. A common solution to reduce the size by half is weight tying (Press and Wolf, 2017), which shares the input and output embedding matrices. Almost all small LLMs use weight tying, which can substantially reduce the model’s parameter count without hurting performance.

Another approach to reducing the number of parameters occupied by the LM head is to factorize its weight matrix into two low-rank matrices. This strategy was notably adopted in ALBERT (Lan et al., 2020), where the embedding dimension was reduced from 768 to 128 without sacrificing downstream performance. More precisely, given the input $\mathbf{x} \in \mathbb{R}^d$ and an LM-head weight matrix $\mathbf{W} \in \mathbb{R}^{d \times V}$, the factorized projection is

$$\mathbf{xW} \longrightarrow (\mathbf{xW}_1)\mathbf{W}_2, \quad (1.17)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times r}$, $\mathbf{W}_2 \in \mathbb{R}^{r \times V}$, and $r \ll H$. This reduces the LM-head parameter count from dV to $dr + rV$.

However, this bottleneck architecture can hurt optimization. Some works have identified what is known as the *softmax bottleneck* (Yang et al., 2018): a low-dimensional LM head can only produce a limited variety of output distributions. Therefore, reducing its rank too aggressively can limit the model’s ability to predict complex token distributions. A study of LM heads with different ranks found that

reducing the rank hurts performance and limits the expressiveness of the output distribution (Godey et al., 2024), as shown in Figure 1.3. More importantly, this also hurts the learning signal during training (Godey and Artzi, 2026). There are many solutions to avoid the *softmax bottleneck*. Since this phenomenon arises mainly when $r \ll V$, an obvious solution is to increase r or reduce the vocabulary size V . Reducing the vocabulary size can be achieved using a tokenizer with fewer tokens or falling back to a character- or byte-level tokenizer. It is also technically possible after training to remove tokens from the input and output embeddings. But this is risky as it can affect critical tokens. This discussion is related to the research we will present later, which proposes to reduce the rank of the LM Head instead of pruning tokens in multilingual transformer models.

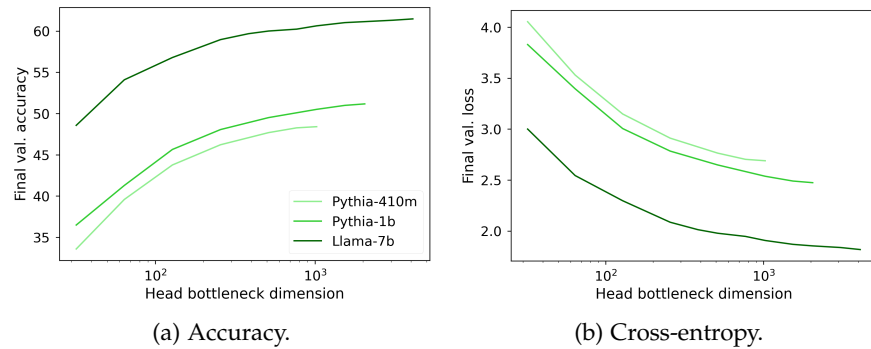


Figure 1.3: Effect of reducing the LM Head rank on accuracy and cross-entropy. Figures from (Godey et al., 2024).

1.2.3 A Transformer Is Never Large Enough

What made the Transformer architecture particularly successful in language modeling is its ability to scale both the number of parameters and the amount of training data. Since Transformer training can be efficiently parallelized across multiple GPUs, increasingly large models can be trained on increasingly large datasets, leading to improved performance and the emergence of new capabilities. However, scaling cannot be done blindly: given the high cost of pretraining, the allocation between model size and training data must be estimated before the final training run. For a fixed compute budget, how large should the model be, and on how many tokens should it be trained?

Denoting the number of parameters by N , the number of training tokens by D , and the training compute by C , the compute cost of a dense Transformer can be approximated as $C \approx 6ND$. Given a fixed training budget C (the number of floating-point operations available for training), how should we allocate compute between model size and training tokens? Both Kaplan (Kaplan et al., 2020) and Chinchilla

(Hoffmann et al., 2022) scaling laws indicate that the compute-optimal model size and dataset size follow power laws in the compute budget: $N_{\text{opt}} \propto C^\alpha$ and $D_{\text{opt}} \propto C^\beta$. **Kaplan scaling laws** estimate the compute-optimal allocation as $N_{\text{opt}} \propto C^{0.73}$ and $D_{\text{opt}} \propto C^{0.27}$. Thus, model size should increase much faster than training data, which favors very large models trained on comparatively few tokens. On the other hand, the **Chinchilla scaling laws** later found that model size and training data should instead scale approximately equally: $N_{\text{opt}} \propto C^{0.5}$ and $D_{\text{opt}} \propto C^{0.5}$.

Beyond guiding compute allocation, scaling laws also reveal how model capabilities evolve with scale. While performance often improves predictably as training compute increases, some abilities appear only after the model reaches a certain scale. The model may remain close to random performance across several scales before suddenly crossing a plateau and acquiring a new ability. Several such *emergent abilities* have been identified (Wei et al., 2022), as shown in Figure 1.4. We can see that many linguistic skills such as language

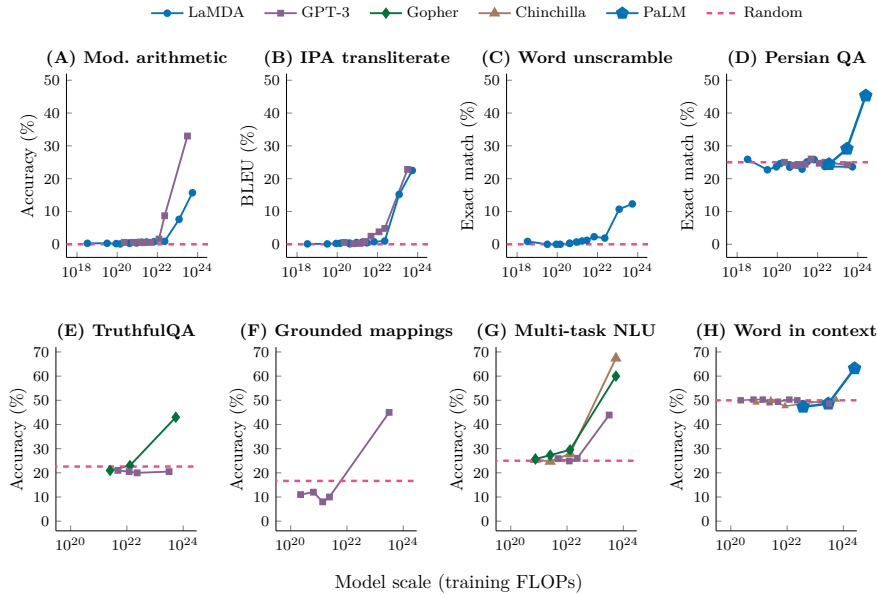


Figure 1.4: Examples of emergent abilities in large language models. Figure from (Wei et al., 2022). An ability is emergent when its score surpasses the random baseline after remaining near that baseline at smaller scales.

understanding, especially for low-resource languages, only show up at a given model scale. Then a question arises. Is there a point beyond which increasing the number of parameters no longer produces meaningful improvements? Testing this hypothesis directly is difficult, since training models at the largest scales is extremely costly. Classical scaling laws predict *diminishing returns*: test loss continues to decrease with scale, but each additional increase in model size produces a smaller improvement (Kaplan et al., 2020; Hoffmann et al.,

2022). However, whether this implies diminishing returns in model capabilities depends on how performance is measured. It has been argued that short-task benchmarks may create an illusion of saturation (Sinha et al., 2026). Even small improvements in single-step accuracy can compound into large improvements in the length of tasks that a model can reliably execute. Therefore, diminishing returns on loss or short-task accuracy do not necessarily imply diminishing returns in practical capabilities.

BACKGROUND ON EFFICIENT LANGUAGE MODELS

We’ve discussed Transformers and how scaling them yields substantial improvements on natural language tasks. However, scaling a Transformer-based language model is costly in terms of both training and inference compute. In this chapter, we discuss LLM efficiency mainly through the lens of **model compression** and **activation sparsity**, since our contributions concern these two aspects. We present common approaches for scaling LLMs efficiently during training and serving them efficiently during inference.

When talking about *efficiency* in LLMs, there are some important concepts to know. The first distinction is between **compute-boundness** and **memory-boundness**. A model or a task is compute-bound when the bottleneck is the number of arithmetic operations that can be done per second. In that case, the limitation is not only the theoretical peak compute of the hardware, but also how well the model and the implementation can use this compute. A model or a task is memory-bound when the bottleneck is instead the movement of data: loading weights, reading activations, writing tensors, or moving the KV-cache from the GPU memory. Depending on whether we are training, doing prefill, or decoding tokens, these two concepts do not have the same consequences. This distinction explains why diffusion-based LLMs such as LLaDA or Gemma-Diffusion are much faster than autoregressive LLMs. Diffusion LLMs are compute-bound and more FLOP-intensive than autoregressive LLMs. However, their parallel decoding reduces data movement relative to autoregressive decoding and leads to faster generation.

LLM training is mostly compute-bound, because training is highly parallel. Many tokens are processed at once, and the forward and backward passes are dominated by large matrix multiplications. So the key metrics to check are **FLOPS** (how many floating-point operations are executed per second), **MFU** (how much of the theoretical peak compute of the hardware is really used), and **Tokens/s** (the number of tokens processed per second). The goal is to make the model perform a lot of useful operations per second while minimizing communication: between GPUs, between CPU and GPU, and between HBM and the compute units of the GPU. This is why training efficiency is not only about reducing the number of parameters. It is also about keeping the hardware busy. We will later see that small dense models, sparse models, and compressed models do not all use the hardware in the same way. FLOPS and MFU are hardware-oriented numbers, so they

are often complemented by **Tokens/s**, which measures how quickly the model consumes the training data.

LLM inference can be compute-bound, but is more often memory-bound because tokens are generated one at a time. At each decoding step, the model has to read its weights and the KV-cache to produce only one new token, which creates a lot of memory traffic for a relatively small amount of computation. There are two phases in LLM inference. The first is the **prefill**, where the system prompt, the user prompt and the context are fed in parallel to the LLM. This phase is closer to training and is often compute-bound. The second phase is **decoding**, where the model generates the text token by token. This is usually the bottleneck, because the model repeatedly accesses the same weights and an increasingly large KV-cache. This difference is important because a model that requires more FLOPS can sometimes be faster if it uses the hardware better or moves less data. The key metrics for inference follow these two phases. For prefill, we usually look at **Time To First Token (TTFT)**, which measures how long the user waits before seeing the first generated token, and **prefill Tokens/s**, which measures how fast the prompt is processed. These two metrics are related, but they are not exact inverses because TTFT also includes overheads and the generation of the first token. For decoding, we look at **Time Per Output Token (TPOT)**, which measures the time needed to generate one new token, and **generated Tokens/s**, which measures how many new tokens are produced per second. These two metrics describe the same decoding speed with inverse units. Finally, we also care about the **memory footprint**, especially the weights and KV-cache memory.

In the next sections, we detail approaches for improving LLM efficiency on these key metrics during both inference and training. This part is heavily inspired by the blog post we wrote in collaboration with the Datacraft team in 2024: <https://medium.com/@yayasy-nlp/a-comprehensive-overview-of-large-language-models-compression-methods-8b59dd0ff4f8>

2.1 MODEL COMPRESSION

An approach to reduce the memory footprint, and sometimes also accelerate inference by reducing the FLOPS, is model compression: making the model smaller while trying to preserve most of its performance. There is a tension here. Scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022) usually advocate for more training compute, and often this means increasing the number of parameters. Inference efficiency, in contrast, advocates for less compute, less memory movement, and a smaller memory footprint.

We present two ways of thinking about this tension. First, we discuss small LLMs such as Llama (Touvron et al., 2023a; Touvron et al., 2023b;

Phase	Metric	What it measures
Training	FLOPS $\uparrow$	Number of floating-point operations per second.
	MFU $\uparrow$	Fraction of the theoretical hardware compute that is effectively used.
	Tokens/s $\uparrow$	Number of training tokens processed per second.
Inference - Prefill	TTFT $\downarrow$	Time before the first generated token is returned to the user.
	Prefill Tokens/s $\uparrow$	Speed at which the input prompt and context are processed.
Inference - Decoding	TPOT $\downarrow$	Time needed to generate one new token.
	Generated Tokens/s $\uparrow$	Number of new tokens generated per second. It is the reciprocal of TPOT.
Training and inference	Memory footprint $\downarrow$	Memory occupied by weights, activations, optimizer states during training, and the KV-cache during inference.

Table 2.1: Main efficiency metrics for LLM training and inference. Arrows indicate the optimization direction.

(Grattafiori et al., 2024)), where the objective is not only to be compute-optimal during training, but also to account for inference-optimality. The idea is to train smaller models much longer than what the training-optimal rule would suggest, so that they remain small at inference time while still performing well. We then present the approach of “train large, then compress” (Li et al., 2020), which consists of training a large model first, then compressing it by keeping the most important components of the model, or by distilling the knowledge of this large model into smaller ones.

Table 2.2: Model sizes prescribed by different scaling laws for a fixed training-data budget of 16.55T tokens.

Training tokens	Kaplan N_{opt}	Chinchilla N_{opt}	Llama 3 N_{opt}
16.55T	$\sim 100,000\text{T}$	$\sim 827.5\text{B}$	402B

2.1.1 Small Llama Models, or How to Account for Inference Optimality

The Llama series of models (Touvron et al., 2023a; Touvron et al., 2023b; Grattafiori et al., 2024) was among the first to strongly push the idea that training should also account for inference compute. Instead of only following what Chinchilla compute-optimality suggests (Hoffmann et al., 2022), they train small models on many more tokens. The result is a model that is not training-optimal according to Chinchilla, but is much more efficient at inference time: it is smaller, cheaper to run, and still very performant. This point is important for open models. A smaller but strong model can be deployed more easily by independent users, especially those who do not have access to large compute resources. The release of Llama 1 (Touvron et al., 2023a) and Llama 2 (Touvron et al., 2023b) in 2023 enabled the emergence of an ecosystem of open-source tools for local LLM usage with projects such as Llama-cpp, Llama-index, and Ollama becoming central for running and using LLMs outside large GPU clusters. Llama 3 (Grattafiori et al., 2024) pushed this idea further by training models such as the 8B model on 16.55 trillion tokens. In this work, the authors develop their own scaling laws, whose estimated compute allocation between model size and training tokens favors training smaller models on more tokens. Table 2.2 presents the estimated number of parameters that should be trained for a fixed budget of 16.55 trillion tokens by Kaplan, Chinchilla and Llama scaling laws.

How can we explain these differences in compute-optimal allocation estimates? The history of scaling laws is also a history of scaling the compute used to study them and improving the methods used to fit the experimental results. The resulting scaling laws may also differ depending on the hyperparameters, model architecture, and data quality. This explains why the Llama 3 scaling-law estimates differ from those of Kaplan and Chinchilla. By the time Llama 3 was released, the field had evolved considerably in terms of data quality, model design, and optimization techniques for large-scale LLM training.

More importantly, the Llama 3 work advocates training small models much longer, beyond what is compute-optimal. Figure 2.1 compares Llama 3 8B and Llama 3 70B with the Chinchilla compute-optimal estimate. We can see that the Llama 3 models are trained beyond what is compute-optimal. While this is costly at training time, the resulting small models are stronger and more efficient to serve for inference.

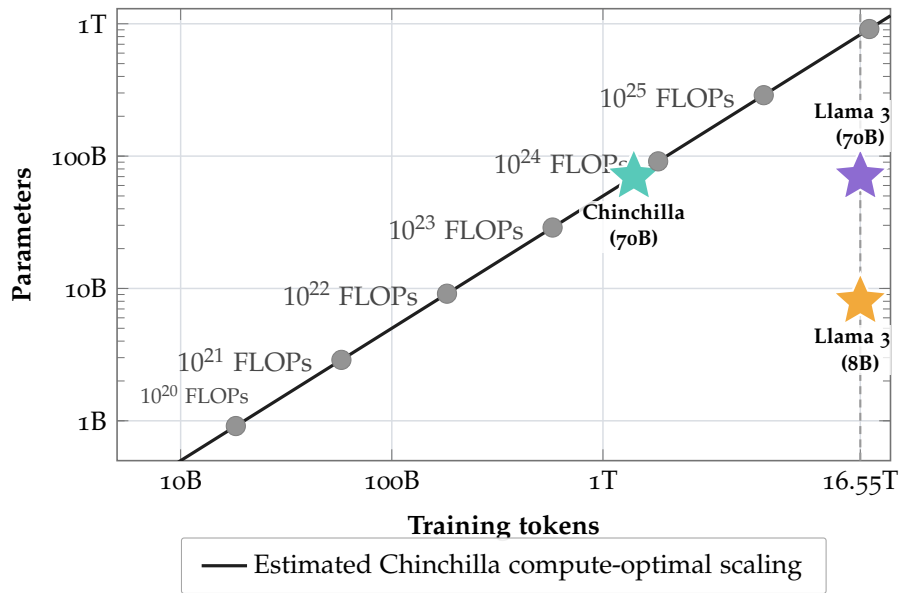


Figure 2.1: Fixed-token scaling view of Chinchilla and Llama 3.1 models. Adapted from <https://deepmind.google/blog/an-empirical-analysis-of-compute-optimal-large-language-model-training/>

2.1.2 Unstructured Pruning

Instead of training small models for longer, another approach to produce small models is to train large models first and then compress them or distill them into efficient architectures. Compressing larger and more powerful pre-trained models is a viable alternative. This can be achieved using many complementary methods: pruning, distillation, quantization, and more.

The intuition behind pruning is to identify unimportant weights in the network and remove them when possible or set them to zero. There are two main pruning methods: **unstructured pruning** and **structured pruning**.

2.1.2.1 Unstructured Pruning

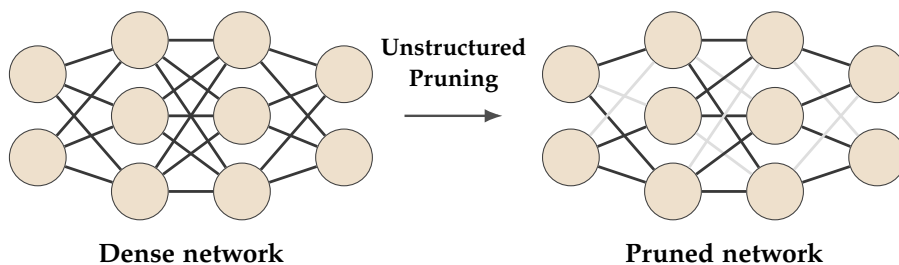


Figure 2.2: Illustration of unstructured pruning. The architecture is unchanged, but individual weights are removed independently.

Unstructured pruning identifies and removes individual unimportant weights from the model. Figure 2.2 illustrates this process, where some model weights are individually removed, producing a sparse model. It may be more accurate to call these approaches **weight sparsity**, because setting weights to zero does not directly reduce the model footprint. The sparse weights can only reduce the model’s memory footprint when they are stored using a specific sparse format. However, this format must also store the positions of the non-zero weights and therefore usually becomes beneficial only at high sparsity levels. In pretrained LLMs, achieving such high sparsity without significantly degrading performance remains difficult. Similarly, unstructured sparsity can provide speed-ups only at high sparsity levels and with specialized GPU kernels. Another approach is **semi-structured sparsity**, which constrains a *pattern* in the positions of the removed weights. For example, 2:4 sparsity removes exactly two weights from every group of four. This regular pattern facilitates the development of GPU kernels that can accelerate computation in hardware.

SparseGPT (Frantar and Alistarh, 2023a) and Wanda (Sun et al., 2024a) are two popular post-training methods that support both unstructured and semi-structured sparsity. SparseGPT uses activations collected from a small calibration dataset to estimate the effect of removing each weight. After pruning a weight, the remaining weights are adjusted to compensate for the resulting reconstruction error. Wanda is much simpler and cheaper: it removes weights that are both small and connected to weakly activated features. For each weight W_{ij} , Wanda computes the importance score $S_{ij} = |W_{ij}| \|\mathbf{X}_j\|_2$, where $|W_{ij}|$ is the magnitude of the weight and $\|\mathbf{X}_j\|_2$ is the ℓ_2 norm of its corresponding input feature over the calibration data. Weights with lower scores are removed. Beyond SparseGPT and Wanda, many studies have proposed new unstructured and semi-structured algorithms. These include improved weight-importance criteria, such as Plug-and-Play (Zhang et al., 2024c), non-uniform sparsity allocation across layers, such as OWL (Yin et al., 2024), or methods that iteratively refine the initial pruning mask, such as DSnoT (Zhang et al., 2024d).

2.1.3 Structured Pruning

Unstructured pruning works well, but the resulting models require specific implementations to accelerate inference or reduce memory requirements. To easily achieve acceleration and memory savings out of the box, it is better to shrink components of the model. This is the purpose of **Structured Pruning** which is illustrated in Figure 2.4. In structured pruning, entire substructures (or blocks) are removed directly, for example, the attention heads, rows or columns of the matrices, or even whole layers. Since entire substructures are pruned, Structured pruning imposes more constraints compared to unstruc-

tured pruning or semi-structured pruning where individual weights are removed more independently and freely. This explains why unstructured pruning usually performs better than structured pruning. However, structured pruning has greater potential to accelerate inference and reduce the memory footprint.

LLM-Pruner (Ma et al., 2023) uses gradient information to estimate group importance and removes the least important groups. The method can remove approximately 20% of the parameters of LLaMA-7B while retaining 89.8% of its original performance without post-training. However, more aggressive parameter reduction generally leads to a significant performance drop. Therefore, larger reductions often require continued training. This is illustrated by **Sheared Llama** (Xia et al., 2024), which learns binary masks over layers, attention heads, MLP dimensions, and hidden dimensions. Since binary masks are not directly differentiable, the method uses a continuous reparameterization to learn which structures should be removed. The authors prune LLaMA2-7B into models containing 2.7B and 1.3B parameters, and continue their training on 50B tokens using the language modeling loss. The pruned model converges quickly as shown in Figure 2.3, which shows the speed of convergence of the pruned model compared with training from scratch. Hence, pruning larger models and then continuing pretraining can be a better way to create small models than training them from scratch. Although the resulting models do not retain the full performance of LLaMA2-7B, they are competitive with models of similar sizes trained on much larger datasets. The Minitron (Muralidharan et al., 2024; Sreenivas et al., 2024b) approach similarly combines structured pruning with knowledge-distillation-based retraining. It removes complete layers or reduces the hidden, attention, and MLP dimensions of a pretrained model. The pruned model is then retrained on much more data than in Sheared-Llama, over 380 billion tokens for the pruning of Mistral NeMo 12B.

Some LLM releases have also experimented with pruning to produce smaller models from larger ones. For example, the Llama 3.2 1B and 3B models were obtained through one-shot structured pruning of Llama 3.1 8B.¹ The pruned models were then trained using knowledge distillation, with the logits of Llama 3.1 8B and 70B used as training targets. Meanwhile, Ministral 3 (Liu et al., 2026a) uses cascaded pruning and distillation. Starting from Mistral Small 3.1 24B, the authors successively produce a smaller model at each stage, which is trained using the logits of the original model. This “prune–distill–repeat” procedure is similar to Minitron.

There are some approaches that focus specifically on layer pruning, based on the observation that several Transformer layers are redundant (Men et al., 2024; Yang et al., 2024b). For example, **ShortGPT** (Men et al., 2024) measures the contribution of each layer using the cosine

¹ <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>

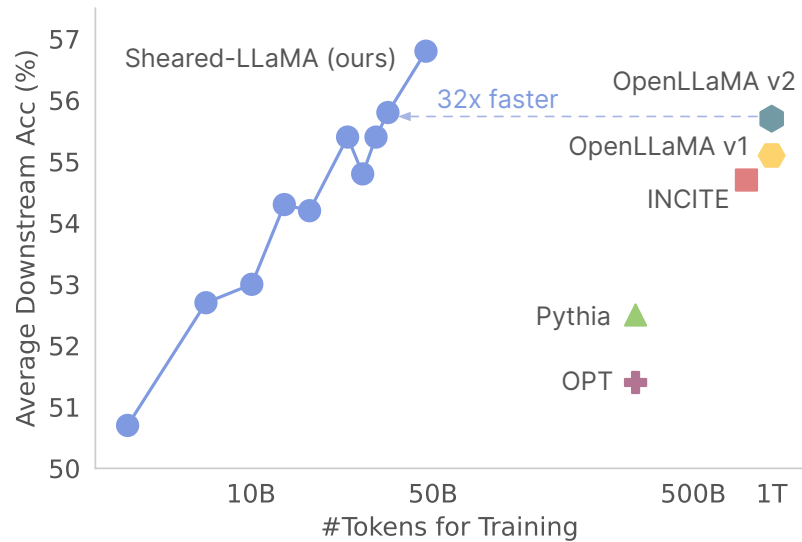


Figure 2.3: Convergence of Sheared LLaMA after structured pruning and continued pre-training, compared to other models of similar sizes. Plot taken from the original paper (Xia et al., 2024)

similarity between its input and output activations. The intuition is that an unimportant layer does not substantially modify its input representation. Therefore, layers whose input and output activations are highly similar can be removed. However, layer pruning generally incurs a significant performance drop, and the resulting model often requires additional training to recover. It is therefore interesting to ask whether, in decoder-only LLMs, high layer similarity is a misleading signal for pruning. Information flows progressively through the network, and each layer may add only a small but important update through the residual connection. Although these updates are individually small, they accumulate across dozens of layers. Consequently, a high similarity between the input and output of a layer does not necessarily mean that the layer is useless. Removing several such layers can disrupt this accumulation and degrade performance unless a recovery stage is applied. Later evaluations of one-shot layer pruning have notably found that it can preserve performance on simple or multiple-choice benchmarks while substantially harming open-ended generation and long-chain reasoning capabilities (Wang et al., 2025a; He et al., 2026).²

² https://www.reddit.com/r/MachineLearning/comments/1bc1638/r_shortgpt_layers_in_large_language_models_are/

2.1.4 The best training objective for compression

We mentioned distillation as a way to compress the capacity of large models into small models, but there are many other objectives. What are the best training objectives for compressing LLMs and training small LLMs? The common training objective for compression is maximum likelihood estimation, but small LLMs take time to learn. Hence,

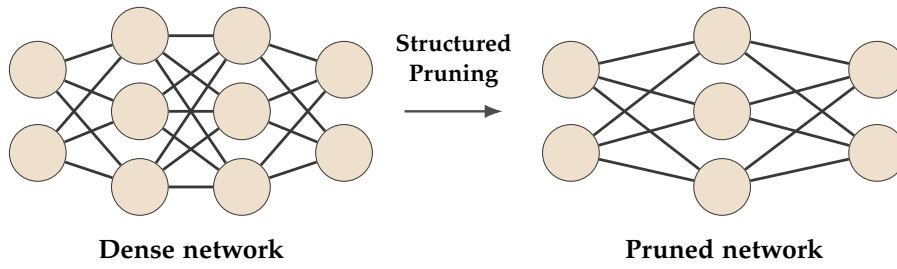


Figure 2.4: Illustration of structured pruning. A full layer is removed, together with all its incoming and outgoing weights.

it has been proposed to use training objectives that give more *learning signal* to the small LLM. Let’s list them and see how they’re used in recent LLM releases.

2.1.4.1 Sequence-Level Distillation

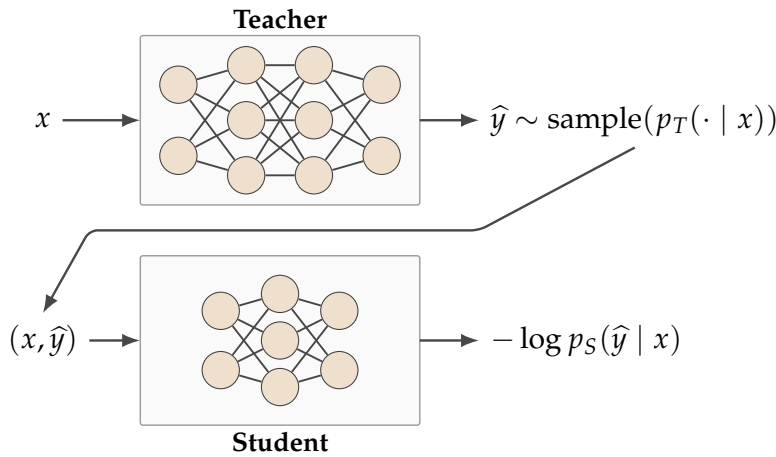


Figure 2.5: Sequence-level distillation. The teacher first generates a complete answer, and the student is then trained on this synthetic sequence.

In sequence-level distillation, the small model is trained on texts generated by a larger and more capable model. This approach is illustrated in Figure 2.5. The concept was introduced for machine translation (Kim and Rush, 2016), where the translations generated by the teacher are used as targets for the student. In the context of LLMs, this approach became popular with Alpaca (Taori et al., 2023). Alpaca trained a small LLaMA model on 52K instruction–response pairs generated by the closed-source text-davinci-003 model. However, generating useful synthetic data is not that simple. The generated answers must be correct, diverse, sufficiently difficult, and representative of the capabilities that we want to model. The data must also be filtered to remove redundant, low-quality, or unsafe examples. This motivated the development of many methods for automating and improving synthetic-data generation. Orca (Mukherjee et al., 2023), for

example, uses GPT-4 to generate detailed explanations and reasoning traces rather than only short answers. Magpie (Xu et al., 2025), on the other hand, generates both instructions and responses from a chat LLM without requiring human-written seed instructions. Persona Hub (Ge et al., 2024) improves data diversity by conditioning the generation on a large collection of synthetic personas representing different backgrounds, occupations, and perspectives.

There has also been a debate about whether a student trained on teacher-generated data can outperform its teacher. This appears possible on specific tasks: for example, a much smaller model can outperform a teacher prompted with a few examples by learning from both its answers and its rationales (Hsieh et al., 2023). However, this does not mean that the student becomes generally more capable than the teacher. Such improvements often require generating multiple candidate responses and retaining only the best ones through correctness verification, rejection sampling (Yuan et al., 2023), or learned quality and reward functions.

2.1.4.2 Off-policy Knowledge Distillation

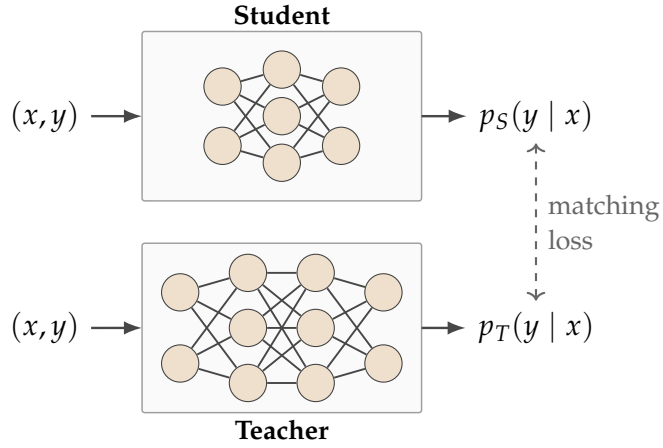


Figure 2.6: Off-policy distillation. The student and the teacher are evaluated on the same data, and the student distribution is trained to match the teacher distribution.

In sequence-level distillation, the training signal provided by each token is limited, as the student is trained only on the token sampled by the teacher. When the synthetic dataset is sufficiently large, the student can nevertheless approximate the teacher’s probability distribution underlying the generated texts. A more direct approach to mimicking the teacher’s probability distribution is **logit-based knowledge distillation** (Hinton et al., 2015). This approach provides a richer training signal since the student learns from the teacher’s probability distribution over the vocabulary at every token position, rather than from only the generated token. This process is illustrated in Figure 2.6. The data

are passed through both the student and the teacher, which produce a probability distribution over the vocabulary at each token position of the gold answer. The student distribution is then compared with the teacher distribution on the same data. The loss can be designed in different ways, but the most common choice is the *Kullback–Leibler divergence*, which encourages the student distribution to match the teacher distribution.

Logit-based distillation has been used to train several open-weight small language models. In particular, Gemma 2 and Gemma 3 were trained by distilling larger teacher models (Gemma Team, 2024; al., 2025). And as discussed earlier, knowledge distillation is used as a recovery stage after pruning in small model series such as Llama 3.2 and Ministral 3. Logit-based knowledge distillation usually outperforms training on hard targets alone as the teacher’s probability distribution provides a richer training signal for every token (Gemma Team, 2024; Anshumann et al., 2025). For example, the Gemma 2 ablations show that distillation improves both validation perplexity and downstream performance across several student sizes. This is particularly true in low-data regimes (Liu et al., 2026c). Hence logit-based distillation seems to be more sample-efficient.

2.1.4.3 On-policy Knowledge Distillation

In sequence-level and off-policy knowledge distillation, the student remains passive: it is trained on sequences generated by the teacher or on fixed sequences for which the teacher provides feedback. This creates what is known as the *training–inference mismatch*. During training, the student is trained using *teacher forcing*, meaning that the prediction of the i -th token is conditioned on the previous tokens from the teacher-generated or gold sequence. At inference time, however, the student must generate its own tokens and condition subsequent predictions on its previous outputs. It may therefore encounter contexts resulting from its own mistakes that were never observed during training. **On-policy distillation** addresses this mismatch by allowing the student to generate sequences during training and using the teacher to provide feedback on the states visited by the student. Prior work shows that knowledge distillation is more effective when the student learns from feedback on its own generated sequences (Agarwal et al., 2024). This method outperforms distillation based only on fixed teacher-generated data.

Figure 2.7 illustrates the process of on-policy distillation. The student takes a prompt and generates an answer, which is then evaluated by the teacher. More precisely, the teacher provides a probability distribution at each token position of the student-generated sequence. The training loss can be designed in different ways, using, for example, the forward or reverse Kullback–Leibler divergence or the Jensen–Shannon divergence. On-policy distillation can be understood through

the lens of **reinforcement learning**: the student’s generations can be seen as actions, while the teacher provides the reward on those actions. Since the reward is differentiable, training does not necessarily require policy-gradient algorithms such as REINFORCE. The student can instead be trained directly by minimizing the token-level divergence from the teacher distribution. Subsequent studies have extended on-policy distillation beyond the compression of small models and used it as a post-training framework. Recently, **On-Policy Self-Distillation** has emerged as a new post-training approach (Zhao et al., 2026). In this setting, the student and the teacher are the same model but receive different information. The teacher is given *privileged information*, such as the gold answer or a verified reasoning trace, whereas the student receives only the original prompt. The student generates its own response, and the privileged version of the model provides dense token-level feedback on this response.

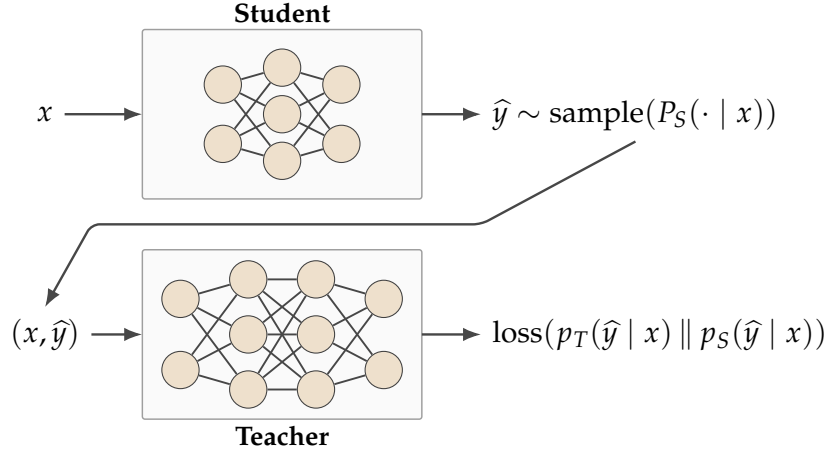


Figure 2.7: On-policy distillation. The student first samples an output, then the teacher evaluates this student-generated output.

2.2 OUR CONTRIBUTION TO MODEL COMPRESSION

Our contributions address two limitations of post-training compression for pre-trained Transformers:

1. We study low-rank decomposition for compressing pretrained LLMs and propose **Lillama**. The method combines low-rank weight approximation with layer-wise distillation, and it explicitly addresses the **training-inference mismatch** discussed so far by making the distillation more **on-policy**: each student layer is trained on activations produced by preceding student layers, rather than on teacher-only features. Compared with unstructured or semi-structured pruning methods such as **SparseGPT** (Frantar and Alistarh, 2023a) and **Wanda** (Sun et al., 2024a), which rely on sparse weight formats and specialized

kernels to achieve speedup, Lillama shrinks the parameters of the LLM and naturally saves computation. It also differs from structured pruning pipelines such as **Sheared LLaMA** (Xia et al., 2024) and **Minitron** (Muralidharan et al., 2024; Sreenivas et al., 2024b), which require more expensive retraining stages after pruning. In contrast, Lillama keeps the compression local and one-shot: it compresses only selected layers, initializes them with SVD, and distills them with a small calibration set while preserving the dense execution path and the original model structure.

2. Post-training compression often leaves the LM head untouched because compressing it can distort the output probability distribution and force a costly recovery stage. We propose **BaldWhisper**, showing that low-rank compression of the head together with feature distillation offers a good speed–accuracy trade-off despite the **softmax bottleneck**. We also study **layer merging** as an alternative to layer pruning, reducing decoder depth to lower self-attention and cross-attention costs during autoregressive decoding, which is **memory-bound**. This places our work in the same compression regime as low-rank approximations, distillation, and structured pruning, but targets a part of the model that is commonly left untouched in standard post-training compression pipelines.

2.3 ACTIVATION SPARSITY

While post-training compression of LLMs has proved to be an effective approach to create small LLMs, larger LLMs nevertheless remain more performant. They generalize better, contain more knowledge, and are much better at in-context learning. However, as discussed, they are also more costly to train and to serve for inference. Can we get some of the benefits of large models, but at a lower training and inference cost? Sparse activation is one answer to this question. The model dynamically selects which parameters should be used for the current input token. The selected parameters are called **active parameters**, while the others remain inactive for this specific input. When associated with efficient GPU kernels, sparsity reduces the activation memory, so it gives more room for larger batches and hence higher training and inference **throughput**. It can also accelerate the end-to-end decoding speed.

Sparse activations make it possible to partly decouple model size from computation, and provide new scaling axes compared to dense models:

1. **Sparsity scaling axis:** keeping the number of active parameters per input fixed, while increasing the total number of model parameters.
2. **Computation scaling axis:** keeping the total number of parameters fixed, while increasing the computation by activating more parameters per input.
3. **Granularity scaling axis:** increasing the number of experts while keeping the number of activated parameters and the model parameters fixed.

Training sparse models is difficult because the process of selecting which parameters to activate is often discrete and therefore not directly differentiable. However, the know-how has evolved a lot, especially for sparse mixtures of experts, which are now the most successful class of sparse models at scale. We will present different classes of sparse models, their pros and cons, and particularly the optimization tricks that make their training possible.

2.3.1 Sparse Mixture of Experts (MoE)

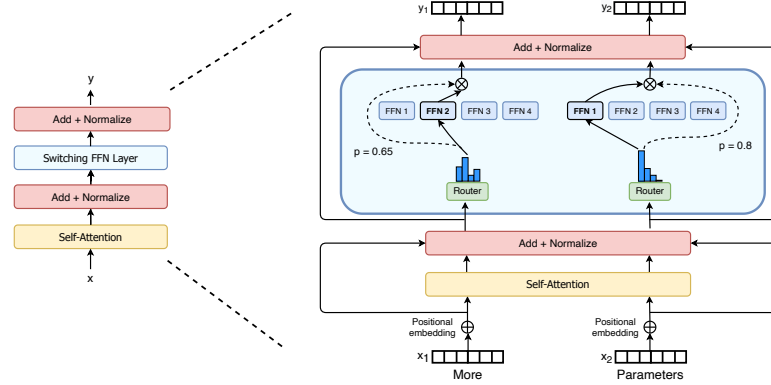


Figure 2.8: Switch Transformer encoder block, where the dense feed-forward layer is replaced by a sparse Switch FFN layer. Figure adapted from (Fedus et al., 2022).

2.3.1.1 The MoE layer

In a sparse MoE, the weight matrix $\mathbf{W}$ is split into e submatrices called experts, $\mathbf{W}_1, \dots, \mathbf{W}_e$. The model is trained to decide which experts are useful for the current input $\mathbf{x}$, and evaluates only these experts. Thus, the model can contain many parameters, while each token only activates a small fraction of them. The selection of the experts is performed by a router network R_θ and the top- k operator. More

formally, let $\mathbf{x} \in \mathbb{R}^{d_1}$ be the input, and let each expert $\mathbf{W}_j \in \mathbb{R}^{d_1 \times d_e}$. The router produces one logit, or score, per expert, $\mathbf{r} = R_\theta(\mathbf{x}) \in \mathbb{R}^e$. The top- k operator is first applied to these router logits to select the subset of active experts, $S = \{j \mid r_j \in \text{TopK}(\mathbf{r}, k)\}$. Then, the routing weights are computed with a softmax only over these selected experts. Thus, for each selected expert $j \in S$, we have $g_j = \exp(r_j) / \sum_{\ell \in S} \exp(r_\ell)$. The output of a sparse MoE layer is then the weighted sum of these selected experts:

$$\mathbf{o} = \text{MoE}(\mathbf{x}) = \sum_{j \in S} g_j \mathbf{W}_j^\top \mathbf{x}. \quad (2.1)$$

The output of the MoE layer is then projected back to the model dimension d_{model} .

2.3.1.2 The nice scaling properties of MoEs

MoEs give a way to increase the capacity of the model without increasing the computation by the same factor. Switch Transformer (Fedus et al., 2022) shows that at a FLOP-matched budget, increasing the number of experts improves the overall performance. This means increasing the model parameters while keeping the same number of activated parameters is beneficial. Figure 2.9 illustrates this. On the left plot, we can see that keeping the number of activated parameters fixed, while increasing the number of experts, leads to substantial improvements. However, it doesn't seem linear, and at some point increasing the number of parameters without increasing the number of activated parameters stops being beneficial. And since bigger models converge faster, large MoEs also accelerate training as shown by the right plot.

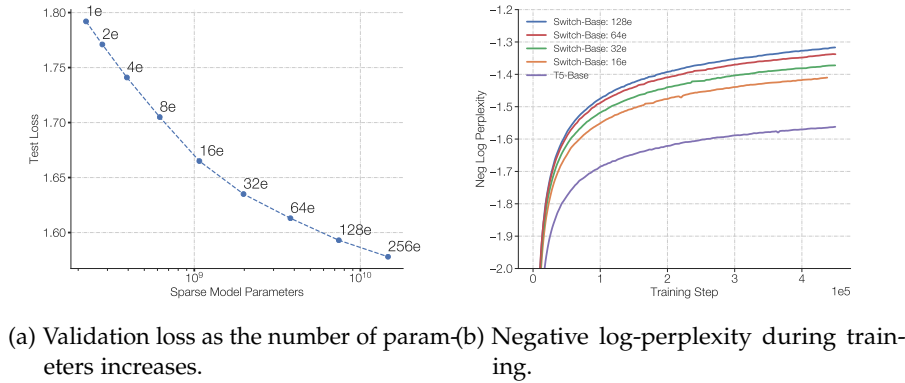


Figure 2.9: Scaling properties of Switch Transformer. Increasing the number of experts improves performance and sample efficiency at equal computational budget. Figure from (Fedus et al., 2022).

Sparsity has been pushed much further in recent sparse MoEs. The DeepSeek series particularly pushed fine-grained experts and very

sparse activation. DeepSeek-V3 uses 256 routed experts and activates only 8 of them for each token, which represents approximately 97% sparsity (DeepSeek-AI, 2024b). DeepSeek-V4-Pro pushes this even further: it has 384 routed experts and activates only 6 per token, so more than 98% of the routed experts are inactive at each step (DeepSeek-AI, 2026). Since MoEs decouple parameter scaling from computation, recent models therefore scale to multi-trillion-parameter sparse MoEs, such as Kimi-K3 with 2.8T parameters and 104B active parameters (Kimi Team, 2026), or Qwen3.8-Max with 2.4T parameters and 95B active parameters (Qwen Team, 2026b).

Beyond the scaling axis between active parameters and total parameters, MoEs also introduce another important axis: **granularity**, namely how many experts are available. This is different from sparsity. For example, with 50% sparsity, one can activate 1 expert out of 2, or 1024 experts out of 2048. Both designs activate the same fraction of experts, and can have the same total number of parameters, but the second design is much more fine-grained. It gives the router many more possible combinations of experts. In (Krajewski et al., 2024), the authors study this effect while keeping the model size and the number of training tokens fixed. As illustrated in Figure 2.10, their main result is that increasing granularity usually improves loss, probably because it gives the router more flexibility to combine experts. The model can represent a more precise mixture of skills.

2.3.1.3 Stabilizing MoEs

Scaling large, sparse, and fine-grained MoEs produces strong, efficient models, but also makes training challenging. This is true in terms of infrastructure: more experts create communication overhead between nodes and GPUs, as well as uneven GPU memory use when some experts are used more than others. It is also true in terms of optimization dynamics: TopK-based routing is not differentiable, which complicates optimization and requires many tricks to stabilize training.

LOAD BALANCING. The router must learn to distribute tokens across experts. If most tokens are routed to the same few experts, these experts become overloaded, while the other experts receive almost no training. Their hosting GPUs also receive more work than the others, which unbalances the GPU cluster. Switch Transformer avoids this by adding what is called a **load balancing** auxiliary loss, a loss added to the language modeling loss to encourage more uniform token routing at the batch level. There are several load balancing approaches. DeepSeek-V2 uses three types of auxiliary load balancing losses (DeepSeek-AI, 2024a):

- *Expert-level load balance*: encourages tokens to be distributed more uniformly across experts to avoid router collapse, so that no expert is overloaded and no expert remains undertrained.

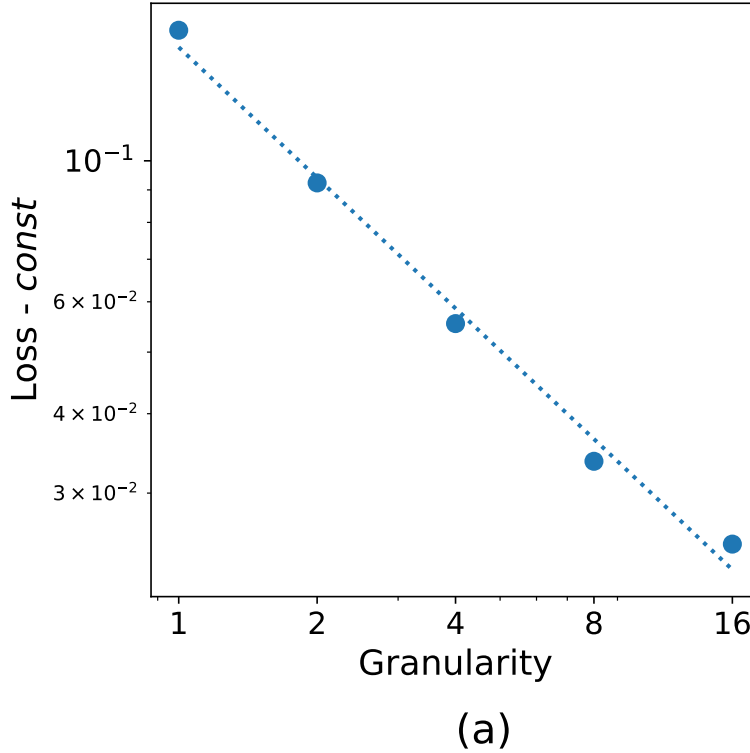


Figure 2.10: Effect of MoE granularity on loss for fixed model size and number of training tokens. Figure adapted from (Krajewski et al., 2024).

- *Device-level load balance*: encourages the total expert load to be balanced across devices, since groups of experts are stored on different GPUs or nodes.
- *Communication balance*: avoids sending too many tokens to the same device during routing.

These are auxiliary losses added to the main task loss, which is the language modeling loss. This can hurt the optimization of the main language modeling task, because the model is partly optimized to satisfy a routing constraint instead of only predicting the next token. For this reason, recent large sparse MoEs often move toward **auxiliary-loss-free load balancing**, where balancing is not directly added to the training loss. DeepSeek-V3 uses a bias-based auxiliary-loss-free balancing strategy (DeepSeek-AI, 2024b; Wang et al., 2024a). A bias $\mathbf{b} \in \mathbb{R}^e$ is added to the router scores for the top- k selection. The final routing weights are still computed from the original router scores, without this bias. During training, the expert load is monitored at each step: if an expert is overloaded, its bias is decreased by a small constant; if it is underloaded, its bias is increased. Hence the router is progressively pushed toward balanced expert usage, but without creating additional gradients for a balancing constraint. Kimi-K3 uses

a related but more direct approach called **Quantile Balancing** (Kimi Team, 2026), which computes an expert-specific threshold from the quantiles of the router scores such that overused experts receive a harder threshold and underused experts receive an easier one.

WAKING THE DEAD. Even with load balancing constraints, large-scale fine-grained sparse MoEs can lead to instabilities. And some of them are silent. In Step 3.5 Flash (StepFun Team, 2026), the authors report what they call **dead experts**, in comparison to the **dead neurons** phenomenon observed in ReLU-like activations. An expert is dead when it receives almost no useful training signal. In the simplest case, the router does not dispatch tokens to this expert for a long period, so the expert receives almost no gradient. But the Step 3.5 Flash report also shows a more subtle case: the router statistics can look healthy, while the expert itself is collapsing. The expert is selected, but its activations are not meaningful, and its parameter norms become stagnant or decay. A recent blog post³ on ultra-sparse MoEs makes a similar observation: lower-layer experts can silently collapse even when training loss, validation loss and load balancing metrics look normal (Jin et al., 2026). Their proposed solution, **LM Loss as Auxiliary Loss** (LLAL), is to temporarily compute the language modeling loss using the output of the collapsing MoE layers, so they are "forced" to become useful.

³
<https://alltoall.notion.site/save-lower-layer-moe-experts-llal>

2.3.1.4 Are "experts" really experts?

There is a debate about whether experts can specialize in particular domains. So far, there is no clear evidence of **expert specialization** in MoE LLMs, at least not in the intuitive sense of "specialization" on domains such as code, math, literature, etc. Mixtral is among the first open sparse MoE LLMs to study this question directly (Jiang et al., 2024). The authors measure how often each expert is selected on different subsets of The Pile, such as ArXiv, GitHub, PubMed, PhilPapers or Wikipedia. If experts were domain experts, we would expect some experts to be often selected for some domains. But this is not what happens. As shown in Figure 2.11, the routing across different domains is mostly uniform. However, they find that neighboring tokens in the sequence tend to be assigned to the same experts.

Expert specialization can be more salient on surface forms, such as scripts and language families. NLLB-200, which is a sequence-to-sequence MoE machine translation model, studies this by measuring the similarity between languages according to their gating decisions (NLLB Team et al., 2022). As shown in Figure 2.12, the late decoder MoE layer clearly separates languages: languages from the same family tend to select similar experts. The early encoder layer also shows some language-expert specialization. This suggests that expert specialization exists, but it is not necessarily a clean seman-

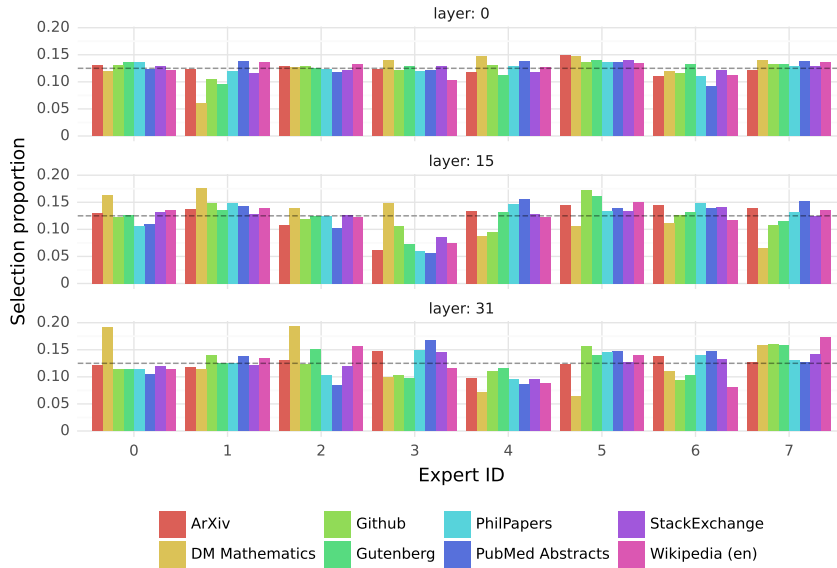


Figure 2.11: Expert selection proportions in Mixtral across domains and layers. The dashed line corresponds to uniform routing over the eight experts. Figure adapted from (Jiang et al., 2024).

tic specialization. This finding is also supported by observations in OLMoE, where the authors observe a strong vocabulary-level specialization of experts (Muennighoff et al., 2025). Some experts specialize in scripts or alphabets, others in punctuation, religious terms, dates, places, family words, or technical tokens. In other words, experts can specialize, but not always in high-level domains such as math or code. The specialization can also reflect scripts, language families, token forms, and vary depending on the position of the MoE layer.

2.3.1.5 On the non-adaptivity of TopK

Liam Fedus, first author of Switch Transformer together with Noam Shazeer, reports that Shazeer once intuited: “Parameters is knowledge, FLOPs is intelligence.”⁴ In this view, increasing the number of experts while keeping the number of activated parameters fixed mainly increases the model’s knowledge capacity. It does not increase the computation spent on each token. So the gain should be weaker on tasks that require more computation rather than more memorized knowledge. The natural next step is therefore to make sparse LLMs adaptive: use more experts when the token or the task is hard, and use fewer experts when it is not.

This intuition is now widely shared in the field, and many works propose ways to make MoE routing compute-adaptive⁵. One proposed routing strategy is inspired by TopP decoding (Huang et al., 2024). Unlike TopK, TopP does not activate a fixed number of experts. It sorts the router probabilities and keeps the smallest set of experts whose cumulative probability is above a threshold p . When the router

⁴ <https://x.com/LiamFedus/status/2090363702042304847?s=20>

⁵ See this blog post for an overview of the approaches <https://huggingface.co/blog/Spico/dynamic-routing>

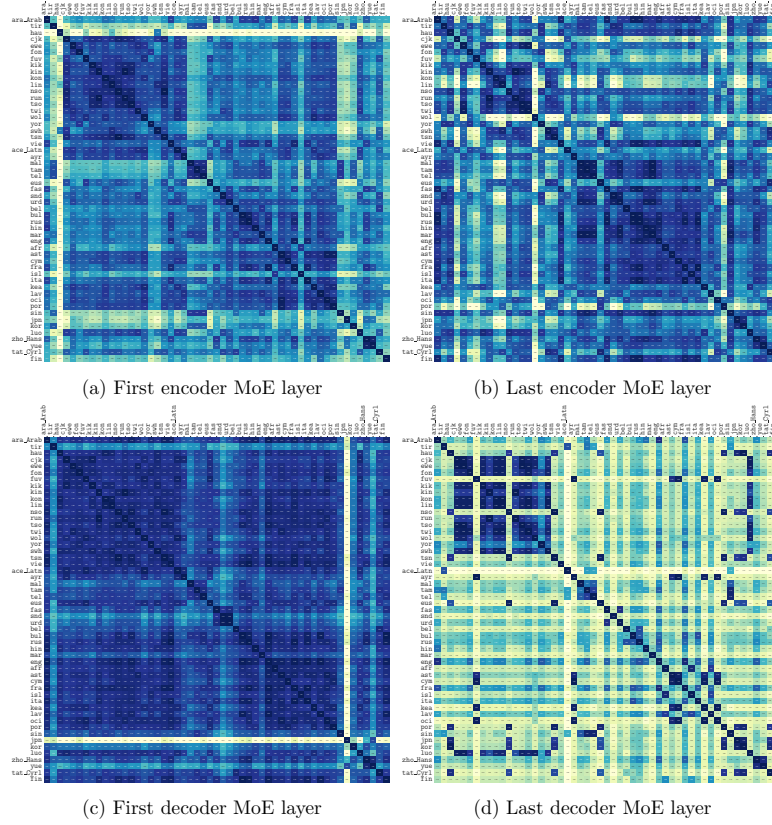


Figure 2.12: Cosine similarity between languages based on expert-choice patterns in NLLB-200. Figure adapted from (NLLB Team et al., 2022).

is confident, only a few experts are used. When the distribution is flatter, more experts are activated.

Another approach is to replace the hard TopK operation with a continuous ReLU router (Wang et al., 2025b). In that case, experts with positive router scores are active, and sparsity is encouraged during training by regularizing these scores. The number of active experts is therefore not fixed, but emerges from the input and from training.

The most commonly used adaptive sparse MoE approach at scale is probably the **zero-computation expert**. This method adds identity experts that let the input pass through without computation. This approach is used, for example, in LongCat-Flash (Meituan LongCat Team et al., 2025), where they encourage a sparsity budget by selecting K experts among both FFN experts and zero-computation experts. The constraint is on the average number of real FFN experts selected by the router. The auxiliary-free bias is then adjusted during training to push this average toward the target budget. In practice, the model still routes each token to K experts, but some of these experts cost almost nothing, so the effective compute can change from token to token.

Expert ID	Input token IDs	Token IDs generated while the expert is active
27	© (100%) ħ (100%) ³ (100%) í (100%) ĩ (100%) § (100%) © (100%) Ĵ (100%) ĵ Ĩ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) 𐌺 (100%) 𐌶𐌵 (100%) 𐌶 (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%) ̈́ (100%)	
58	(“ (100%) (“ (100%) ‘ (94%) ‘ (92%) “ (92%) ((92%) “ (90%) ‘ (89%) “ (88%) \$ (87%) [(87%) pound (86%)	such (100%) 486 (100%) see (95%) which (91%) driving (91%) UK (90%) who (88%) including (88%) normal (88%)
7	Him (100%) inde (100%) Jesus (98%) God (90%) pray (81%) Holy (80%) Quran (80%) God (77%) Lord (76%) glory (75%) Spirit (66%) Christ (65%)	rella (100%) Him (94%) sin (90%) prince (80%) glory (72%) Jesus (69%) Lord (68%) Christ (65%) Spirit (55%) Holy (53%) God (50%) Prayer (50%)
37	Sunday (100%) Tuesday (100%) Thursday (100%) Olympic (100%) Christmas (100%) rugby (100%) Championship (100%) weekends (100%)	days (91%) anniversary (90%) month (88%) week (84%) mpi (83%) semester (81%) mand (80%) Olympics (78%) cent (76%) season (76%) perm (75%)
3	grandmother (92%) brother (91%) Daisy (83%) daughter (78%) father (72%) wife (70%) husband (70%) lady (63%) dad (62%) boy (61%)	hood (36%) mother (35%) inde (31%) boy (29%) girl (28%) married (27%) tri (21%) Gab (20%) died (18%) taught (14%) lived (13%) knew (10%)

Table 2.3: Vocabulary specialization in the seventh layer of OLMoE-1B-7B. Table adapted from (Muennighoff et al., 2025).

2.3.2 Sparse Memory Layers

We discussed **granularity**, and have shown that increasing the number of experts, while keeping both the total model size and the number of activated parameters fixed, can greatly improve performance (see Figure 2.10). The extreme case of granularity would be at the neuron level: each column of the MLP weights can be seen as a small expert, and only a few of them are activated for a given token. Doing so turns the dense MLP into a **sparse memory layer**. We mentioned earlier, in the introduction, that the MLP in a Transformer can be seen as a key-value memory, and that several empirical results support this view. If this is true, then we can also argue that not all key-value entries are needed for every input token. Some entries should be useful for one token, and irrelevant for another one. The natural idea is therefore to make the MLP sparse. While this approach is less common than MoE to build sparse Transformers, it has some interesting advantages.

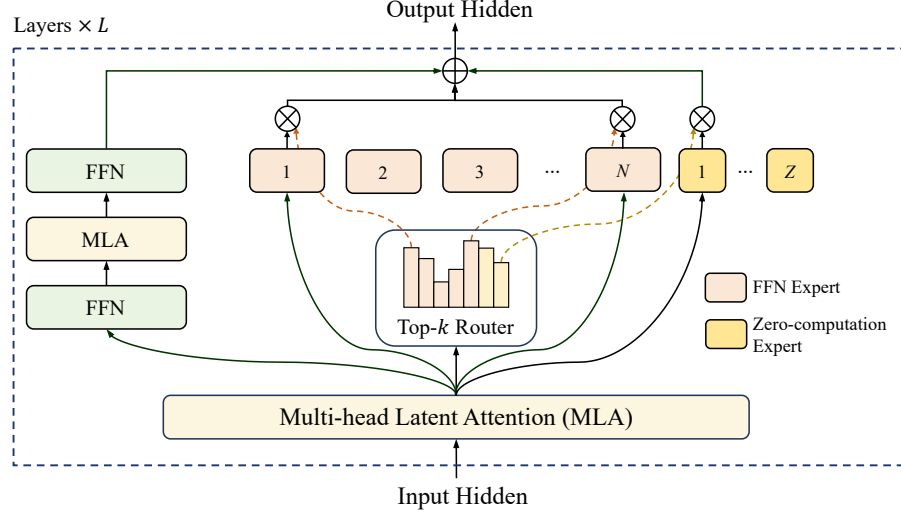


Figure 2.13: LongCat-Flash architecture with zero-computation experts. The router still selects K experts, but some selected experts are identity paths, which makes the effective compute adaptive. Figure from (Meituan LongCat Team et al., 2025).

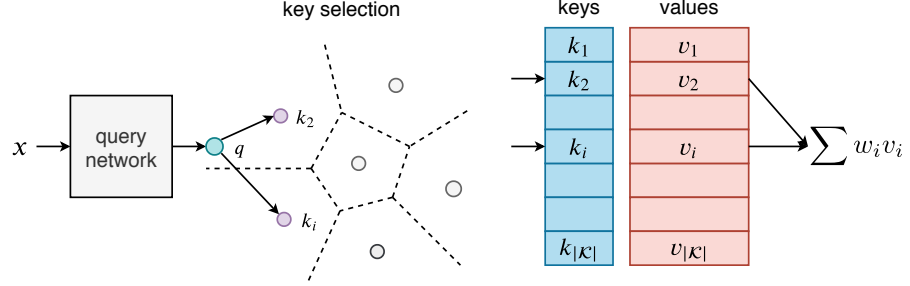


Figure 2.14: Overview of a key-value memory layer. The input is projected into a query, compared to the memory keys, and only the values associated with the selected key are used for computation. Figure from (Lample et al., 2019).

The idea of augmenting neural networks with a large memory is not new (Weston et al., 2015), but it has been developed into an efficient approach that can scale to modern LLMs (Lample et al., 2019).

Following the notation used in prior work (Lample et al., 2019), a memory layer is a function $m : \mathbb{R}^d \rightarrow \mathbb{R}^n$. Given an input $x \in \mathbb{R}^d$, a query network produces a query vector $q(x) \in \mathbb{R}^{d_q}$. This query is compared to a set of trainable keys $\mathcal{K} = \{k_1, \dots, k_{|\mathcal{K}|}\}$, with $k_i \in \mathbb{R}^{d_q}$, and each key is associated with a trainable value $v_i \in \mathbb{R}^n$. The memory does not read all values. It first selects the k keys whose inner product with the query is the largest:

$$\mathcal{I} = \mathcal{T}_k \left(\left(q(x)^\top k_i \right)_{i=1}^{|\mathcal{K}|} \right), \quad (2.2)$$

where $\mathcal{T}_k$ denotes the TopK operator. Then, the scores of the selected keys are normalized:

$$w = \text{Softmax} \left(\left(q(x)^\top k_i \right)_{i \in \mathcal{I}} \right), \quad (2.3)$$

and the output of the memory layer is the weighted sum of the selected values:

$$m(x) = \sum_{i \in \mathcal{I}} w_i v_i. \quad (2.4)$$

The query decides which memory entries are relevant for the input, and only these entries are read and updated. The issue is that, in this simple version, selecting the TopK keys requires comparing the query to all keys. This is extremely costly for modern LLM scaling. And the promise of Sparse Memory Layers is to scale the memory database to millions or even billions of entries, which is impossible to achieve in the current form. There are some efficient alternatives to this, which we will present next.

2.3.2.1 *Scaling Memory Layers*

PRODUCT-KEY MEMORY Product-Key Memory (PKM) (Lample et al., 2019) makes key search scalable by using a product structure. Instead of searching through one huge set of keys, PKM trains two smaller sets of keys, $\mathcal{K}_1$ and $\mathcal{K}_2$. A full key is not stored directly. It is formed by combining one key from the first set and one key from the second set. If $|\mathcal{K}_1| = |\mathcal{K}_2| = m$, this gives m^2 possible memory entries while storing only $2m$ key vectors. The query is also split in two, $q(x) = [q_1(x), q_2(x)]$, and the score of a composed key is computed as $q_1(x)^\top k_i^1 + q_2(x)^\top k_j^2$. So the system first finds the best keys in $\mathcal{K}_1$ and the best keys in $\mathcal{K}_2$, then combines these candidates to recover the best pairs. Each selected pair points to a value vector, and the output is still the weighted sum of the selected values, exactly as in the ordinary memory layer. In a standard memory, each memory entry has its own explicit key, and the query must be compared to all keys. The cost of retrieval therefore grows directly with the number of memory entries. In PKM, the memory is still large, but its keys are factorized. The model first searches in two small key tables, which is cheap, and only then forms a small set of candidate pairs. The expensive comparison with all possible composed keys is avoided.

ULTRA SPARSE MEMORY Ultra-Sparse Memory, or UltraMem, is a modern attempt to make PKM actually competitive at LLM scale (Huang et al., 2025) by improving several practical weak points of PKM. First, they increase the diversity of the values that can be activated. They achieve this by halving the value dimension D_v , which allows them to double the number of values while keeping the number of

activated value parameters fixed. The model can access more, smaller values and compose a richer output. Second, they observe that PKM is **memory-bound** during training. A very large sparse memory layer requires few FLOPs, but it can be expensive in memory access and communication. By splitting the huge memory into several smaller memories placed across the network, UltraMem avoids concentrating all memory access in a single blocking operation. The memory lookups become smaller and more distributed, making it easier to overlap them with the standard Transformer computation during training. Finally, they revisit the product-key decomposition itself with a richer approach for the query-key retrieval.

MIXTURE OF MILLION EXPERTS Another direction is to keep the product-key retrieval mechanism, but remove the key-value memory interpretation. This is the idea of PEER (Parameter Efficient Expert Retrieval) (He, 2024). Conceptually, PEER is similar to a very fine-grained MoE. Instead of a simple linear layer, the router uses product keys to retrieve experts efficiently, as in PKM, but each retrieved item is not a value vector stored in memory but a tiny MLP expert. Each expert is reduced to the smallest possible MLP: one up projection vector, one non-linearity, and one down projection vector. So an expert is a single hidden neuron. The layer retrieves only a few of these tiny experts and sums their outputs with router weights. The difference with PKM is therefore what is retrieved. PKM retrieves values from a key-value memory whereas PEER retrieves tiny experts. This makes the layer more naturally connected to MoE scaling and granularity: instead of having a few large experts, PEER pushes the idea to the extreme by using a large number of very small experts, while product-key retrieval keeps the routing cost cheap. Listing 2.1 gives the intuition of the forward pass, following the pseudocode in the original paper (He, 2024).

2.3.3 *Embedding Memory Layers*

Some solutions exist to augment the LLM memory without the complexity and cost of searching a large key-value store. Embedding memory layers can be seen as a special case of sparse key-value memory layers, where the keys are not learned dense vectors but fixed discrete indices. For a standard embedding table, the key is simply the token index, and the value is the corresponding embedding vector. During the forward pass, each token x_i maps to exactly one index. In that sense, the retrieval is a TopK with $K = 1$, but without any query-key similarity to compute. Embedding memory layers are used in several recent LLMs as a simple trick to augment the model memory without increasing the forward cost too much. A simple version appears in the modded-nanoGPT speedrun as *value embeddings* where

Listing 2.1: Pseudo-code implementation of a PEER layer forward pass.

```

def peer_forward(self, x):
    # Embedding layers storing the down/up projection weights of
    # all experts
    self.w_down_embed = nn.Embed(num_embeddings=self.n_experts,
                                  features=self.d_model)
    self.w_up_embed = nn.Embed(num_embeddings=self.n_experts,
                                features=self.d_model)

    # Retrieve the weights of the top matching experts using
    # product keys
    # indices and scores have shape 'bthk',
    # where h is the number of heads and k the number of
    # retrieved experts
    indices, scores = self.get_indices(self.query_proj(x),
                                       self.sub_keys,
                                       top_k=self.k)

    w_down = self.w_down_embed(indices)
    w_up = self.w_up_embed(indices)

    # Compute weighted average of expert outputs
    x = jnp.einsum('btd,bthkd->bthk', x, w_down)
    x = self.activation(x)
    x = x * nn.softmax(scores)
    x = jnp.einsum('bthk,bthkd->btd', x, w_up)
    return x

```

extra token embeddings are summed with the value vectors in the attention layers.⁶ More generally, the embedding table can store not only token embeddings, but also n -gram embeddings (Roy et al., 2022; Liu et al., 2026b) to augment the model memory. This approach is used and has been studied in many architectures, including LongCat-Flash (Liu et al., 2026b) and Qwen3.8-Flash-Next⁷ Figure 2.15 shows how n -gram embeddings are used in LongCat-Flash-Lite. The embedding memories are used only in the input embedding module, before the Transformer layers. For each token position i , the model forms left-context n -grams ending at the current token, such as (B, C) or (A, B, C) . These n -grams are mapped through hash functions to embedding tables. The retrieved unigram and n -gram embeddings are projected when needed, averaged, and used as the initial hidden representation $x_i^{(0)}$ passed to the Transformer stack.

There are several ways to inject embedding-memory information into a Transformer architecture. LongCat-Flash-Lite (Liu et al., 2026b) uses it to construct the input token representations for the Transformer backbone. Qwen3.8-Flash-Next instead injects the retrieved n -gram vectors inside the Transformer, at a selected layer. The authors found that when the n -gram vocabulary is scaled by adding extra param-

⁶ <https://github.com/KellerJordan/modded-nanogpt>

⁷ https://github.com/QwenLM/Qwen3.8-Flash-Next/blob/main/tech_report.pdf

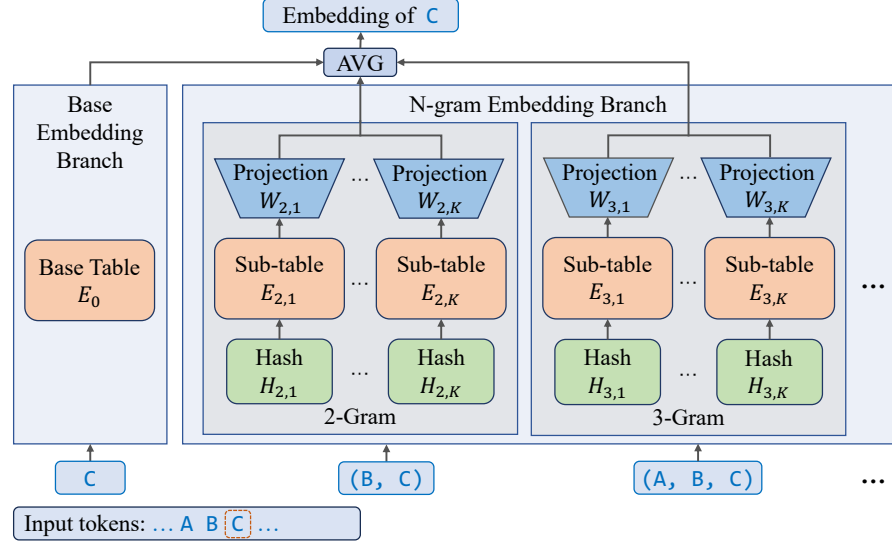


Figure 2.15: Architecture of an n -gram embedding layer. The token embedding is augmented with hashed n -gram embedding branches. Figure from (Liu et al., 2026b).

eters, loss decreases monotonically, while downstream performance stagnates, as shown in Table 2.4.

Vocab. Scale	Loss	Knowledge			STEM	
		MMLU	MMLU-Pro	SuperGPQA	MATH	GSM8K
None	1.585	62.78	33.43	20.97	32.52	59.21
20×	1.553	64.14	34.46	21.83	37.38	65.09
50×	1.541	64.71	35.80	21.49	37.32	64.00
100×	1.534	64.70	35.87	21.93	36.98	63.08
200×	1.526	64.85	35.21	21.11	35.34	62.96

Table 2.4: Effect of n -gram vocabulary scaling in Qwen3.8-Flash-Next. Vocabulary scales are measured relative to the base tokenizer vocabulary size. Larger vocabularies add parameters. Table adapted from (Qwen Team, 2026a).

2.3.4 Post-training Activation Sparsity

We presented several approaches that induce activation sparsity during training, but sparsity can also be introduced after pre-training. These post-training activation-sparsity methods are used to reduce the inference cost of dense pre-trained LLMs.

Most modern LLMs use gated MLPs with smooth activation functions such as GELU, SiLU, or Swish, which produce few exact zeros. A natural way to recover activation sparsity is therefore to replace, after training, these smooth activations with ReLU-like activations. This

process is called **ReLUfication**(Mirzadeh et al., 2023). In gated MLPs, this makes parts of the intermediate activation exactly zero, so the corresponding intermediate channels in the up and down projections can be skipped during inference. However, directly changing the activation function creates a distribution shift in the pre-trained model, so additional calibration or continued training is usually needed to recover performance.

Instead of replacing the MLP activation function, other approaches exploit the fact that many intermediate activations are concentrated near zero and can therefore be sparsified by magnitude thresholding. TEAL (Liu et al., 2025) applies training-free magnitude pruning to input activations of the MLP, using layer-dependent thresholds estimated from calibration data. CATS (Lee et al., 2024) thresholds the absolute value of the gate activation $\text{SiLU}(xW_{\text{gate}})$ and sets small values to zero. The resulting mask makes it possible to skip the corresponding channels in the up and down projections during inference, improving latency. Learn-To-be-Efficient (LTE) proposes a training-aware approach to learn sparsity via sigmoid-router thresholding (Zheng et al., 2024).

There are also approaches that transform dense pre-trained models into MoEs. **MoEfication** is a method for converting dense FFN layers into routed experts, while **upcycling** more generally means initializing an MoE from one or several dense checkpoints and then continuing training. For example, Qwen1.5-MoE-A2.7B is upcycled from Qwen-1.8B, and Qwen2 extends this line with a larger MoE model (Qwen Team, 2024; Yang et al., 2024a). Combining several dense models is not trivial, because their layers, notably attention layers, can differ a lot. Several studies therefore focus on how to combine different dense models into an MoE. BAM (Zhang et al., 2024a) proposes to reuse both FFN and attention parameters from dense models, using a mixture-of-attention variant. Nexus studies a continual domain-adaptation setting, where new domain experts are added to an existing MoE using an adaptive router initialized from domain embeddings (Gritsch et al., 2025).

2.3.5 Matryoshka Learning

Activation sparsity reduces the number of parameters used for each token, but it does not reduce the memory footprint of the model, since the full model parameters must be hosted in memory. Another approach is **Matryoshka Learning**, where a single model is trained to nest multiple model or embedding sizes within it. At inference time, it is possible to use the embedding dimension or model size that fits the compute and memory budget. The idea first emerged in representation learning as **Matryoshka Representation Learning** (MRL) (Kusupati et al., 2022). Given a set of target embedding di-

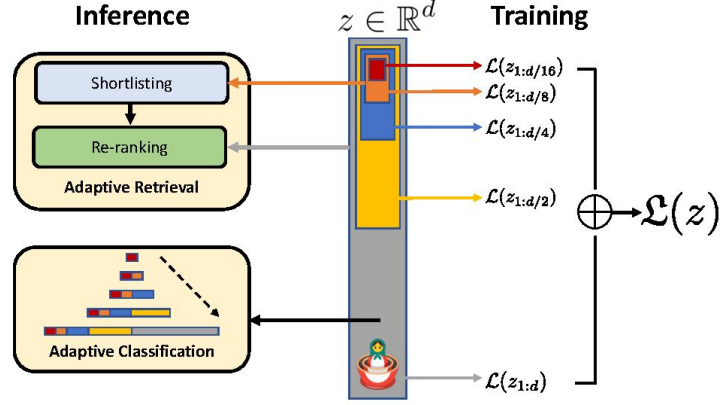


Figure 2.16: Matryoshka Representation Learning trains a representation to be useful at several embedding dimensions. Figure adapted from (Kusupati et al., 2022).

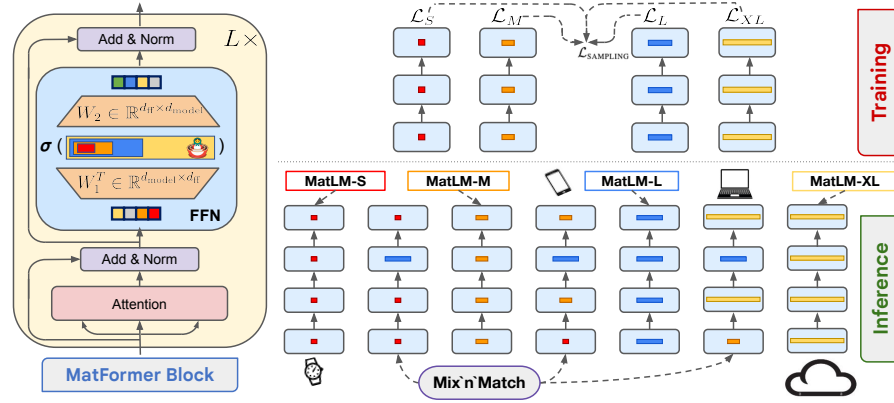


Figure 2.17: MatFormer nests several FFN sizes inside the Transformer block and trains the corresponding submodels jointly. Figure from (Devvrit et al., 2024).

mensions $\mathcal{D} = \{d_1, d_2, \dots, d_m\}$, MRL trains these dimensions jointly by summing the task loss over the different representation sizes: $\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{D}} \mathcal{L}(y, f_d(x))$ where x is the full representation and f_d selects or projects it to a d -dimensional representation. Most of the time, this function is simply a slicing operation that selects the first d dimensions of the representation, but it can also be a learned projection. The MRL framework is illustrated in Figure 2.16 and is widely used in retrieval models. It helps reduce the disk usage of the embedding store, and allows a trade-off between retrieval speed and accuracy. MRL can be seen as a form of activation sparsity since slicing a vector $x_{[:d]}$ means implicitly setting the remaining dimensions $x_{[d:]}$ to zero.

More recent work extends the same principle to LLM architectures. These methods train nested Transformer submodels of different sizes inside a larger model (Devvrit et al., 2024). This design is used in Gemma 3n E2B and E4B⁸ and Gemma 4 E2B and E4B.⁹ At inference

⁸ <https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/>.

⁹ https://ai.google.dev/gemma/docs/core/model_card_4.

time, it is possible to select a submodel of the desired size depending on the compute and memory budget.

2.4 OUR CONTRIBUTION TO ACTIVATION SPARSITY

Our contributions examine the limitations of fixed representation sizes and explore more adaptive sparsity:

1. We study Matryoshka Representation Learning (MRL) across speech and text. We show that MRL doesn't achieve optimal sparsity, as Matryoshka embeddings concentrate information in relatively few components, leaving the remainder unused. This suggests scope for more effective sparsity than fixed prefix slicing.
2. In a preliminary study, still ongoing at the time of writing, we explore **ZeroNeuron**, an adaptive, fine-grained, differentiable activation-sparsity mechanism for MLP activations and embedding representations. ZoN follows the same motivation as fine-grained MoEs and sparse memory layers: a larger hidden representation can be used, but only a subset of coordinates should be active for each input token (Krajewski et al., 2024; Lample et al., 2019; He, 2024). However, unlike TopK-based MoE routing or memory lookup, ZoN does not select a fixed number of experts or retrieve a subset of keys through explicit routing; instead, it learns a pointwise gate over the intermediate coordinates of the dense MLP. This preserves the dense tensor-parallel communication pattern while allowing the level of sparsity to vary with the input, which is the key difference from discrete expert routing and memory retrieval methods (Huang et al., 2024; Wang et al., 2025b; Huang et al., 2025). ZeroNeuron realizes a communication- efficient form of adaptive activation sparsity directly in the standard MLP computation.

Part II

COMPRESSION OF TRANSFORMER MODELS

You can put some informational part preamble text here.
Illo principalmente su nos. Non message *occidental* anglo-romanica da. Debitas effortio simplicate sia se, auxiliar summarios da que, se avantiate publicationes via. Pan in terra summarios, capital interlingua se que. Al via multo esser specimen, campo responder que da. Le usate medical addresses pro, europa origine sanctificate nos se.

3.1 INTRODUCTION

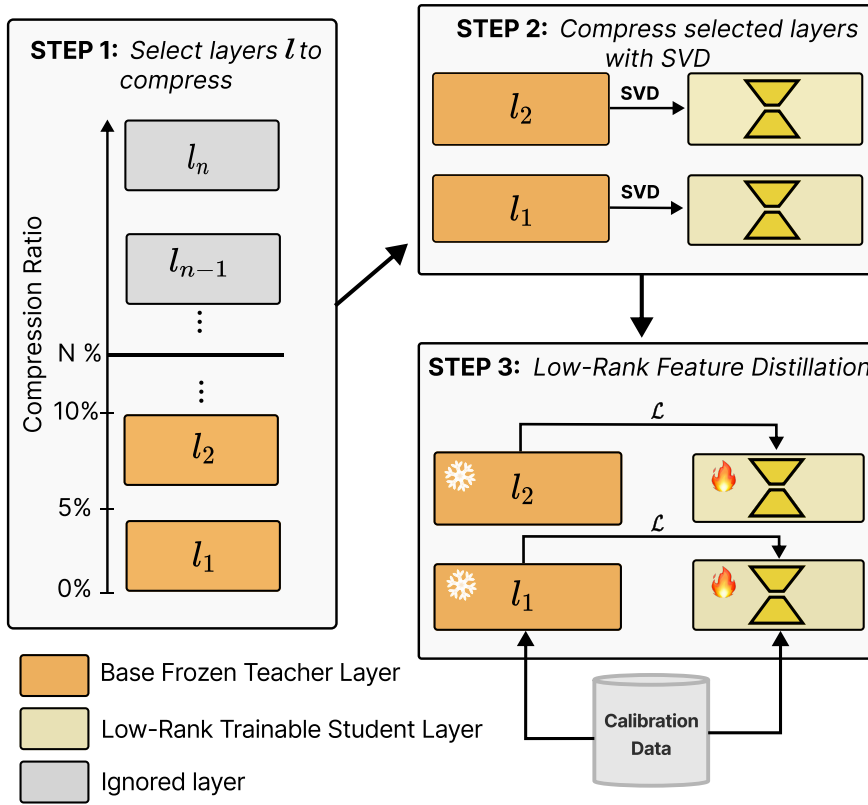


Figure 3.1: **Lillama approach**: **STEP 1** selects layers to compress for a target compression ratio (e.g., $N\%$) using various strategies (see Section 3.5). **STEP 2** compresses and initializes the chosen parameters via SVD. **STEP 3** distills the low-rank weights with a small calibration dataset.

As discussed earlier, compressing large language models is a crucial challenge that can be addressed through several complementary approaches: quantization, pruning, and distillation. In this section, we introduce a new one-shot compression method that locally distills low-rank weights and satisfies the following three objectives:

1. The compression algorithm must be compute-efficient, e.g., it should require minimal computational resources and run in a reasonable amount of time. For instance, iterative magnitude pruning as used in prior work (Frankle and Carbin, 2019) is too costly to be applied to large-scale LLMs.

Method	No Custom Kernel	No Ampere-only	Memory Gain	Is One-Shot	Calib. Data
SparseGPT (Frantar and Alistarh, 2023b)	✗	✗	✗	✓	-
Wanda (Sun et al., 2024b)	✗	✗	✗	✓	-
Teal (Liu et al., 2025)	✗	✗	✗	✓	-
ShearedLLaMA (Xia et al., 2024)	✓	✓	✓	✗	52B
Minitron (Sreenivas et al., 2024a)	✓	✓	✓	✗	94B
Lillama	✓	✓	✓	✓	0.013B

Table 3.1: Positioning of our approach within the compression literature. **No Custom Kernel** indicates that a custom GPU kernel is not required to observe speedup and **No Ampere-only** indicates that speedup is not restricted to Ampere GPUs. **Calib. Data** is the total number of tokens in billions used for compression.

2. It must achieve fast convergence using as little training data as possible; this objective is difficult to meet with many model pruning methods that require significant retraining after pruning to recover the lost performance.
3. The method should not cause a significant drop in performance; ideally, all the capabilities of the original LLM should remain intact.

To address the first constraint, we compress the model with a local distillation objective, as opposed to a global objective that is more costly. We further reduce the cost by compressing and distilling only a subset of all layers. To address the second constraint, we initialize the compressed layers with Singular Value Decomposition (SVD), which enables reducing the required number of gradient steps as well as the calibration dataset size. We further improve convergence by combining Teacher and Student activations with a joint distillation loss. We experimentally show the robustness of our approach by compressing several state-of-the-art small and large Transformers, Mixture-of-Experts, and Mamba LLMs. Figure 3.1 gives an intuitive overview of the proposed method, which is detailed in Section 3.5 and validated in Section 3.6.

Our approach addresses some limitations of the compression approaches presented in Section 2.1. Compared to unstructured pruning and semi-structured pruning, which result in models with sparse weights and do not achieve speedup without specially designed

hardware-specific kernels, our approach, which shrinks the LLM weights, saves memory across all hardware and leads to speedup, as fewer computations are performed. Structured pruning approaches can be costly: for example, the pruning step of Sheared-LLaMA (Xia et al., 2023) is five times as expensive as standard LM training, according to the authors. In contrast, thanks to local gradient updates, our approach is computationally efficient, allowing us to compress a 47B model within minutes on a single A100 GPU.

There are also some low-rank decomposition approaches that compress models by approximating the pretrained matrices with lower-dimensional ones. Building on the observation that neural networks have low intrinsic dimensions (Li et al., 2018), subsequent work shows that Transformer language models also require lower intrinsic dimensions depending on the language task (Aghajanyan et al., 2020). However, other studies show that the weights in some pretrained Transformer models are full-rank, unlike their activations, and propose activation-aware compression methods (Yu and Wu, 2023; Chen et al., 2021). Our work is related to these approaches, as we also decompose the matrices of the models. However, we apply a lightweight layer-wise feature distillation objective to better recover from compression and propose different distillation strategies to accelerate convergence and improve performance.

Standard knowledge distillation is data-intensive when the student model’s parameters are randomly initialized. Hence, this process is as costly as a standard pretraining stage. Layer-wise distillation can be more suitable in some cases, as intermediate layers may contain certain information, such as speaker identity or phonemes in pretrained speech models (Baevski et al., 2020a), (Pasad et al., 2022), (Chang et al., 2022), (Chang et al., 2021). We leverage the observation that layer activations are low-rank and approximate them with fewer parameters by distilling the Transformer layer activations into low-rank weights initialized from the SVD of the base weights. This layer-wise distillation approach enables us to perform local gradient updates, hence making the distillation process cheaper.

3.2 MOTIVATION: LOW-RANK ACTIVATIONS

A motivation for compressing model weights with SVD would be a low-rank structure in those weights. However, previous studies (Chen et al., 2021), (Yu and Wu, 2023) have shown that the weights of pretrained Transformer models are often full-rank, while their activations have lower rank. We empirically confirm this observation on three recent models: Falcon-Mamba, Mistral-v0.1 7B, and Phi-2 3B. There are different measures of rank. In this study, we use a simpler

one: stable rank (*srank*) as a proxy measure for rank. The stable rank of a matrix is defined as the following ratio:

$$\text{srank}(X) = \frac{\sum_{i=1}^r \sigma_i^2}{\sigma_1^2} \quad (3.1)$$

with $X \in \mathbb{R}^{d_1 \times d_2}$ being the input matrix (activations or weights), $r = \min(d_1, d_2)$, and $\sigma_1 \geq \dots \geq \sigma_r$ the singular values of X .

Figure 3.2 shows the stable rank of various weight matrices and layer activations for three models: Falcon-Mamba, Mistral-v0.1 7B, and Phi-2 3B. For all models, activations are significantly lower-rank than weight matrices, which confirms previously published results (Yu and Wu, 2023).

This not only motivates the use of activations for compression, but also explains why applying SVD to weights alone does not work well, as we will show in the experiments. How can we explain the fact that weights are full-rank while activations are low-rank? This is an open question. We know that modern LLMs are pre-trained on large and diverse datasets from the web that contain a great deal of knowledge. Hence, the weights of the model need to be full-rank to capture all this knowledge. However, for a given input from a specific task, only a subset of this knowledge is needed, which may explain the low-rank structure of activations.

3.3 BACKGROUND: LOW-RANK APPROXIMATION OF TRANSFORMER MODELS

Before presenting our approach, we first introduce two methods for low-rank approximation of Transformer models: low-rank approximation from weights and from features.

3.3.1 Low-Rank Approximation from Weights

Given $W \in \mathbb{R}^{d_1 \times d_2}$, a pretrained weight matrix, and an example $x \in \mathbb{R}^{d_2}$, the activations $y \in \mathbb{R}^{d_1}$ are computed as $y = Wx$. It is possible to reduce the computations in this operation by using a low-rank approximation of W : $y \simeq \Delta W x = ABx$ with $\Delta W = AB$ the low-rank decomposition of W , composed of $A \in \mathbb{R}^{d_1 \times r}$ and $B \in \mathbb{R}^{r \times d_2}$. When the rank r is small enough, the number of parameters $r(d_1 + d_2)$ in the low-rank equation is smaller than $d_1 d_2$ in the full-rank equation. Estimating the low-rank matrix $\Delta W = AB$ can be formulated as a minimization problem to find the low-rank ΔW that best approximates W :

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \|W - \Delta W\|_F \quad (3.2)$$

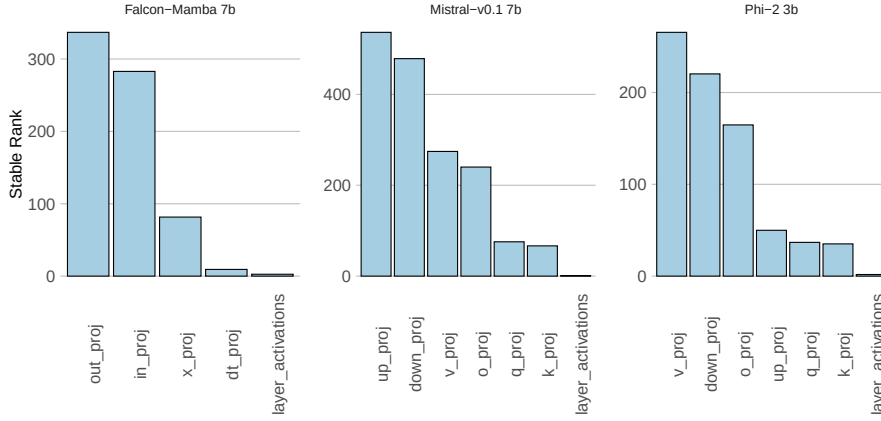


Figure 3.2: **Activations are low-rank.** Comparison of the stable rank of weights (*_proj) and layer activations (layer_activations). Each bar is the average stable rank across all layers.

Observation 1: Transformer layer activations are extremely low-rank

Transformer layer activations have extremely low stable rank compared to the weights. This extends the previous observations discussed above, which focused on linear activations, to the outputs of entire Transformer layers. Interestingly, the key projection weights also appear to have low stable rank.

It is well-known that this minimization problem can be approached using SVD, a low-rank approximation method that offers the optimal rank- r approximation of a given matrix with respect to the Frobenius norm $\|\cdot\|_F$. SVD decomposes a matrix W into three matrices: $W = USV^T$, where $S \in \mathbb{R}^{d_1 \times d_2}$ is a diagonal matrix containing the singular values of W sorted in descending order, and $U \in \mathbb{R}^{d_1 \times d_1}$ and $V \in \mathbb{R}^{d_2 \times d_2}$ are orthonormal matrices. The optimal low-rank approximation of $W \in \mathbb{R}^{d_1 \times d_2}$ can be obtained by keeping the first r singular values, with $r < \min(d_1, d_2)$:

$$\widehat{\Delta W} = (U_{:,r} S_{:,r,r} V_{:,r}) = AB \quad (3.3)$$

with $A = U_{:,r} S_{:,r,r}$ and $B = V_{:,r}$, where the notation $_{:,b}$ refers to the slicing operation.

3.3.2 Low-Rank Approximation from Features

As discussed previously, many studies (Chen et al., 2021), (Yu and Wu, 2023) have shown that the activations (i.e., features) of pretrained transformers have lower rank than the weights. Recent work shows

that Transformer activations can be sparsified up to 60% (Liu et al., 2025). This motivates using the activations of the linear layers to estimate the low-rank matrices.

Let $\mathcal{D} = \{x_i \in \mathbb{R}^{d_2}\}_{1 \leq i \leq N}$ be a calibration dataset, and $W \in \mathbb{R}^{d_1 \times d_2}$ the parameters of a linear layer. Finding the low-rank matrix ΔW that best reproduces the activations $\{y = Wx\} \quad \forall x \in \mathcal{D}$ involves solving:

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \quad \frac{1}{N} \sum_{x \in \mathcal{D}} \|Wx - \Delta Wx\|_F \quad (3.4)$$

An analytic solution to this minimization problem is the eigendecomposition of the covariance matrix of the activations. We first collect all activations $\mathcal{Y} = \{y = Wx\}_{x \in \mathcal{D}}$. The covariance matrix of the activations can be estimated as $\Sigma = \mathbb{E}_{y \in \mathcal{Y}} [yy^T] - \mathbb{E}[y]\mathbb{E}[y]^T$ with $\Sigma \in \mathbb{R}^{d_1 \times d_1}$.

Since Σ is a diagonalizable matrix, we can apply its eigendecomposition: $\Sigma = USU^T$, where $U \in \mathbb{R}^{d_1 \times d_1}$ contains the eigenvectors of Σ and $S \in \mathbb{R}^{d_1 \times d_1}$ is the diagonal matrix containing the eigenvalues sorted in decreasing order. As for the SVD equation above, we can keep the eigenvectors corresponding to the r largest eigenvalues, which gives us $A = U_{:,r}$ and $B = U_{:,r}^T W$.

Compared to the low-rank matrices in the SVD equation above, the weights A and B here are learned to reproduce the activations of the base weight W , thanks to the minimization objective above. In the next section, we highlight the limitations of this approach and introduce our proposed method.

3.4 PROPOSED APPROACH

We next identify and address three potential limitations of the approach presented in Section 3.3.2.

3.4.1 Non-linear Feature Approximation.

The analytical solution above is limited to activations from linear layers. To generalize to non-linear modules, we first observe that this objective can be seen as a feature distillation objective that may be optimized numerically by gradient descent rather than analytically by eigendecomposition. Given the input batch $X \in \mathbb{R}^{d \times b}$ of b examples, we denote the output activations of the i^{th} Teacher module $\mathcal{T}^{(i)}$ as

$$Y^{(i)} = \mathcal{T}^{(i)}(X; \Theta^{(i)}) \quad (3.5)$$

where $\Theta^{(i)}$ are the original pretrained matrices of the i^{th} Teacher, and $Y^{(i)} \in \mathbb{R}^{d \times b}$ are its output activations. Similarly, we denote the output activations of the i^{th} Student module $\mathcal{S}^{(i)}$ as:

$$\hat{Y}^{(i)} = \mathcal{S}^{(i)}(X; \Delta \Theta^{(i)}) \quad (3.6)$$

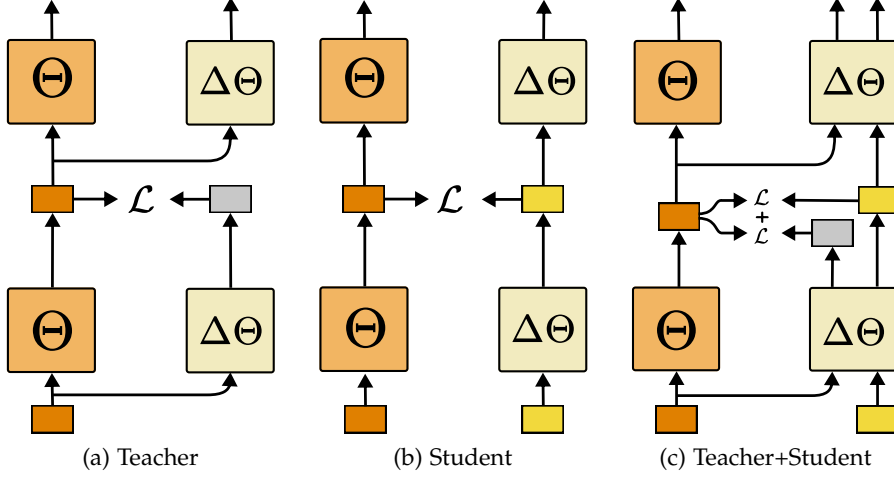


Figure 3.3: **Accelerating convergence by learning from Teacher and Student activations through a joint loss.** We compare three distillation strategies: **(a) Teacher**: the input to the compressed student layer comes from the output of the previous teacher layer; **(b) Student**: the input to the compressed student layer comes from the output of the previous student layer; **(c) Teacher+Student**: the compressed student layer is applied separately to the output of the previous teacher layer and to the output of the previous student layer, and the joint loss is the sum of the two corresponding distillation losses.

where the Student module is parameterized with the low-rank matrices $\Delta\Theta^{(i)}$ and $\hat{Y}^{(i)} \in \mathbb{R}^{d \times b}$ are the output activations. We can express the distillation objective of the i^{th} Student module as:

$$\widehat{\Delta\Theta}^{(i)} = \underset{\Delta\Theta^{(i)}}{\operatorname{argmin}} \mathcal{L}^{(i)}(Y^{(i)}, \hat{Y}^{(i)}) \quad (3.7)$$

where $\widehat{\Delta\Theta}^{(i)}$ are the estimated low-rank matrices of the i^{th} Student module and $\mathcal{L}^{(i)}(Y^{(i)}, \hat{Y}^{(i)})$ is the loss measuring the distance between the activations of the Teacher and the Student. We use the combined loss from prior work (Chang et al., 2022) because we observed instability with the ℓ_1 loss alone:

$$\begin{aligned} \mathcal{L}^{(i)} &= \mathcal{L}_{\ell_1}^{(i)} + \mathcal{L}_{\cos}^{(i)} \\ &= \sum_{t=1}^b \left[\frac{1}{D} \left\| Y_t^{(i)} - \hat{Y}_t^{(i)} \right\|_1 - \log \sigma \left(\cos \left(Y_t^{(i)}, \hat{Y}_t^{(i)} \right) \right) \right] \end{aligned} \quad (3.8)$$

where D is the hidden-vector dimension, σ is the sigmoid activation and $\cos(\cdot, \cdot)$ is the cosine similarity. We propose to approximate the low-rank weights of the Students through gradient descent. In the linear case, this should converge towards the eigendecomposition solution described in Section 3.3.2. However, as we show later, we extend

the distillation process to non-linear modules, which requires numerical optimization. We also show that initializing the Student’s low-rank parameters with SVD, rather than randomly, improves convergence.

3.4.2 Beyond Teacher-only Activations.

The second potential limitation of the approach described in Section 3.3.2 and equations above is that the Student and the Teacher modules take the same input X , which is the output of the previous layer of the Teacher. This means that the Student is trained from the activations of the Teacher module only, which are not available at inference time. This approach has been used to approximate the linear layers of Transformer models under the name Atomic Feature Mimicking (Yu and Wu, 2023). Let $\hat{Y}_T^{(i)} = \mathcal{S}^{(i)}(Y_T^{(i-1)}; \Delta\Theta^{(i)})$ be the output activations of the Student module when fed $Y_T^{(i-1)}$, the output activations of the Teacher at the previous layer $i - 1$. The loss can then be written as:

$$\mathcal{L}_T^{(i)} = \mathcal{L}^{(i)}(Y_T^{(i)}, \hat{Y}_T^{(i)}) \quad (3.9)$$

where $Y_T^{(i)}$ are the gold activations of the current Teacher. This approach can lead to fast convergence as the Student modules learn from gold and high-quality activations produced by the Teacher modules. However, it can cause a performance drop during inference as the Teacher activations are not available. An alternative approach is to have the Student module take as input the activations of previous Student modules rather than those of Teacher modules:

$$\mathcal{L}_S^{(i)} = \mathcal{L}^{(i)}(Y_T^{(i)}, \hat{Y}_S^{(i)}) \quad (3.10)$$

where $\hat{Y}_S^{(i)} = \mathcal{S}^{(i)}(\hat{Y}_S^{(i-1)}; \Delta\Theta^{(i)})$ denotes the output activations of the current Student module when taking as input the output activations $\hat{Y}_S^{(i-1)}$ of the Student module at the previous layer, and $Y_T^{(i)}$ are the gold activations of the corresponding Teacher module. However, although we found that this loss generally performs better than the previous one, it leads to slow convergence due to the errors in the activations of the Student modules. To address both issues, we propose to combine the two losses:

$$\mathcal{L}_{T+S}^{(i)} = \mathcal{L}_T^{(i)} + \mathcal{L}_S^{(i)} \quad (3.11)$$

This distillation approach is illustrated in Figure 3.3c. We don’t introduce any hyperparameter for controlling the importance of one loss over another in the joint loss. We leave this for future work.

3.4.3 Module-wise Distillation.

Finally, we observe that the modules need not be limited to linear layers: any sub-part of the model that outputs low-rank activations can

be compressed with our method. In our experiments, we distill at the Transformer layer level and leave comparisons with other distillation levels for future work. We also experiment with models other than pretrained Transformer-based language models.

3.5 FAST AND MEMORY-EFFICIENT COMPRESSION

Computing the optimal compression rank of each matrix individually may be costly: while search algorithms can be a solution for smaller models, they can be difficult to scale to LLMs with billions of parameters. Assuming a target compression rate N , we consider three simple strategies: **uniform**, **top**, and **bottom**-first compression. Figure 3.4 summarizes the strengths and weaknesses of each strategy.

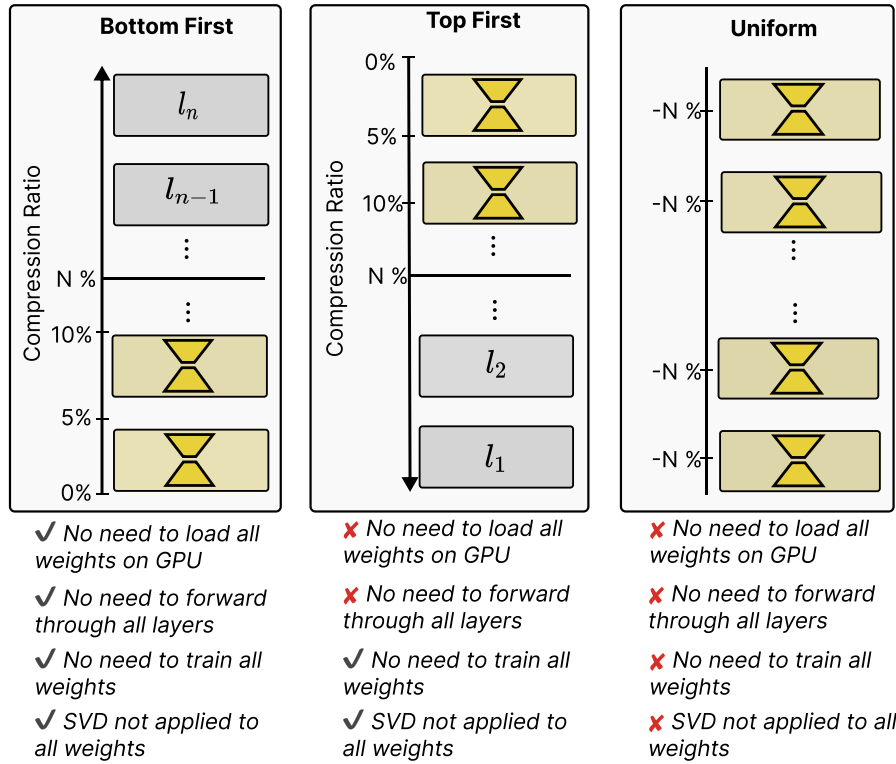


Figure 3.4: Illustration of 20% compression when using the three strategies: bottom, top, and uniform. Green boxes are compressed layers with low-rank matrices while gray boxes are non-compressed ones. $l_1, l_2, \dots, l_{n-1}, l_n$ are the layer indices.

3.5.1 Uniform Weights Compression

This approach removes $N\%$ of the parameters from each weight matrix in the model. It makes no assumptions about the importance of different layers but is costly because it requires computing the low-rank approximation of all weight matrices. It also requires training

all the model's layers, increasing GPU memory and computation requirements through forward and backward passes across the entire model.

3.5.2 Top Layers First Compression

This approach removes $N\%$ of the parameters from the model by prioritizing the top layers. This is more efficient than the uniform approach as it requires computing the low-rank approximation of fewer weight matrices and training fewer layers. However, it still requires loading the full model in GPU memory and performing a forward pass through all the layers, although only a subset is compressed and trained.

3.5.3 Bottom Layers First Compression

This approach removes $N\%$ of the parameters from the model by prioritizing the bottom layers. Compared to the previous two approaches, this one is more efficient as it requires computing the low-rank approximation of fewer weight matrices and training fewer layers. It also allows us to load only a subset of the model into GPU memory and run forward passes through only the first compressed layers, which is not the case for the previous two approaches. As we will show, this approach can compress large models that could not fit into GPU memory. Algorithm 1 describes the bottom-first compression approach.

Algorithm 1: Bottom Layers First Compression Algorithm

Input: $\mathcal{M}$ is the base model; S is the target size of the compressed model; k is the minimum possible rank to set; m is the increment when generating ranks

Result: Low-rank compressed $\mathcal{M}$

```

1  $\mathcal{R} \leftarrow \emptyset;$ 
2 for each weight matrix  $W \in \mathbb{R}^{d_1 \times d_2}$  in  $\mathcal{M}$  do
3    $b \leftarrow \min(d_1, d_2);$ 
4   for  $r = k; r \leq b; r = r + m$  do
5     if  $r \times (d_1 + d_2) < d_1 \times d_2$  then
6        $\mathcal{R} \leftarrow \mathcal{R} \cup \{(r, W)\};$ 
7 sort  $\mathcal{R}$  by layer index in increasing order, then by rank in decreasing order;
8 while  $|\mathcal{M}| > S$  do
9   get the next  $(r, W)$  from  $\mathcal{R};$ 
10  compute  $A, B$  from  $W$  using Eq. 3.3 with rank  $r;$ 
11  replace  $W$  by  $A, B$  in  $\mathcal{M};$ 
12 return  $\mathcal{M};$ 

```

3.5.4 Ease of Implementation

The approach is straightforward to implement. Figure 3.5 shows a simple pseudocode implementation that can be easily integrated into most recent PyTorch models.

```
def low_rank_layer(layer, ranks):
    for name, module in layer.named_modules():
        if isinstance(module, torch.nn.Linear):
            # get the rank of this matrix, as computed with Algorithm 1
            r = ranks[name]
            # create the Up-Down linears for the low-rank matrices
            l1 = torch.nn.Linear(module.in_features, r, bias=False)
            l2 = torch.nn.Linear(r, module.out_features, bias=False)
            # initialize the Low-Rank matrices with SVD
            u, s, v = torch.linalg.svd(module.weight)
            w1 = torch.diag(s)[:r, :r] @ v[:r, :]
            w2 = u[:, :r]
            l1.weight = torch.nn.Parameter(w1)
            l2.weight = torch.nn.Parameter(w2)
            # replace the old linear layers with the low-rank linear layer
            low_rank_linear = torch.nn.Sequential(l1, l2)
            setattr(layer, name, low_rank_linear)

# 'ranks' are the rank values computed using Algorithm 1
# 'base_layer' is the uncompressed pretrained layer.
compressed_layer = low_rank_layer(copy(base_layer), ranks)
optimizer = torch.optim.AdamW(compressed_layer.parameters())

def feature_approximation_hook(module, inputs, outputs):
    optimizer.zero_grad()
    outputs_ = compressed_layer(inputs)
    loss = loss_fn(outputs_, outputs)
    loss.backward()
    optimizer.step()

# attach the function to the base layer as a forward hook
# so the function is called at each forward call of the base layer
base_layer.register_forward_hook(feature_approximation_hook)
```

Figure 3.5: Example of a PyTorch implementation of our approach using the Teacher-only distillation strategy.

3.6 EXPERIMENTS ON TRANSFORMERS

3.6.1 Setup

3.6.1.1 Models.

We evaluate our method on several Transformer-based LLMs: a 47B Mixture-of-Experts model (Mixtral-8x7B-v0.1), medium-sized LLMs (Mistral-v0.1 7B and Phi-3 14B), and a smaller model (Phi-2 3B).

3.6.1.2 Data.

For calibration, we use 13 million tokens randomly sampled from Slim-Orca (Lian et al., 2023), an open-source replication of Orca (Mukherjee et al., 2023). In preliminary experiments, we also tested other datasets

such as RedPajama (Computer, 2023), but observed better results with instruction data.

3.6.1.3 Implementation Details.

We implement our approach using version 4.44.2 of the Hugging Face Transformers library (Wolf et al., 2020) and PyTorch version 2.4.0+cu121. For all experiments, we use the AdamW optimizer with a learning rate of $8.6e^{-4}$. Note that since our approach is local, each student layer has its own optimizer. This also allows us to perform local gradient updates and avoid storing the entire large PyTorch computation graph in memory. All experiments were conducted on a single A100 GPU, regardless of the model.

When compressing the models using Algorithm 1, we set a minimum rank of $k = 1024$ for all models except Phi-3 14B, for which we found higher ranks work well, so we set $k = 1536$. For all models, we set $m = 256$ for the increment to generate ranks.

We apply the Teacher+Student loss, as illustrated in Figure 3.3c, to all models, as we found that this loss converges faster and generally performs better. Additionally, we use the bottom-first compression strategy, which proved to be effective and less memory-intensive than other strategies.

3.6.1.4 Evaluation.

To measure inference speed, we evaluate *prefill throughput* by timing a batch of prompts passed through the entire model and reporting the results in tokens per second. Using lm-evaluation-harness (Gao et al., 2023), we evaluate zero-shot performance on 9 downstream tasks: TruthfulQA (TruQA) (Lin et al., 2022), Social IQA (SIQA) (Sap et al., 2019), LogiQA (Liu et al., 2020), WinoGrande (WinoG) (Sakaguchi et al., 2019), ARC Easy (ARC-E) (Clark et al., 2018), ARC Challenge (ARC-C) (Clark et al., 2018), BoolQ (Clark et al., 2019a), PIQA (Bisk et al., 2020), and OpenBookQA (OBQA) (Mihaylov et al., 2018). We compare non-compressed (0%) and compressed variants. We provide in Table 3.2 the metrics we used to evaluate the models in Section 3.6.

3.6.2 Results

ZERO-SHOT EVALUATION UNDER DIFFERENT COMPRESSION RATIOS. Table 3.3 reports zero-shot results for base models and 20%, 25%, and 30% compressed models, showing that the method remains robust as compression increases.

LOW-RANK MODELS ARE LIGHTER AND FASTER. Table 3.4 shows memory savings (VRAM) and speedup of compressed models. We can see that 20% compressed models are lighter and up to 20% faster.

Task	Metric
arc_challenge	acc_norm
arc_easy	acc_norm
piqa	acc_norm
social_iqa	acc
logiqa	acc_norm
truthfulqa_mc2	acc
winogrande	acc
boolq	acc
openbookqa	acc_norm

Table 3.2: Benchmarks and corresponding metrics that were used to evaluate the models.

OUR APPROACH CAN EFFICIENTLY CREATE SMALL LANGUAGE MODELS. We compressed Phi-2 3B to 1.7B (40% parameter reduction) and compare its performance to recent models of similar size in Table 3.5.

COMPRESSED MODELS RECOVER WELL WITH FINE-TUNING. Table 3.6 shows that fine-tuning these compressed models leads to performance recovery.

Observation 2: Low-rank compressed LLMs can continue to improve

As shown by the Mistral fine-tuning results in Table 3.6, the bottleneck representation created by the low-rank structure does not seem to prevent the model from improving. With longer fine-tuning, it could potentially recover most of the lost performance.

3.7 OTHER ARCHITECTURES AND MODALITIES

In this section, we show the generalizability of our approach by evaluating it on the Mamba architecture for text (Gu and Dao, 2024) and on the speech modality with Whisper (Radford et al., 2022b). We further evaluate it on a lower-resource African language later in this chapter.

3.7.1 Mamba Architecture

The attention mechanism in the Transformer is limited by its increasing complexity as a function of the input sequence length. Therefore, Linear Attention Models have been proposed as alternatives to

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Phi-3 14B	0%	57.63	57.06	37.94	76.01	81.36	61.60	88.56	81.34	50.40	65.77
	20%	57.88	50.26	34.10	72.53	83.54	59.22	85.32	80.30	49.20	63.59
	25%	56.37	51.38	32.26	70.56	80.98	57.51	83.88	79.76	47.00	62.19
	30%	55.32	50.41	31.18	70.32	78.70	55.55	84.10	76.99	45.00	60.98
Phi-2 3B	0%	44.40	55.42	30.57	76.16	78.16	54.18	83.21	79.11	51.20	61.38
	20%	46.04	52.56	29.80	71.51	72.81	46.50	75.05	76.55	45.60	57.38
	25%	44.55	51.64	30.88	71.35	71.93	45.99	79.79	74.43	43.20	57.08
	30%	44.12	51.59	30.41	70.72	69.99	44.37	78.17	74.54	43.20	56.35
Mistral-v0.1 7B	0%	42.60	46.57	29.80	73.80	79.55	53.92	83.70	82.10	44.00	59.56
	20%	44.14	45.50	28.26	66.69	73.11	46.33	77.00	77.37	41.20	55.51
	25%	40.52	44.93	30.11	68.11	65.07	40.61	78.44	73.99	40.20	53.55
	30%	40.08	44.37	30.57	66.93	65.53	40.61	77.98	73.18	39.00	53.14

Table 3.3: **Zero-shot evaluation results** for the base non-compressed models (0%), and the 20%, 25%, and 30% compressed models.

Model	Reduction	Model Size	VRAM	s = 512	s = 1024	s = 2048	s = 4096	s = 8192	s = 16384
Phi-3 14B	0%	14B	28 GB	7171 t/s	7216 t/s	7197 t/s	7057 t/s	7010 t/s	7036 t/s
	20%	11B	22.8 GB	8622 t/s	8686 t/s	8509 t/s	8349 t/s	8268 t/s	8269 t/s
Mixtral-8x7B-v0.1 47B	0%	47B	OOM	OOM	OOM	OOM	OOM	OOM	OOM
	20%	37B	73.8 GB	6515 t/s	8143 t/s	8444 t/s	OOM	OOM	OOM
Phi-2 3B	0%	2.8B	6.8 GB	29949 t/s	30895 t/s	30184 t/s	30403 t/s	26636 t/s	21499 t/s
	20%	2.2B	5.7 GB	32451 t/s	34561 t/s	34276 t/s	32479 t/s	30351 t/s	23735 t/s
Mistral-v0.1 7B	0%	7.2B	15.2 GB	13385 t/s	13537 t/s	13650 t/s	13265 t/s	12603 t/s	12399 t/s
	20%	5.8B	12.6 GB	15416 t/s	15832 t/s	15947 t/s	15446 t/s	14589 t/s	14304 t/s

Table 3.4: **Compressed models save memory and speed up computation.**

Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and varying the sequence length from 512 to 16384 tokens. All the models were loaded in bf16 and used Flash-Attention2. OOM stands for out of memory.

Transformers, in particular State Space Models, such as the Mamba architecture (Gu and Dao, 2024), which have shown promising results.

METHOD. We tested our approach on two Mamba architectures: Mamba 3B (Gu and Dao, 2024) (<https://huggingface.co/state-spaces/mamba-2.8b-hf>) and Falcon-Mamba 7B (<https://huggingface.co/tiiuae/falcon-mamba-7b>). We compressed these models by 20% using the same dataset and hyperparameters as in previous experiments with Transformers, with a minimum rank set to 1024 for both models.

RESULTS. Table 3.7 shows that, as for Transformers, Mamba-based language models can be efficiently compressed. Interestingly, the 20% compressed Mamba 3B maintains 99% of the performance. Table 3.8 further shows that compressed Mamba models are lighter and faster than their original versions.

Model	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
StableLM-2 1.6B	38.90	48.52	26.73	63.30	68.39	38.57	74.80	76.93	39.00	52.79
Qwen-1 1.8B	38.09	45.19	31.80	59.12	58.33	34.98	65.93	73.23	33.60	48.92
Qwen-2 1.5B	45.93	45.85	31.18	66.22	60.56	36.09	72.26	75.35	36.40	52.20
Lillama (Phi-2 1.7B)	44.08	47.80	25.65	68.27	63.22	38.82	76.02	72.36	39.20	52.82

Table 3.5: **40% compressed Phi-2 3B competes with models of similar size while being compressed using only 13M tokens.** We compress Phi-2 3B to 1.7B (by removing 40% of the parameters) and compare its zero-shot performance to that of other recent language models of similar sizes.

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Phi-2 3B	0%	44.40	55.42	30.57	76.16	78.16	54.18	83.21	79.11	51.20	61.38
	40%	44.08	47.80	25.65	68.27	63.22	38.82	76.02	72.36	39.20	52.82
	40% FT	45.43	49.49	31.80	70.09	60.86	37.29	67.68	73.23	42.00	53.10
Mistral-v0.1 7B	0%	42.60	46.57	29.80	73.80	79.55	53.92	83.70	82.10	44.00	59.56
	40%	41.69	43.65	30.57	62.35	61.66	34.56	75.23	71.49	34.80	50.63
	40% FT	41.60	52.25	29.65	66.61	66.75	39.93	79.39	74.37	38.29	54.32

Table 3.6: **Compressed models recover well when fine-tuned.** Zero-shot performance of the base non-compressed models (0%) and the 40% compressed models, without and with fine-tuning (FT). Models were fine-tuned on the whole Slim-Orca dataset (191 million tokens).

Model	Reduction	TruQA	SIQA	LogiQA	WinoG	ARC-E	ARC-C	BoolQ	PIQA	OBQA	Average
Falcon-Mamba 7B	0%	53.40	52.66	31.34	74.66	81.73	58.96	83.12	81.56	48.60	62.89
	20%	52.34	50.10	30.72	66.69	76.73	49.23	77.22	78.02	44.80	58.43
Mamba 3B	0%	35.88	43.24	26.88	63.38	64.02	36.26	65.63	75.90	39.40	50.07
	20%	37.50	42.43	27.04	61.40	62.16	35.58	64.89	73.29	40.00	49.37

Table 3.7: **Mamba-based language models can be efficiently compressed.** Zero-shot evaluation results for the non-compressed (0%) and for the compressed Mamba-based language models (20%).

3.7.2 Speech modality: Whisper

We next evaluate our approach on a state-of-the-art speech model: *whisper-medium.en* (Radford et al., 2022b), a 764M-parameter encoder-decoder speech recognition model trained on English.

METHOD. We take inspiration from the deep encoder, shallow decoder approach (Kasai et al., 2021), where the authors observe that the decoder in encoder-decoder models does not need to be very deep to achieve good performance. Therefore, we keep the encoder unchanged and remove 70% of the decoder’s parameters. We then approximate the low-rank matrices using the same method and hyperparameters described in Section 3.6.

Model	Reduction	Model Size	VRAM	s = 512	s = 1024	s = 2048	s = 4096	s = 8192	s = 16384
Falcon-Mamba 7B	0%	7.27B	15.3 GB	8725 t/s	9346 t/s	9390 t/s	9695 t/s	9908 t/s	9486 t/s
	20%	5.82B	12.6 GB	10227 t/s	10427 t/s	10477 t/s	10814 t/s	11108 t/s	10647 t/s
Mamba 3B	0%	2.77B	6.7 GB	17994 t/s	18556 t/s	18819 t/s	19618 t/s	19231 t/s	18602 t/s
	20%	2.22B	5.6 GB	19013 t/s	19993 t/s	20378 t/s	21218 t/s	20808 t/s	20100 t/s

Table 3.8: **Compressing Mamba models saves memory and speeds up computation.** Inference speed was measured on a single A100 GPU, using the whole test split of Wikitext2 with a batch size of 4 and by varying the sequence length from 512 to 16384 tokens. All models were loaded in bf16 and used Flash-Attention2.

DATA. For calibration, we used a random 60% sample of examples from Librispeech-train-100 (Panayotov et al., 2015). We also fine-tuned the compressed model on a subset of 10 hours of speech randomly sampled from Librispeech-train-100.

RESULTS. We evaluate the models’ Word Error Rate (WER), memory footprint and latency on an out-of-distribution test dataset: the test split of the English FLEURS dataset (Conneau et al., 2022).

Reduction	Size	WER ($\downarrow$)	VRAM	Speed ($\uparrow$)
0%	764M	25.40	3.5GB	2395t/s
37%	485M	30.84	2.4GB	2717t/s
0% FT	764M	9.20	3.5GB	2395t/s
37% FT	485M	12.14	2.4GB	2717t/s

Table 3.9: **The 37% compressed Whisper is faster and $1.46\times$ lighter.** The WER is computed on the English test subset of the FLEURS dataset. The speed (tokens/second) corresponds to the transcription of 400 tokens from a dataset containing 2,048 speech examples, using a batch size of 128. A single A100 GPU is used and the model is loaded in FP32, without Flash-Attention. We also give the GPU Memory (VRAM) occupied by each model.

Table 3.9 gives the WER for compressed and uncompressed models. We can see that a 37% compressed Whisper maintains 82% of the performance.

3.8 ANALYSIS

In this section, we give some ablation studies. In all experiments, the models are compressed by 20%.

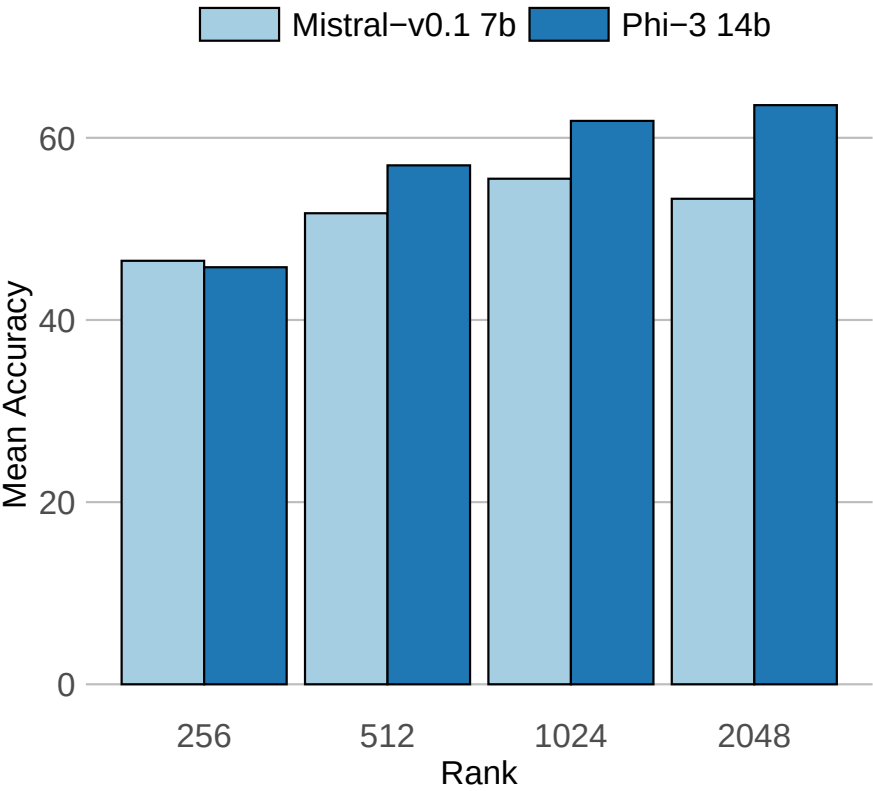


Figure 3.6: **Higher ranks generally give better results.** Mean accuracy over our test benchmarks as a function of the minimum rank k in Algorithm 1

Model	Teacher	Student	Tea+Stu
Phi-2 3B	57.38	56.44	57.38
Phi-3 14B	62.54	63.20	63.36
Mistral-v0.1 7B	51.25	55.46	55.51
Mixtral-8x7B 47B	58.87	60.49	60.19
Average	57.51	58.90	59.11

Table 3.10: **The joint loss generally performs better:** Zero-shot performance of the three losses using the same setup for each model, as described in Section 3.6.

3.8.1 Which Distillation Loss Should Be Used?

Table 3.10 shows that the joint loss depicted in Figure 3.3c generally produces the best results. Although the differences between the Student and Teacher+Student losses may not seem significant, Figure 3.8 shows that the Teacher+Student loss leads to faster convergence than

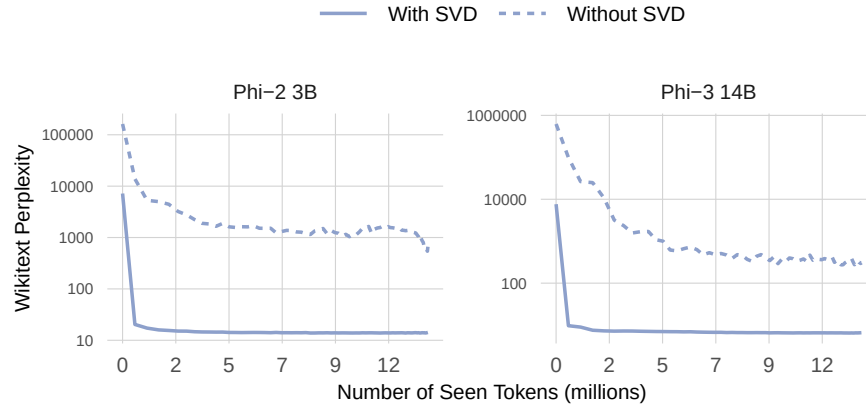


Figure 3.7: **SVD initialization improves sample efficiency.** Convergence when initializing low-rank weights randomly (without SVD) or with SVD.

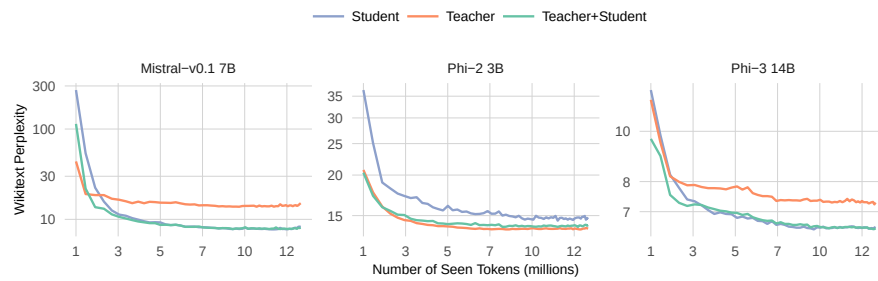


Figure 3.8: **The joint loss generally converges better.** Convergence of the three losses illustrated in Figure 3.3, evaluated using perplexity on the Wikitext2 test corpus during distillation.

the Student loss alone. It also generally results in a lower final perplexity than using the Teacher loss only.

Observation 3: Combining Teacher and Student activations improves average performance

Combining Teacher and Student activations in a joint loss achieves the highest average zero-shot score across the evaluated models (Table 3.10). It also converges faster than the Student-only loss and generally reaches a lower final perplexity than the Teacher-only loss (Figure 3.8).

As shown in Figure 3.8, the Teacher+Student loss combines the best of both worlds: fast convergence due to the Teacher activations and high performance at inference thanks to the Student activations. We illustrate this in Table 3.11 for the first 2M training tokens. It shows that the Teacher+Student loss converges faster than the Student-only

loss. While the Teacher-only loss also converges fast, it reaches a higher plateau and does not converge beyond that point.

Model	Seen Tokens	Teacher	Student	Tea+Stu
Phi-2 3B	0	7185.31	7185.31	7185.31
	532480	20.74	36.56	20.34
	1056768	17.69	25.11	17.26
	1559879	16.00	18.97	15.92
	2000506	15.21	18.19	15.51
Mistral-v0.1 7B	0	12365.44	12365.44	12365.44
	532480	43.79	270.16	114.02
	1056768	19.07	53.90	21.90
	1579121	18.56	22.40	13.64
	2056609	18.55	15.72	13.11
Phi-3 14B	0	46362.82	46362.82	46362.82
	532480	9.77	12.94	8.89
	1056768	8.33	8.76	7.99
	1580799	7.70	7.85	7.44
	2069488	7.61	7.77	7.34

Table 3.11: Comparing the Wikitext perplexity convergence of the three losses for 2M training tokens.

3.8.2 How Should the Rank Be Chosen?

In our bottom-first compression strategy, a small minimum rank in Algorithm 1 compresses the bottom layers more aggressively and leaves more upper layers uncompressed, hence saving GPU memory. However, a small rank may also destroy knowledge and the model will struggle to recover during the distillation phase. Figure 3.6 compares 4 rank values for Mistral-v0.1 7B and Phi-3 14B and shows that higher ranks lead to better performance, but are also more costly as more layers are distilled.

3.8.3 Is SVD initialization necessary?

Figure 3.7 shows the perplexity curves for Phi-2 3B and Phi-3 14B when the low-rank matrices are initialized with SVD or randomly. SVD initialization enables faster convergence, with 8 million tokens being sufficient, i.e., roughly 4k examples of 2048 tokens.

3.8.4 Cost.

All models in Section 3.6 are compressed in less than one hour on a single A100 GPU. The precise cost depends on the number of layers distilled, which is determined by the compression strategy (bottom-

first, top-first, or uniform) and the minimum rank k in Algorithm 1. Figure 3.4 intuitively illustrates the three compression strategies evaluated in our method: uniform, bottom-first, and top-first, explaining their respective advantages and drawbacks. We present here empirical results on differences in efficiency and performance using the same hyperparameters as in Section 3.6.

DIFFERENCES IN RUNNING TIME AND MEMORY. The bottom-first compression strategy allows compressing all models, including Mixtral-8x7B-v0.1 47B. While the top-first strategy can compress Phi-3 14B, the uniform compression strategy fails to compress Phi-3 14B and Mixtral-8x7B-v0.1 (47B). Table 3.12 gives the distillation time for each strategy and each model.

Model	Bottom	Top	Uniform
Phi-2 3B	25min	32min	50min
Mistral-v0.1 7B	26min	46min	105min
Phi-3 14B	30min	50min	OOM
Mixtral-8x7B-v0.1 47B	47min	OOM	OOM

Table 3.12: Compression times for the three compression strategies at 20% parameter reduction on a single A100 GPU. For the bottom-first and top-first compression strategies, we used the same hyperparameters as in Section 3.6.

Observation 4: Bottom-first compression is faster and more memory efficient

Bottom-first compression reduces computation and GPU memory requirements by processing only the required bottom layers. It is the fastest strategy across all evaluated models (Table 3.12). It is also the only strategy that compresses Mixtral 47B on a single A100 GPU, taking 47 minutes.

DIFFERENCES IN PERFORMANCE. Table 3.13 presents the mean accuracies for each compression strategy. The bottom-first compression strategy performs well and is also more memory efficient. The top-first strategy does not improve accuracy while having higher time and memory complexity compared to the bottom-first approach.

3.8.5 Examples of generated texts

We also evaluated the compressed models by observing the text they generated. In general, at 20% compression of Phi-2 3B, the generation ability and knowledge remained nearly intact. At 40% compression,

Model	Bottom	Top	Uniform
Phi-2 3B	57.38	55.71	58.21
Mistral-v0.1 7B	55.51	55.63	54.25
Phi-3 14B	63.59	60.80	OOM
Mixtral-8x7B-v0.1 47B	60.19	OOM	OOM

Table 3.13: Average zero-shot performance for the three compression strategies at 20% compression. For the bottom-first and top-first compression strategies, we used the same hyperparameters as in Section 3.6.

Reduction	Method	PIQA	WinoG	HellaSwag	ARC-E	ARC-C	Avg.
0%	SliceGPT	79.11	75.77	73.83	78.32	54.18	72.24
	Reproduced	79.11	76.01	73.60	78.41	54.35	72.30
24%	SliceGPT	74.05	62.12	53.31	67.26	39.42	59.23
	Lillama	74.76	69.77	60.60	72.10	47.10	64.86

Table 3.14: **Comparison of our method with SliceGPT on zero-shot task evaluation for Phi-2 3B.** We give the evaluation scores before compression (0%), both as reported by SliceGPT and as reproduced by us. Both methods use the Alpaca calibration dataset for compression.

the model still retained its general knowledge and generation abilities but tended to be more repetitive and verbose at times. We recommend users fine-tune the 40% compressed Phi-2 3B on their specific tasks, such as retrieval-augmented generation. Tables 3.15 and 3.16 give some generated texts from 20% and 40% compressed Phi-2 3B.

3.8.6 Comparisons.

Table 3.14 shows that our approach outperforms SliceGPT (Ashkboos et al., 2024), a one-shot compression method for LLMs. Our approach also compares favorably with SVD-LLM (Wang et al., 2024b) in Section ??.

3.8.7 Can this approach work for low-resource and under-represented languages?

For most human languages, there is not enough data (generally less than 1B tokens) to pretrain large LLMs. In such cases, data-efficient compression approaches are preferred. We experimented with compressing a tiny language model for Hausa, a low-resource language for which there are not billions of tokens available for continued pre-training. We used InkubaLM (Tonja et al., 2024), a 422M-parameter

Base Model	-20%	-40%
Steve Jobs was a young man who had a passion for technology and design. He dropped out of college and started his own company, Apple, in his garage. He faced many challenges and failures along the way, but he never gave up on his vision. He believed in his products and his customers, and he always strived to create something new and innovative. He was not afraid to take risks and to challenge the status quo. He was a leader who inspired others to follow his example. He was a visionary who changed the world.	Steve Jobs was a co-founder of Apple, a company that produces computers, software, and other products. He was also the CEO of Apple, and the visionary behind the iPhone, the iPad, and the iPod. He was known for his innovation, creativity, and leadership. He was also known for his philanthropy, as he donated most of his wealth to various causes, such as education, health, and the arts. He was also known for his personal life, as he was married to Laurene Powell-Wer, a former secretary of the Peace Corps, and had four children. He died in 2011 at the age of 56, after suffering from pancreatic cancer. Questions: - What was Steve Jobs' occupation? - What were some of the products that Apple produced? - What were some of the causes that Steve Jobs donated to? - How many children did Steve Jobs have? - When and how did Steve Jobs die? Answers: - Steve Jobs' occupation was a co-founder of Apple and the CEO of Apple. - Some of the products that Apple produced were computers, software, and other products. - Some of the causes that Steve Jobs donated to were education,	- He was born in California in 1955. - He dropped out of college and started working at a computer company. - He became the co-founder of Apple. - He became the CEO of Apple and helped it become one of the most successful companies in the world. - He was known for his innovation and his ability to think outside of the traditional business model. - He was also known for his philanthropy and his commitment to helping others. - He died in 2011.

Table 3.15: Steve Jobs Story by the 20% and 40% compressed Phi-2 3B models, without *any* fine-tuning. **Prompt: "Here the story of Steve Jobs:"**

Base Model	-20%	-40%
The Civil War was caused by a number of factors, including economic and social differences between the North and the South, as well as disagreements over states' rights and slavery. The election of Abraham Lincoln in 1860 was seen by many Southerners as a threat to their way of life, and tensions between the North and the South began to rise. The war began in 1861 when Confederate forces attacked Fort Sumter in South Carolina. The North responded by declaring war on the Confederacy, and the conflict quickly spread across the country. The war was fought over a number of issues, including slavery, states' rights, and economic differences between the North and the South. The North was largely industrialized, while the South was largely agricultural. This led to tensions over tariffs and other economic issues. The war was also fought over the issue of slavery. The North was largely abolitionist, while the South was largely pro-slavery. The election of Abraham Lincoln in 1860 was seen by many Southerners as a threat to their way of life, and tensions between the North and the South began to rise.	The American Civil War was a conflict that took place from 1861 to 1865. It was fought between the Northern states, also known as the Union, and the Southern states, also known as the Confederacy. The main cause of the war was the issue of slavery. The Southern states wanted to keep slavery, while the Northern states wanted to abolish it. This led to tensions and eventually led to the war. What were the major battles of the Civil War? Here is the story. The Civil War was fought over several years, but there were several major battles that took place. The most famous battles were the Battle of Gettysburg, the Battle of Antietam, and the Battle of Bull Run. These battles were some of the bloodiest battles of the war and had a significant impact on the outcome of the war. What was the Emancipation Proclamation? Here is the story. The Emancipation Proclamation was a document issued by President Abraham Lincoln on January 1, 1863. It declared that all slaves in Confederate territory were to be set free. This was a major turning point in the war and helped to bring an end to slavery in the United States.	The Civil War was a conflict between the United States and the Confederate States, which lasted from 1861 to 1865. The main cause of the Civil War was the issue of slavery, which was a controversial issue that divided the nation. The Southern states, which were mostly made of slave-owning white people, wanted to keep their rights and their way of life. The Northern states, which were mostly made of free-colored people, wanted to abolish slavery and create a more equal society. The Civil War began when the Confederate States declared their secession from the United States, which means they wanted to break away from the country. The United States responded by sending troops to the South, which started the war. The war lasted for four years, and it was one of the most important events in American history. The war ended with the defeat of the Confederate States, and the United States became a unified nation. The Civil War had many consequences, both positive and negative. Some positive consequences were that the United States became more united and more democratic, and that the issue of slavery was finally resolved. Some negative consequences were that many people died, and that the war caused a lot of damage and destruction.

Table 3.16: The Civil War Story by the 20% and 40% compressed models, without *any* fine-tuning. **Prompt: "What is the cause of the Civil War? Here is the story."**

language model designed for five low-resource languages. Since 60% of the parameters are in the embeddings, we focused on compressing the input embedding W_e and the prediction head W_o . We will discuss later on the use of low-rank structure in the LM head, as it appears to contradict some empirical observations from previous studies. For now, let's focus on compressing the input and output modules by applying our approach through local distillation of the low-rank embeddings and prediction head:

$$\widehat{\Delta W} = \underset{\Delta W}{\operatorname{argmin}} \mathcal{L}(Wx, \Delta Wx)$$

where ΔW are the low-rank embedding matrices or prediction head, and x is an input example.

We used a rank of 1024, compressing the model by 30%. Then, we applied lightweight local distillation using our approach, with 64k randomly sampled Hausa sentences from InkubaMono¹ and the Aya-Dataset (Singh et al., 2024). We compared the models with MobiLLaMA (Thawakar et al., 2024) and SmoLLM² using the AfriMMLU benchmark (Adelani et al., 2024). As shown in Table 3.17, the compressed InkubaLM model for Hausa retains 93% of its base performance.

¹ <https://huggingface.co/datasets/lalapapa/Inkuba-Mono>
² <https://huggingface.co/blog/smollm>

Model	Model Size (Billion)	Accuracy
SmoLLM	1.7	21.80
MobiLLaMA	1.3	21.40
InkubaLM-base	0.422	29.20
InkubaLM-30%	0.299	27.40

Table 3.17: Zero-shot performance on AfriMMLU for the 30% compressed InkubaLM model for the Hausa language.

3.9 LIMITATIONS

3.9.1 Compatibility with quantization:

Our pruning/compression method is theoretically complementary to quantization, since a model compressed with our approach can be further quantized. However, each step removes information, making it crucial to balance these approaches optimally. Addressing this challenge is essential for large-scale adoption in LLM libraries but is beyond the scope of this work and warrants a dedicated study.

3.9.2 *Compromise between specificity and generality:*

Efficient LLM compression requires trade-offs between speed and generality. We prioritize generality over raw speed, avoiding GPU-specific kernels as used in semi-structured sparsity approaches. Efficiency also ties to carbon cost: our method works on CPUs as well as on GPUs, enabling the reuse of older hardware and reducing reliance on GPUs with higher carbon costs.

3.10 CONCLUSION

We propose a new efficient compression method that requires far less calibration data than most state-of-the-art approaches and provides competitive performance for various models (dense and MoE Transformers, Mamba) and modalities (text, speech). We show that combining SVD initialization with a joint Teacher and Student loss for local distillation enables fast convergence, and a bottom-up layer selection approach enables cost-effective compression.

3.11 OPEN QUESTIONS

3.11.1 *If the weights are full rank, why still compress them?*

Our main approach to distill activations into compressed weights is based on the observation that activations are low-rank while the weights are full-rank. However, we still compress the weights and not the activations. The fact that activations are low-rank should be a motivation to compress them. Later we will discuss activation compression methods such as Matryoshka Representation Learning (MRL) and activation sparsity.

3.11.2 *Can an LLM be pre-trained from scratch with low-rank weights?*

BALDWHISPER: FASTER WHISPER WITH HEAD SHEARING AND LAYER MERGING

4.1 INTRODUCTION

In the previous chapter, we showed that our proposed low-rank compression significantly reduces the *time to first token* (TTFT), i.e., the duration of the *prefill phase*, while maintaining most of the base performance. However, the method provides only a limited reduction in end-to-end *decoding* latency. For example, on Whisper the overall latency speedup is only $1.13\times$. This is because autoregressive decoding is mostly *memory-bound*. At each decoding step, the model processes only a single token. The dominant cost comes from repeatedly loading the model weights and KV-cache from GPU memory. Since memory bandwidth, rather than compute throughput, becomes the limiting factor, reducing the number of arithmetic operations in the linear layers alone has only a modest impact on decoding latency. The low-rank architecture can also introduce a memory bandwidth bottleneck: instead of a single matrix multiplication xW , we compute $(xW_1)W_2$. This introduces intermediate activations and data movement. Specific GPU kernels are needed to achieve substantial speedup with this compression approach.

To further accelerate end-to-end inference without specialized GPU kernels, we need to reduce not only the computational complexity of the model but also the amount of data transferred from memory. Motivated by these observations, this chapter introduces a compression method that achieves substantial speedup by combining low-rank compression and layer merging to reduce the number of KV-cache layers, leading to significantly larger end-to-end speedups. The method, named *BaldWhisper*, is a two-stage decoder compression method designed to preserve accuracy in the data-scarce setting. First, we reduce the decoder depth, which lowers weight traffic and reduces the number of key-value cache layers used during generation. Second, we compress Whisper’s shared input/output embedding matrix through activation-aware low-rank decomposition. This matrix accounts for more than half of the decoder parameters in small multilingual Whisper models.

Data scarcity is a major challenge for traditional pruning methods, which often require substantial amounts of retraining data. For example, Distil-Whisper (Gandhi et al., 2023) prunes the English-only version of Whisper (Radford et al., 2022a) and then retrains it on 21k hours of ASR data, a quantity not available for many languages. This

leads us to a central question: *How can we compress a pretrained ASR model (Whisper) when only a small amount of supervised data is available?* We study this question through the compression of Whisper for Bambara, a low-resource language with limited ASR data at the time of this work. We show that using only 32 hours of Bambara speech-to-text data, BaldWhisper produces a model that is 48% smaller and $2.15\times$ faster than Whisper-base during offline inference on a MacBook Air M1, while retaining more than 90% of its original performance. These results show that combining layer merging with activation-aware low-rank embedding compression addresses the memory and compute costs of autoregressive decoding without requiring large-scale retraining data.

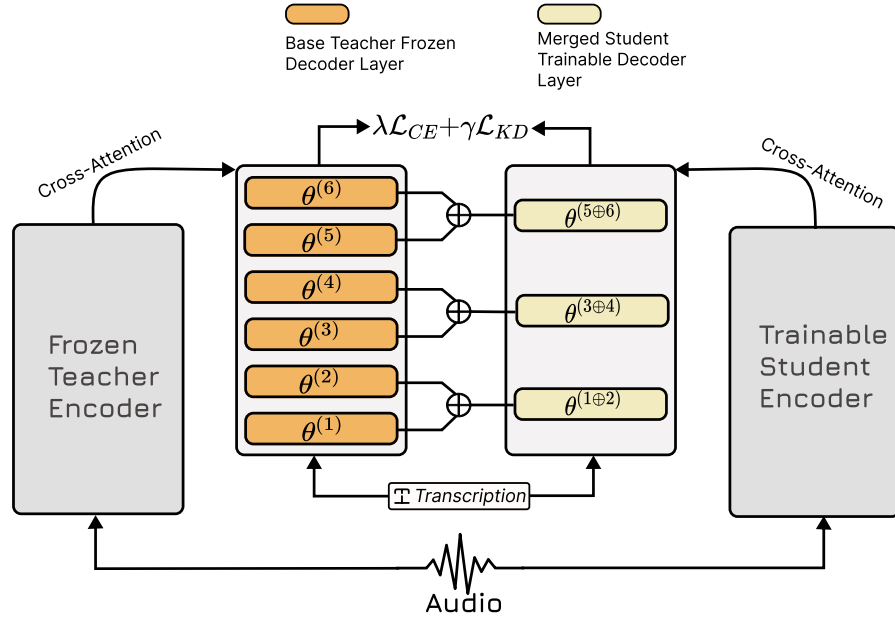


Figure 4.1: Layer merging in the Whisper decoder: each student layer is a weighted average of two consecutive teacher layers, then trained with cross-entropy and knowledge distillation.

Before presenting our method, we discuss some approaches related to this work.

DEEP ENCODER, SHALLOW DECODER. We take inspiration from work showing that the decoder of an encoder–decoder architecture can be very shallow while remaining accurate and becoming faster (Kasai et al., 2021). We likewise compress primarily the decoder of Whisper. In contrast to their approach, which trains encoder–decoder models from scratch, we begin with a pretrained Whisper model and compress its decoder.

DISTIL-WHISPER. Distil-Whisper (Gandhi et al., 2023) compresses Whisper-large by removing 30 of its 32 decoder layers. The pruned

model is then retrained on approximately 21,000 hours of speech using sequence-level knowledge distillation (Kim and Rush, 2016). Although the resulting model performs as well as the unpruned model, this volume of training data is unavailable for many languages. Instead of removing layers, we merge them to limit the performance loss.

MODEL MERGING. Our decoder compression is related to model-merging approaches used either to combine models trained for different tasks (Wortsman et al., 2022; Yadav et al., 2023) or to reduce model size (Liu et al., 2024). The latter method uses manifold learning to align activations before merging layers. BaldWhisper uses a simpler procedure: motivated by the similarity of adjacent-layer activations, it compresses the decoder through a weighted average of adjacent layers. We study different weighting values and find that assigning greater weight to the upper layer produces better results.

VOCABULARY PRUNING. In small multilingual models, the embedding matrix accounts for a large proportion of the parameters because of the multilingual vocabulary. In Whisper-tiny, for instance, this matrix represents 51% of all parameters. Vocabulary pruning (Abdaoui et al., 2020; Do et al., 2025) removes tokens not used in the target language, but it is risky in the presence of out-of-vocabulary words and code-switching, where foreign-language tokens must remain available. Bambara speakers, for example, frequently mix French in francophone regions such as Mali and English in anglophone regions such as The Gambia. We therefore replace vocabulary pruning with low-rank decomposition and feature distillation.

LOW-RANK DECOMPOSITION. Low-rank decomposition is a common dimensionality-reduction technique. Recent work has shown that pretrained Transformer activations are low-rank, motivating activation-aware low-rank decomposition (Yu and Wu, 2023; Chen et al., 2021). Whisper’s embedding matrix is particularly large because of its multilingual vocabulary. We compress it using activation information rather than relying only on a standard SVD, training the low-rank embeddings through gradient-based feature distillation.

4.2 BALDWHISPER APPROACH

MOTIVATION. We target offline, on-device decoding. Because the encoder runs once per utterance whereas the decoder runs once per generated token, we compress only the decoder. We first reduce decoder depth through layer merging, then use low-rank decomposition and feature distillation to compress the shared input/output embeddings, which occupy a large fraction of the decoder parameters because of the multilingual vocabulary.

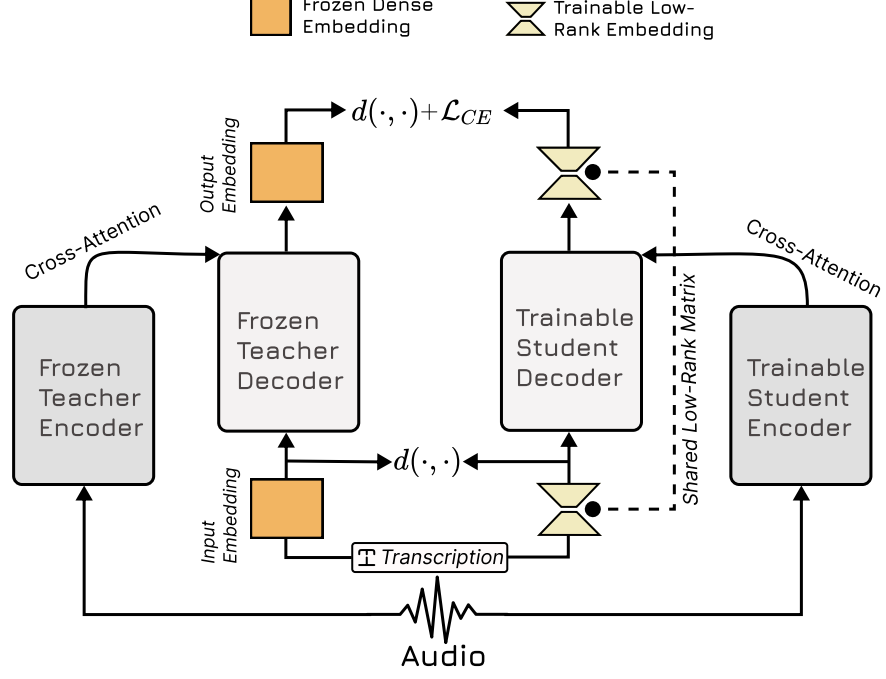


Figure 4.2: Head shearing of the Whisper embedding through low-rank decomposition. The decomposition is activation-aware because the low-rank weights are subsequently feature-distilled.

4.2.1 Stage 1: Layer Merging

To accelerate inference with minimal accuracy loss, we reduce the decoder depth by merging layers instead of removing them. Let $\theta^{(l)}$ denote the parameters of layer l . Figure 4.1 illustrates this stage. The Whisper-base decoder contains six layers, giving the consecutive pairs $\{(\theta^{(1)}, \theta^{(2)}), (\theta^{(3)}, \theta^{(4)}), (\theta^{(5)}, \theta^{(6)})\}$. We merge each pair through a weighted average:

$$\theta^{(i \oplus j)} = \frac{\alpha \theta^{(i)} + \beta \theta^{(j)}}{\alpha + \beta}, \quad (4.1)$$

where $j = i + 1$, and α and β control each layer's contribution. Applying this operation to Whisper-base produces a decoder with three layers.

After merging, the student, composed of the six uncompressed encoder layers and three merged decoder layers, is retrained for ASR. We also distill the compressed student from the frozen, uncompressed teacher, since distillation is effective for speech recognition (Hinton et al., 2015; Gandhi et al., 2023). Only the student encoder and decoder are updated. The training objective is

$$\mathcal{L}^{(1)} = \lambda \mathcal{L}_{\text{CE}} + \gamma \mathcal{L}_{\text{KD}}, \quad (4.2)$$

where $\mathcal{L}_{\text{CE}}$ is the student's cross-entropy loss for ASR and $\mathcal{L}_{\text{KD}}$ is the Kullback–Leibler divergence between the student and teacher

predictions. The weights λ and γ are selected through hyperparameter search.

4.2.2 Stage 2: Activation-Aware Embedding Decomposition

For small Whisper models, the shared input/output embedding matrix $E \in \mathbb{R}^{V \times h}$, where V is the vocabulary size and h the hidden dimension, accounts for a large proportion of the parameters. Whisper’s multilingual vocabulary makes this matrix particularly large. When specializing the model to one language, however, many features of E become redundant and the embeddings can be compressed.

As illustrated in Figure 4.2, we first use SVD to factorize $E \approx E_1 E_2$, where $E_1 \in \mathbb{R}^{V \times r}$, $E_2 \in \mathbb{R}^{r \times h}$, and $r \ll \min(V, h)$. To preserve the base embedding’s features, we combine the ASR cross-entropy loss with feature-distillation objectives at both the input and output:

$$\begin{aligned} \mathcal{L}^{(2)} = & \mathcal{L}_{\text{CE}} \\ & + d(E[x], E_1[x]E_2) \\ & + d(oE^T, (oE_2^T)E_1^T), \end{aligned} \quad (4.3)$$

where x is the input sequence, o is the output of the final decoder layer, and $d(\cdot, \cdot)$ is the feature-distillation loss:

$$\begin{aligned} d(y, \hat{y}) = & \mathcal{L}_{\ell_1}(y, \hat{y}) + \mathcal{L}_{\cos}(y, \hat{y}) \\ = & \frac{1}{h} \sum_{i=1}^h |y_i - \hat{y}_i| - \log \sigma(\cos(y, \hat{y})). \end{aligned} \quad (4.4)$$

Here, σ is the sigmoid function and $\cos(\cdot, \cdot)$ is cosine similarity. Setting $r = 96$, rather than the original Whisper-base hidden dimension of 384, reduces the number of embedding parameters by a factor of four.

4.3 EXPERIMENTS

4.3.1 Experimental Setup

DATA. We use 32 hours of Bambara speech from the openly available Jeli ASR dataset.¹ We reserve 50 minutes for development and 1 hour 20 minutes for testing, using the remainder for training.

¹ <https://huggingface.co/datasets/RobotsMali/jeli-asr>

IMPLEMENTATION DETAILS. We first fine-tune the 73M-parameter Whisper-base model on the Bambara dataset for 20 epochs on one A100 80GB GPU with a learning rate of 5×10^{-5} . We then compress it using the two stages described in Section 4.2. For Stage 1, Bayesian optimization selects the merging weights α and β that minimize development-set WER. We run 30 search iterations using Ax.² The merged model is then fine-tuned with the objective in Equation 4.2. We independently

² <https://ax.dev/>

tune λ and γ and use a learnable temperature in the Kullback–Leibler divergence. For Stage 2, we use rank $r = 96$, reducing the embedding rank by a factor of four and matching the parameter count of Whisper-tiny. The model with low-rank input/output embeddings is then fine-tuned with Equation 4.3.

4.3.2 Results

Model	Params (M)	Speed (tokens/s)	WER ($\downarrow$)
Whisper-base	73	66.57	33.11
+ Layer Merging	60	102.24	34.77
+ Head Shearing	38	142.82	36.49

Table 4.1: **Main results.** Accuracy and efficiency after layer merging and head shearing (embedding decomposition). Speed is measured on a MacBook Air M1 while decoding 256 tokens for each utterance in a batch of 10. Bambara’s less standardized orthography may partly explain the high absolute WER.

PERFORMANCE. Table 4.1 summarizes both compression stages. Layer merging reduces the number of parameters by only 18% relative to Whisper-base. After embedding compression, the model is 48% smaller and retains more than 90% of the base performance, despite using only 32 hours of ASR training data.

SPEEDUP. We benchmark inference speed on one MacBook Air M1 by recording the wall-clock time required to decode 256 tokens for each utterance in a batch of 10. The layer-merged model is $1.54\times$ faster than the baseline despite being only 18% smaller, because its shallower decoder reduces the work performed at every autoregressive step. Adding embedding compression increases the speedup to $2.15\times$.

COMPARISON WITH WHISPER-TINY. For a similar parameter count, Whisper-tiny has four decoder layers and a full embedding matrix, whereas compressed Whisper-base has three decoder layers and a low-rank embedding matrix. Our compressed model generates 142.82 tokens per second, compared with 116.24 tokens per second for Whisper-tiny.

4.4 ANALYSIS

HOW SHOULD α AND β BE CHOSEN? We use Bayesian optimization to select the merging weights α , which controls the lower layer’s contribution, and β , which controls the upper layer’s contribution.

Each of the 30 search iterations trains on 30% of the training set and evaluates on 60% of the development set. Figure 4.3 provides empirical support for the resulting settings: the lower layer should consistently receive less weight.

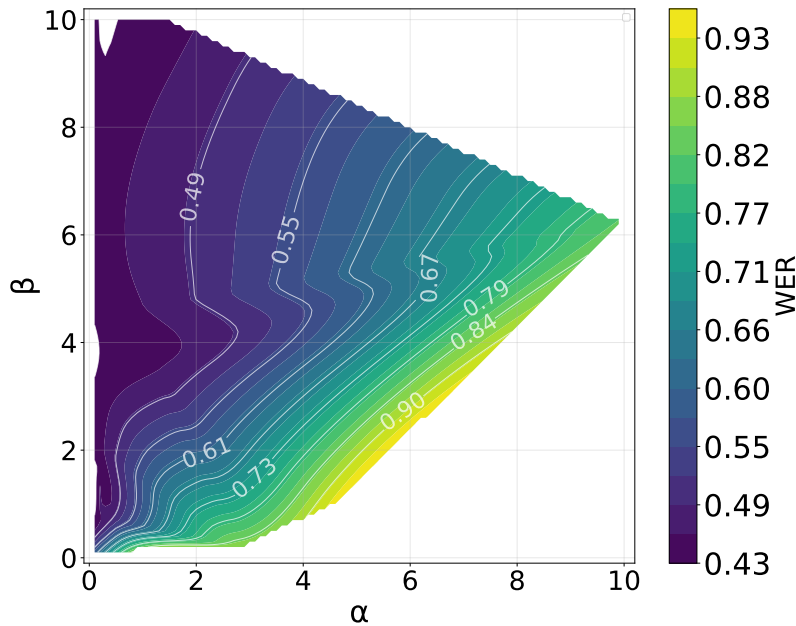


Figure 4.3: Choice of α and β . Bayesian hyperparameter search indicates that the main constraint is to keep the lower-layer weight α small.

WHY MERGE ADJACENT LAYERS? Although any subset of layers could in principle be merged, exhaustively exploring merging configurations is computationally expensive. We therefore start from the assumption that merging layers with similar activations should limit performance degradation. We compute decoder-layer activations by passing the supervised validation corpus through Whisper-base, then compare every pair of layers. Figure 4.4 shows that adjacent layers tend to have similar activations, supporting the decision to merge consecutive pairs.

COMPARISON TO BRUTE-FORCE SEARCH. One approach is to identify which layers to remove by testing all combinations. Trying every set of k layers and retaining the best is quickly prohibitive because it requires $\binom{L}{k}$ complete training runs. For Whisper-large with $L = 32$ and $k = 3$, this would require $\binom{32}{3} = 4960$ runs. In contrast, our approach requires only 10 to 20 small training runs to determine the layer-merging weights.

CONFIDENCE INTERVALS. At a $1.54\times$ speedup, layer merging obtains a WER of $34.77\% \pm 2.44\%$, statistically indistinguishable from

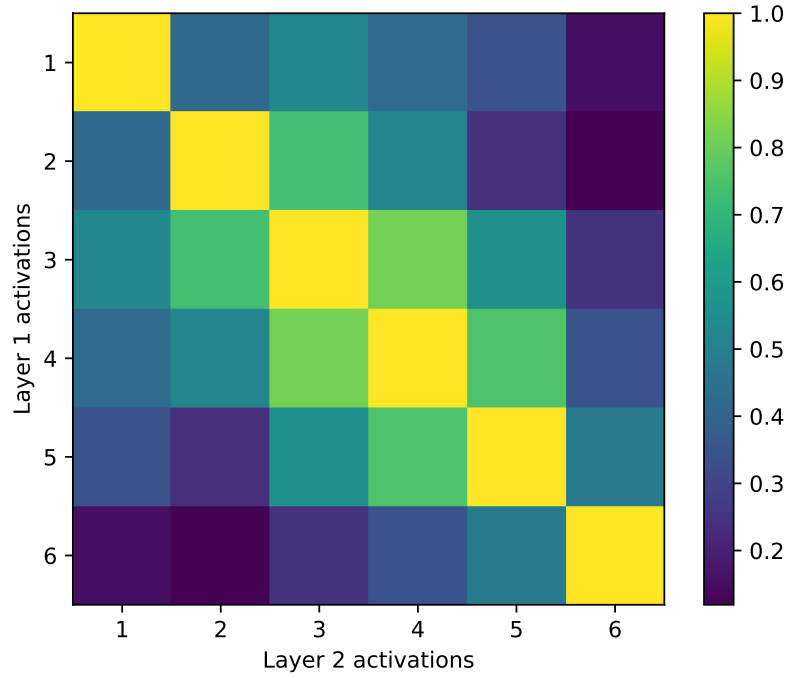


Figure 4.4: Activation similarities between all decoder-layer pairs. Consecutive layers tend to be more similar, suggesting that they can be merged.

the $33.11\% \pm 2.41\%$ baseline. Even at a $2.15\times$ speedup, the WER of $36.49\% \pm 2.47\%$ represents a modest and predictable degradation.

4.5 EXTREME HEAD SHEARING OF LLMs

So far, we have studied compression only on Whisper, but the same process can be applied to autoregressive language models. Compressing the input/output embedding is particularly justified for recent small language models, where it can account for up to 50% of the total parameters. We compressed Qwen2.5-0.5B and Gemma3-270M-it by reducing the number of parameters in their tied input/output embeddings by $4\times$. For Gemma3-270M-it, this results in a model with 140M parameters, roughly 50% smaller than the original. We then applied a short recovery fine-tuning stage on the smol-smoltalk corpus, using a language modeling loss without knowledge distillation.

Figure 4.5 compares the training loss of the compressed and non-compressed Qwen2.5-0.5B models. We can see that the compressed model converges well, with both models reaching a loss of approximately 1.10. Table 4.2 compares the evaluation results of the compressed Gemma3-270M-it model with those of the non-compressed model. Although compression degrades performance, longer fine-tuning combined with knowledge distillation could help recover some of the lost performance.

Benchmark	Gemma	Compressed Gemma
HellaSwag	38.90	26.80
BoolQ	55.72	41.99
PIQA	66.81	51.80
ARC-Challenge	27.73	24.23
WinoGrande	52.17	47.99

Table 4.2: **Extreme head shearing of Gemma.** Scores are percentages (higher is better). We report normalized accuracy for HellaSwag, PIQA, and ARC-Challenge, and accuracy for BoolQ and WinoGrande.

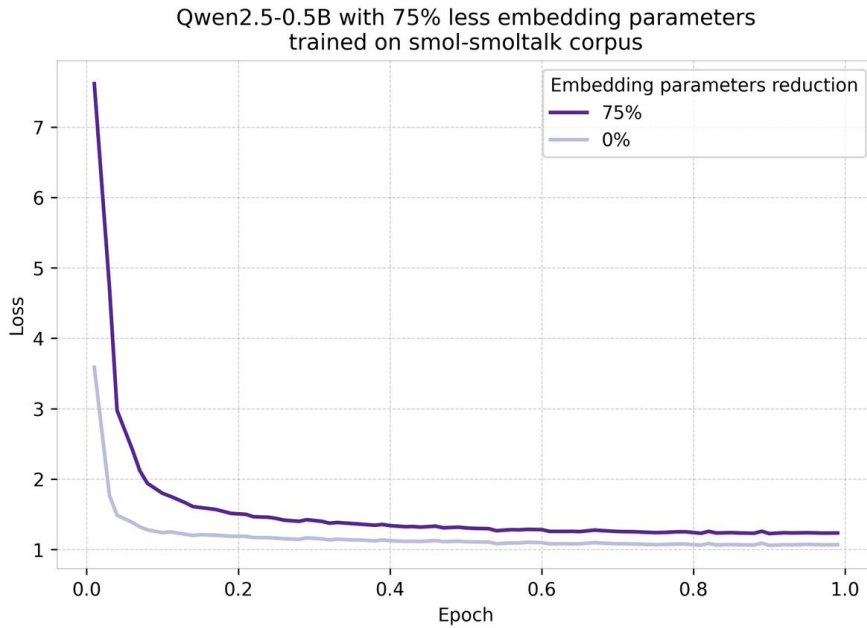


Figure 4.5: Training loss of Qwen2.5-0.5B on the smol-smoltalk corpus, comparing a 75% reduction in embedding parameters with the uncompressed baseline.

4.6 CONCLUSION

We introduced BaldWhisper, a compression approach for Whisper in low-resource settings. Merging adjacent decoder layers rather than simply pruning them limits the performance loss, while low-rank feature distillation compresses the large shared embedding matrix without discarding vocabulary items needed for code-switching. Using only 32 hours of Bambara speech, the final model is $2.15\times$ faster and 48% smaller than Whisper-base while retaining more than 90% of its performance.

Several directions could further improve this trade-off. The merging weights α and β could be specialized per layer, and more sophisticated

merging functions (Yadav et al., 2023; Yu et al., 2024) could replace the weighted average. Because the method operates on standard Whisper Transformer blocks, it should transfer to other languages; validating that transfer remains future work.

Part III

ACTIVATION SPARSITY OF TRANSFORMER MODELS

CROSS-LINGUAL MATRYOSHKA REPRESENTATION LEARNING ACROSS SPEECH AND TEXT

5.1 INTRODUCTION

This chapter studies activation sparsity through the lens of Matryoshka Representation Learning in embedding language models. We show that while Matryoshka representations offer a good trade-off between efficiency and performance, they use only a subset of the available embedding space. This leaves room to further improve how information is represented. In the next chapter, we show that this observation also extends to autoregressive LLMs.

This chapter also reflects a desire to explore an engaging application beyond standard English benchmarks. We ground the study in a concrete use case: retrieving French text documents from Wolof speech queries. This task requires representations that bridge both languages and modalities while remaining efficient. Matryoshka Representation Learning allows us to explore this setting while continuing the thesis’s focus on efficiency, by training embeddings that can be used at different dimensions depending on the computational budget.

Access to information is limited by two major barriers. First, a *language barrier*: most online content is written in a few high-resource languages. Second, a *modality barrier*: retrieval systems generally assume text queries, whereas many under-represented languages are primarily oral. Cascaded automatic speech recognition (ASR) and translation systems are costly and suffer from error propagation. We address both barriers with cross-lingual speech–text Matryoshka representations, enabling direct retrieval of text documents from speech queries with flexible accuracy–cost trade-offs. We focus on Wolof and French as a socially grounded case study. Wolof is spoken primarily in Senegal and remains mainly oral. Because of the country’s colonial history, much administrative, educational, and informational content relevant to Wolof speakers is available in French, while many Wolof speakers have limited French literacy. Information therefore exists as French text, but the natural query modality is Wolof speech. We introduce the first bilingual speech–text Matryoshka embedding model for this setting and, to our knowledge, the first application of MRL to speech-to-text retrieval. We develop data-curation pipelines and new benchmarks, compare modeling architectures, and analyze how embedding dimension affects accuracy, storage, and latency. We also evaluate transfer learning to new tasks and study the effective rank of Matryoshka representations.

This study builds on Matryoshka Representation Learning, introduced in the state-of-the-art chapter, and its application to multilingual and cross-modal representations. MRL (Kusupati et al., 2022) learns representations at multiple dimensions simultaneously, allowing the embedding dimension to be chosen at inference time. It has been generalized to vision–language and audio–visual models (Hu et al., 2024; Cai et al., 2024; Cappellazzo et al., 2025). We extend it to cross-lingual speech–text retrieval and analyze the performance, cost, and rank of its dimensions.

To achieve cross-modal speech and text representations, we draw inspiration from speech language models, which extend LLMs with speech understanding through a speech encoder (Wu et al., 2023; Pareras et al., 2025; Lam et al., 2025). We similarly add speech capabilities to embedding LLMs, an area underexplored relative to generative speech LLMs.

Cross-lingual text retrieval requires a strong multilingual speech representation model for Wolof and French. XLM-R, multilingual BERT, and mBART (Conneau et al., 2019; Devlin et al., 2018; Tang et al., 2020) demonstrate cross-lingual transfer but are not designed primarily for retrieval. NLLB-LLM2Vec (Schmidt et al., 2024) produces cross-lingual representations for more than 200 languages, including Wolof and French. We show that a model approximately ten times smaller, augmented with speech and trained mainly on supervised synthetic bilingual data, enables improved French and Wolof retrieval and transfers well to other tasks.

There are other approaches to achieving cross-modal representations, such as CLIP (Radford et al., 2021), which uses modality-specific encoders with a contrastive objective. CLAP (Elizalde et al., 2022) extends the design to audio and text. We found that these approaches, called *dual architectures*, work well for transcription retrieval but struggle with semantically demanding cross-lingual speech-to-document retrieval.

5.2 BUILDING A WOLOF SPEECH LANGUAGE MODEL

The retrieval system developed later in this chapter assumes a strong Wolof speech encoder. Such an encoder was not readily available: multilingual models cover Wolof only sparsely, and labeled Wolof speech is limited. We therefore first developed a Wolof-specific speech model in two stages. We continued self-supervised pretraining of HuBERT on a large collection of spontaneous Wolof speech, then connected the resulting encoder to a Wolof LLM. This section presents the complete development and evaluation of that model.

5.2.1 Background and Motivation

Self-supervised models such as wav2vec (Schneider et al., 2019; Baevski et al., 2020b) and HuBERT (Hsu et al., 2021) learn general speech representations from unlabeled audio before supervised fine-tuning. wav2vec 2.0 uses a contrastive objective to predict discretized latent targets, requiring careful negative sampling. HuBERT instead clusters intermediate hidden representations and predicts their discrete cluster assignments with cross-entropy. Data2Vec (Baevski et al., 2023) removes discretization and predicts masked continuous representations. We select HuBERT because its objective is simple, widely used, and supported by mature fine-tuning tools.

Multilingual models such as XLS-R and multilingual HuBERT (Arun Babu and Changan Wang and Andros Tjandra and Kushal Lakhotia and Qiantong Xu and Naman Goyal and Kritika Singh and Patrick von Platen and Yatharth Saraf and Juan Pino and Alexei Baevski and Alexis Conneau and Michael Auli, 2022; Zanon Boito et al., 2024) transfer representations across languages, and other work specifically targets African languages (Gauthier et al., 2016; Alabi et al., 2025). Performance on a particular language nevertheless depends on its share and diversity in the multilingual corpus. Wolof accounts for only about 0.1% of the African-centric pretraining data considered here. Because self-supervised learning does not require transcripts, scaling language-specific spontaneous audio is a practical alternative to relying on multilingual transfer alone.

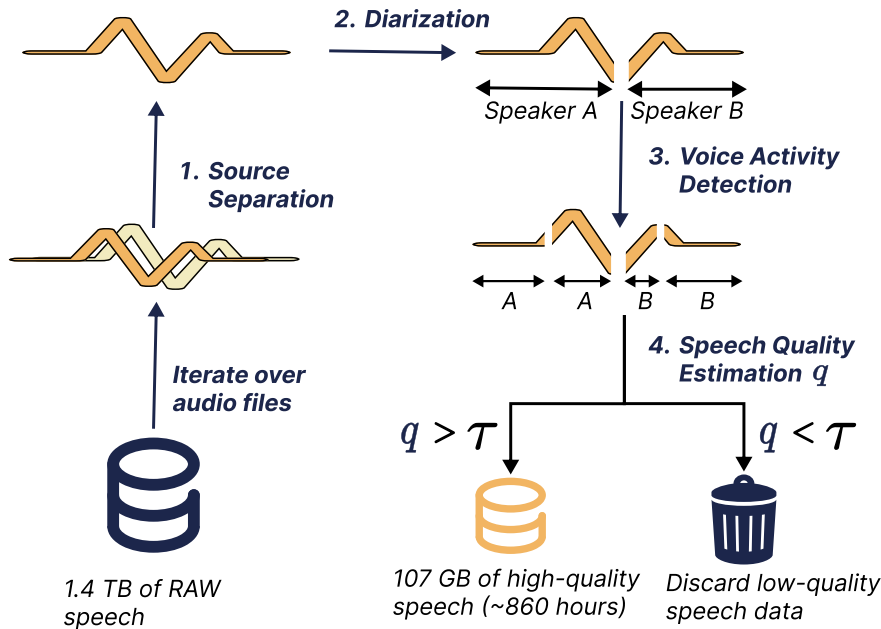


Figure 5.1: Collection of spontaneous Wolof speech for continued self-supervised pretraining. Raw web audio passes through source separation, speaker diarization, voice activity detection, and speech-quality filtering.

5.2.2 Wolof-Specific HuBERT Continued Pretraining

5.2.2.1 Unsupervised Speech Data

We manually curate 1.4 TB of raw Wolof audio from public web sources, prioritizing natural, spontaneous speech by native speakers rather than read speech, which may contain hyper-articulation and artificial emotion. The filtering pipeline in Figure 5.1, inspired by Emilia (He et al., 2024; He et al., 2025; Zhang et al., 2024b), applies source separation, speaker diarization, and voice activity detection. We retain utterances lasting 3–30 seconds with DNSMOS above 3.2 (Reddy et al., 2021). The result is 860 hours of intelligible, spontaneous Wolof speech.

5.2.2.2 Initialization and Training

We initialize the 90M-parameter HuBERT-base model (Hsu et al., 2021), originally pretrained on 960 hours of English LibriSpeech (Panayotov et al., 2015), and start with a fresh optimizer state because the original optimizer was unavailable. The Wolof corpus is nearly as large as the original English corpus, but initialization allows the model to reuse previously learned acoustic and phonetic structure.

Continued pretraining uses 32 A100 GPUs and the original HuBERT hyperparameters: 33 epochs, learning rate 5×10^{-4} , and batches containing 87.5 seconds of audio. HuBERT-base was originally trained for 528 epochs; the shorter adaptation demonstrates the computational benefit of reusing a mature checkpoint rather than training from scratch.

Observation 1: Phonetic features transfer well across distant languages for continued pretraining

Despite English and Wolof being typologically distant, English-pretrained HuBERT learns general phonetic features that transfer effectively to Wolof and require relatively few additional epochs.

5.2.3 Supervised ASR Fine-Tuning and Evaluation

5.2.3.1 Data and Fine-Tuning

The supervised corpus combines FLEURS, ALFA (Gauthier et al., 2016), Common Voice, Kallaama (Gauthier et al., 2024), and Urban Bus (DIOP, 2021). We lowercase transcripts, remove punctuation, and convert digits to words. Official FLEURS splits are retained; the remaining corpora use random train/test partitions.

We convert the weights obtained through continued pretraining to the Hugging Face format and fine-tune with the standard CTC

Split	FLEURS	ALFA	CV	Kallaama	UB	Total
Train	8.72	16.13	34.97	33.60	4.52	97.94
Test	1.75	2.84	6.21	5.91	1.12	17.83

Table 5.1: Hours of supervised Wolof speech used for ASR fine-tuning and evaluation.

speech-recognition recipe. We compare three encoders fine-tuned on exactly the same supervised data: Meta/HuBERT-base, its version continually pretrained on Wolof, and Orange/HuBERT-base (Caubrière and Gauthier, 2024), which was trained from scratch on 65,000 hours of African-language speech.

5.2.3.2 Results and Interpretation

Model	Pretraining hours	WER ↓
Orange/HuBERT-base	65,000	41.11
Meta/HuBERT-base	960	39.48
Wolof HuBERT	960 → 860	35.65

Table 5.2: ASR results after identical supervised fine-tuning. The arrow denotes English pretraining followed by Wolof continued pretraining.

Table 5.2 shows that language-specific continued pretraining reduces WER from 39.48 to 35.65 and outperforms the African-centric model despite using far less audio. Initialization transfers useful phonetic features, while the concentrated Wolof corpus is more valuable for Wolof than a much larger multilingual corpus in which Wolof is rare. Absolute WER remains relatively high partly because Wolof orthography is less standardized than that of high-resource written languages.

Observation 2: Do not rely on multilingualism alone; scale the monolingual corpus

Orange/HuBERT-base underperforms on Wolof despite 65,000 hours of multilingual African speech containing Wolof and related languages. Cross-lingual transfer is useful, but scaling high-quality Wolof-specific data provides a stronger target-language model.

5.2.4 From a Speech Encoder to a Wolof Speech LLM

HuBERT provides strong acoustic representations but its 90M parameters and CTC objective limit higher-level linguistic modeling. We connect it to a Wolof language model both to improve transcription and to enable speech translation.

5.2.4.1 Training the Wolof Text LLM

We adapt Qwen2.5-3B (Yang et al., 2024a; Team, 2024) using approximately two billion tokens. The mixture emphasizes bidirectional Wolof–English translation so that Wolof inputs can access knowledge learned during English pretraining, and includes high-quality monolingual Wolof text from Wolof-focused websites to improve generation.

5.2.4.2 Linking HuBERT to the LLM

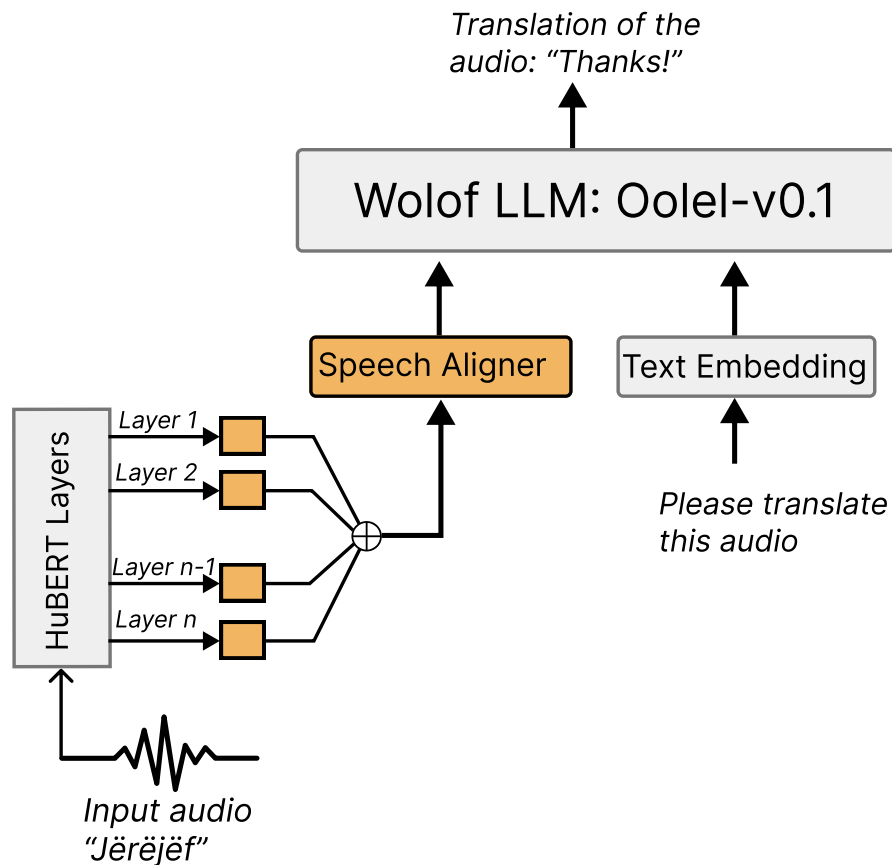


Figure 5.2: Wolof Speech LLM architecture. Features from every HuBERT layer are concatenated, projected by a lightweight aligner, combined with instruction-token embeddings, and passed to the LLM. Only the aligner is trained.

We use late fusion (Liu et al., 2023a; Wu et al., 2023), illustrated in Figure 5.2. Features from all HuBERT layers are concatenated and

projected into the LLM embedding space by an MLP with one hidden layer with ReLU activation. The projected speech sequence is concatenated with text instruction embeddings and passed through the LLM. HuBERT and the LLM remain frozen; only the aligner is optimized. HuBERT already downsamples the waveform substantially, so no additional sequence downsampling is used.

Training examples use an instruction-following format in which the assistant transcribes or translates an audio input. The loss is computed only on assistant completions, with batch size 2 and learning rate 10^{-4} . The speech encoder is the CTC-fine-tuned Wolof HuBERT from Table 5.2.

5.2.4.3 Multi-step Speech Reasoning

Besides direct transcription and translation, we test whether intermediate outputs improve performance. For ASR, the model may `PHONE-MIZE` or `TRANSLATE` before the `TRANSCRIBE` step; phonemes provide an acoustic intermediate representation, whereas translation provides a semantic one. For translation, the model may first `TRANSCRIBE` or `REFORMULATE`. These variants test whether extra generation helps construct a stronger speech representation.

5.2.4.4 Speech LLM Results

We evaluate ASR with WER and character error rate (CER). Speech translation is evaluated on FLEURS using character n -gram F-score (ChRF) and BERTScore F1 (BS-F1), complementing surface overlap with semantic similarity.

Generation order	WER ↓	CER ↓
TRANSCRIBE	29.09	15.26
PHONEMIZE→TRANSCRIBE	34.05	19.08
TRANSLATE→TRANSCRIBE	29.48	15.95

Table 5.3: ASR results of the Wolof Speech LLM.

Generation order	ChRF ↑	BS-F1 ↑
TRANSLATE	33.08	79.78
TRANSCRIBE→TRANSLATE	33.79	79.73
REFORMULATE→TRANSLATE	33.59	77.54

Table 5.4: Speech-to-text translation results of the Wolof Speech LLM.

Direct Speech LLM transcription obtains 29.09 WER, an 18.4% relative improvement over the 35.65 WER of the HuBERT-only CTC model.

Because HuBERT and the LLM are frozen while only the aligner is trained, the gain indicates that the LLM contributes useful Wolof linguistic knowledge rather than merely increasing acoustic capacity. The model also performs speech translation. Its translations are often semantic paraphrases rather than literal matches, explaining why BS-F1 is strong relative to ChRF.

Multi-step generation provides little benefit. Phonemization sometimes causes repetition loops in the 3B model, and transcription-before-translation yields only a small ChRF increase with essentially unchanged BS-F1. This agrees with evidence that the advantage of chain-of-thought diminishes as speech instruction data scales (Pareras et al., 2025); larger language models may also be needed to exploit explicit intermediate reasoning.

Observation 3: Multi-step chain-of-thought does not improve this Speech LLM

The LLM improves direct speech recognition and enables translation, but extra phonemization, transcription, translation, or reformulation steps provide little or no benefit. At this model and data scale, direct task supervision is the stronger operating point.

5.2.5 Implications for Cross-modal Retrieval

The Wolof Speech LLM establishes the speech foundation used in the remainder of this chapter. Its development yields three practical lessons: collect substantial spontaneous monolingual audio; continue pretraining a strong self-supervised checkpoint instead of training from scratch; and connect a frozen LLM through a small trainable aligner when higher-level linguistic knowledge is needed. For retrieval, we retain the Wolof-adapted HuBERT encoder and late-fusion principle, but replace the generative LLM with a Matryoshka embedding LLM and train the speech projection for contrastive retrieval.

5.3 DATASETS

Because Wolof speech–French text retrieval data are scarce, we rely mainly on synthetic documents. Here, a *query* is any speech or text input used for retrieval, whether or not it is phrased as a question.

5.3.1 Training Datasets

TEXT DATA. We combine French mMARCO (Bonifacio et al., 2021), whose queries are translated into Wolof; Senegalese French webpages, for which French queries are generated and translated; and French question-answering datasets with Wolof-translated questions. Wolof–

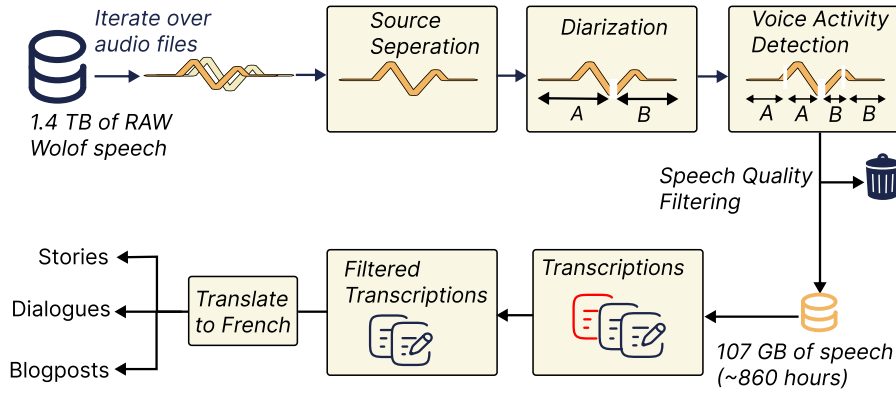


Figure 5.3: **Speech data pipeline.** Raw data are processed through source separation, diarization, voice activity detection, and quality filtering. Retained speech is transcribed before French stories, dialogues, and blog posts are generated.

French translation pairs encourage cross-lingual transfer. The result contains 1,176,908 query–document pairs and approximately 593 million document tokens.

WOLOF SPEECH QUERIES. We reuse the 860-hour collection developed for Wolof HuBERT continued pretraining in Section 5.2.2. Figure 5.3 shows how this same filtered speech is extended into a retrieval-data pipeline: after transcription filtering, each query is paired with several kinds of French target documents.

SYNTHETIC FRENCH DOCUMENTS. We transcribe queries using a Wolof speech language model (Sy et al., 2025). Perplexity and lexical-diversity filters retain roughly one quarter of the transcriptions. These are translated into French, and Gemini-2.5-Flash generates stories, dialogues, and web-style blog posts.

Synthetic targets are a practical way to scale supervision when parallel Wolof speech and French documents are unavailable. We considered reversing the pipeline by starting with natural French documents and synthesizing Wolof speech, but this would require a high-quality Wolof text-to-speech system, which is not available. Generating French documents is more reliable because current LLMs perform strongly in French. Using three document genres also reduces dependence on one synthetic style. Gemini’s free API tier made generation at this scale affordable; preliminary trials indicated that open-source generators could also produce usable targets, but running them locally would have required substantially more GPU compute.

INSTRUCTION-FOLLOWING DATA. The same speech may be associated with a Wolof transcription, French translation, or French document. Instruction-following embedding models (Peng et al., 2024) re-

solve this ambiguity through a task description. We train one prompted model for document, translation, and transcription retrieval. The natural ASR corpora used for these additional tasks are detailed in Section 5.2.3.

5.3.2 Evaluation Datasets

Evaluation artifacts: Wolof-to-French retrieval benchmarks

Kallaama-Retrieval-Eval and *Fleurs-Retrieval-Eval* evaluate French document retrieval from Wolof speech or text.

FLEURS-RETRIEVAL-EVAL. We begin with SIB-Fleurs (Schmidt et al., 2025), derived from FLEURS (Conneau et al., 2022). French translations of Wolof speech are used to retrieve relevant French webpages, and failed or irrelevant results are manually removed. The 166 retained documents remain uncleaned to reflect deployment conditions.

KALLAAMA-RETRIEVAL-EVAL. FLEURS read speech is often hesitant and quiet, so we construct a benchmark from the natural conversations in Kallaama (Gauthier et al., 2024). We select the 150 longest test utterances below 30 seconds, translate them, and use Gemini-2.5-Pro to generate a dialogue, blog post, and story for each. The result contains 150 speech and text queries, each paired with three French documents.

Both benchmarks are modest in size, which limits fine-grained statistical conclusions. Nevertheless, model inference and ranking are deterministic, the FLEURS targets are natural webpages collected from the web, and the Kallaama queries are spontaneous conversations representative of everyday spoken Wolof. We therefore use the two datasets as complementary evidence rather than treating either one as a comprehensive sample of the complete Wolof speech or French document distributions.

5.4 MODELING

Given retrieval loss $\mathcal{L}$ and dimensions $\mathcal{M} = \{d_1, \dots, d_m\}$, MRL optimizes

$$\mathcal{L}_{\text{MRL}} = \sum_{d \in \mathcal{M}} \mathcal{L}(\mathbf{Q}_{:d}, \mathbf{D}_{:d}), \quad (5.1)$$

where $\mathbf{Q}$ and $\mathbf{D}$ are query and document embeddings and $:$ d slices the first d dimensions. We use InfoNCE (Oord et al., 2019) with in-batch negatives. The InfoNCE loss for a specific Matryoshka dimension is defined as:

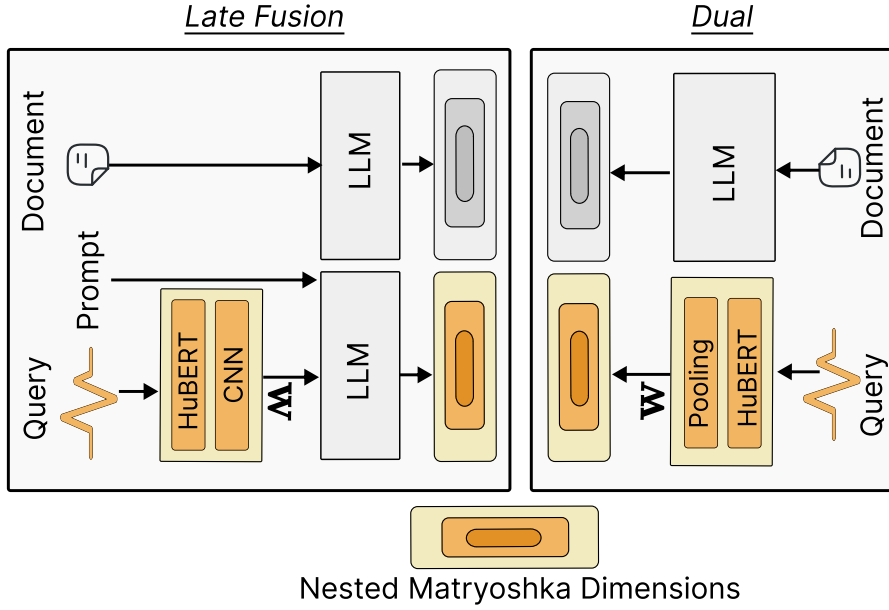


Figure 5.4: **Late-fusion and dual architectures.** Late fusion projects downsampled HuBERT features into the LLM embedding space and concatenates them with prompt embeddings. The dual architecture pools speech features and uses dimension-specific projections.

$$\mathcal{L}_{\text{InfoNCE}} = \frac{\cos(q, d_i)}{\sum_{d_j} \cos(q, d_j)}, \quad (5.2)$$

5.4.1 Text-only Embeddings

We use Qwen3-0.6B-Embedding at dimensions 128, 256, 512, and 1024. It is first fine-tuned on text-only Wolof–French pairs with in-batch InfoNCE, teaching cross-lingual alignment before speech is introduced.

5.4.2 Late Modality Fusion

Late fusion (Liu et al., 2023a; Wu et al., 2023) combines speech features and prompt-token embeddings before the LLM. We use Wolof-adapted HuBERT (Sy et al., 2025). Features from all 12 layers are concatenated, downsampled by a CNN, projected with **W**, concatenated with the prompt, and passed through the embedding LLM. Training uses Equation 5.1; HuBERT and the LLM remain frozen while the CNN and projection are updated.

Model artifact: Cross-lingual speech and text representation models

We introduce text-only and speech–text cross-lingual Matryoshka retrieval models for Wolof and French.

5.4.3 Dual Modality

The dual architecture does not pass speech through the LLM. A learned query $\mathbf{q}$ pools HuBERT features $\mathbf{X}$:

$$\text{pool}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{q}\mathbf{X}^T}{\sqrt{d}}\right)\mathbf{X}. \quad (5.3)$$

Dimension-specific projections produce Matryoshka embeddings. This design is less expressive and cannot directly process prompts. *Dual–Retrieval* uses InfoNCE while freezing the text model. *Dual–Query Alignment* instead aligns speech with transcript representations using cosine-distance and ℓ_1 losses.

5.5 EXPERIMENTS

5.5.1 Setup

We implement all four models with Sentence-Transformers. Each trains for one epoch with batch size 16, maximum length 2048, and learning rate 3×10^{-4} . Although a single epoch may appear short, the dataset is large and both late-fusion and dual models plateau during that epoch; additional training produced no significant gains. The comparison therefore measures the architectures under the same compute budget rather than assuming that the dual models are fully optimized in an unconstrained setting. Fine-tuning Qwen3-0.6B uses eight A100 GPUs for approximately two hours, an estimated \$22.40 at \$1.40 per GPU-hour. The speech models train for approximately eight hours on one H100. They are cheaper because the large text model is frozen: late fusion updates only its small speech aligner, whereas the dual architecture updates the speech encoder and projection modules.

5.5.2 Document Retrieval

We evaluate normalized discounted cumulative gain (nDCG) at ranks 5 and 10. NLLB-LLM2Vec (Schmidt et al., 2024) is an 8B-parameter text-only baseline with fixed 4096-dimensional embeddings. A pipelined baseline first transcribes speech and then uses our text-only model.

Tables 5.5 and 5.6 show that late fusion performs best. On Kallaama, it outperforms NLLB-LLM2Vec despite being ten times smaller with embeddings four times shorter. It also outperforms the ASR pipeline.

Approach	4096		1024		512		256		128	
	@5	@10	@5	@10	@5	@10	@5	@10	@5	@10
NLLB-LLM2Vec	57.98	61.53	–	–	–	–	–	–	–	–
Late-Fusion	–	–	69.85	74.49	66.86	71.04	61.58	67.05	56.13	62.30
Pipelined	–	–	57.09	62.82	54.07	59.14	45.60	51.56	41.21	47.34
Dual-Retrieval	–	–	46.96	53.70	45.27	52.48	41.97	49.78	38.47	44.40
Dual-Alignment	–	–	41.56	47.42	41.02	46.76	38.24	44.18	35.47	40.47

Table 5.5: Document-retrieval nDCG on Kallaama-Retrieval-Eval.

Approach	4096		1024		512		256		128	
	@5	@10	@5	@10	@5	@10	@5	@10	@5	@10
NLLB-LLM2Vec	55.98	59.43	–	–	–	–	–	–	–	–
Late-Fusion	–	–	57.89	61.19	55.03	58.90	50.87	54.43	41.56	46.60
Dual-Retrieval	–	–	41.28	45.54	40.18	45.06	39.84	45.17	39.24	44.34
Dual-Alignment	–	–	38.07	41.82	37.33	41.69	38.04	41.29	34.46	38.53

Table 5.6: Document-retrieval nDCG on Fleurs-Retrieval-Eval.

The dimension gap is larger on FLEURS, suggesting larger representations are more robust to low-quality speech.

The pipelined baseline already includes the first major source of cascading error: speech is transcribed before its text representation is retrieved. Adding machine translation would introduce another imperfect intermediate prediction, as well as additional latency, before retrieval. An ASR–translation–retrieval cascade should therefore be regarded as a less favorable operating point, especially for Wolof, where intermediate ASR and translation models are weaker than their high-resource counterparts. This expectation is an inference from the measured ASR–retrieval result rather than a directly measured three-stage baseline.

Observation 4: Late fusion outperforms dual architectures for cross-modal adaptation

Late fusion provides stronger cross-modal generalization than dual encoders while training fewer parameters.

5.5.3 Speech Keyword Spotting

We evaluate Urban Bus (DIOP, 2021), retrieving the station or landmark spoken by the user. This is equivalent to transcription retrieval.

Late fusion performs best at every dimension. Dual encoders nevertheless perform much better here than on document retrieval, and query alignment outperforms direct retrieval training.

Approach		1024	512	256	128
Late-Fusion	F1	88.79	84.85	79.80	76.86
	Recall	89.79	86.49	81.98	79.88
Dual-Retrieval	F1	68.64	58.33	50.86	44.59
	Recall	71.17	62.76	56.76	52.25
Dual-Alignment	F1	76.07	67.67	67.67	65.22
	Recall	77.78	70.87	70.87	68.47

Table 5.7: Keyword-spotting performance across embedding dimensions.

Observation 5: Dual models remain useful for less semantically demanding tasks

Dual encoders remain viable for transcription retrieval, which requires less semantic understanding than document retrieval.

5.5.4 Generalization to an Unseen Task

We evaluate speech intent detection on WolBanking77 (Kandji et al., 2025), which has ten banking intents. Zero-shot classification selects the label most similar to a speech request. In the n -shot setting, we follow SetFit (Tunstall et al., 2022): contrastive fine-tuning improves representations before a logistic-regression head is trained.

n -shot	F1	Recall
0	44.79	50.64
1	56.29	58.50
2	60.40	61.93
4	87.18	88.46
8	92.54	92.51
16	96.11	96.10

Table 5.8: Few-shot speech intent detection at dimension 1024.

Zero-shot F1 is substantially above the 10% random baseline and reaches 96.11 with 16 examples per class. Figure 5.7 shows that larger embeddings adapt faster, while smaller ones catch up as the amount of data increases.

Observation 6: Larger embeddings adapt quickly; smaller embeddings need more data

Larger embeddings adapt rapidly to new tasks; smaller embeddings require more examples but eventually approach their performance.

5.6 ANALYSIS

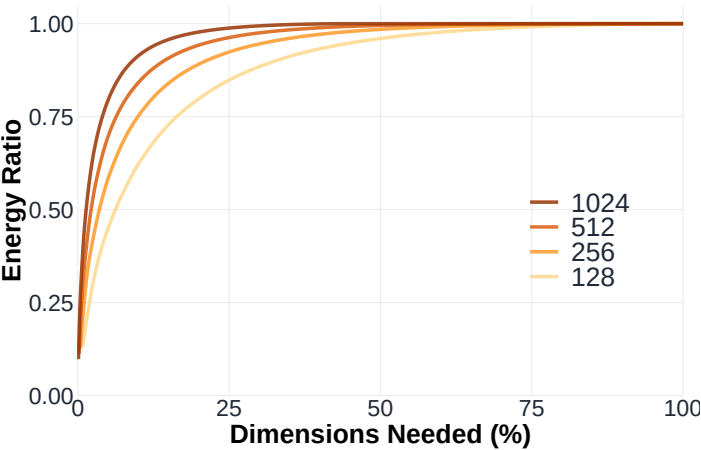


Figure 5.5: Dimensions needed for a given energy ratio.

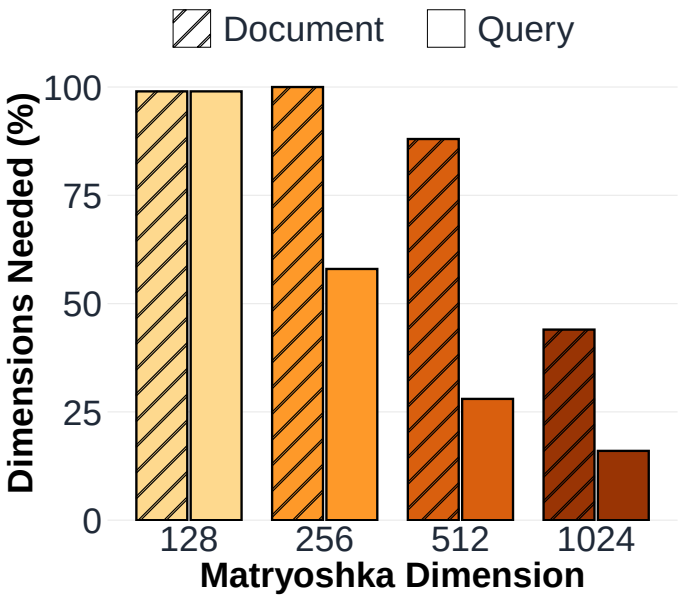


Figure 5.6: Dimensions needed for full energy.

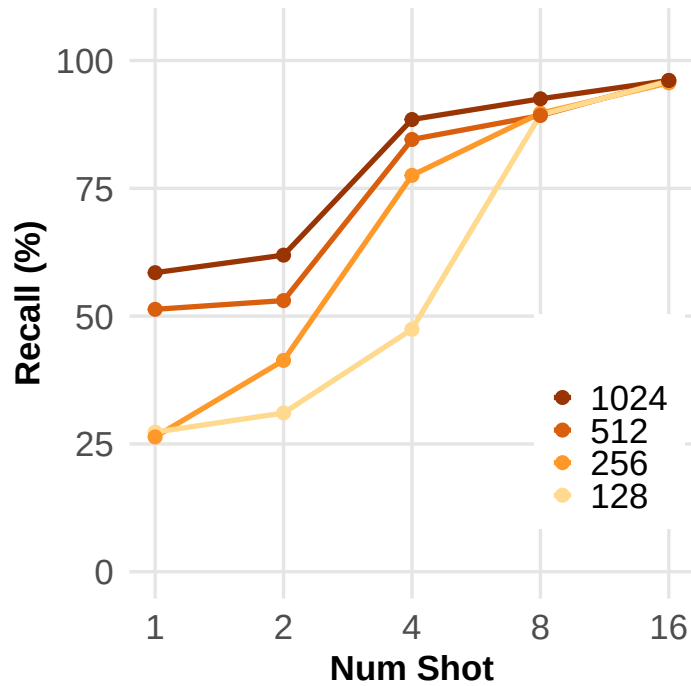


Figure 5.7: Recall by embedding dimension and number of examples.

5.6.1 Speech Quality in FLEURS

FLEURS asks speakers to read translated texts, but Wolof’s standardized writing system is not widely taught. Its hesitations thus primarily reflect difficulty reading Wolof rather than the discourse hesitations that naturally occur in spontaneous conversation. Compared with Kallaama, FLEURS contains more hesitations and lower volume. It averages 7.51 characters per second versus 16.88 for Kallaama, while mean loudness is -49.01 dB versus -23.81 dB. This helps explain Table 5.6 and the benefit of larger dimensions for low-quality speech.

Observation 7: Wolof speech in FLEURS is unnatural

Read Wolof in FLEURS is less natural, more hesitant, and quieter than spontaneous Kallaama speech.

5.6.2 Prompting

Without prompts, late fusion obtains 67.07/71.68 nDCG@5/10 on Kallaama at dimension 1024, compared with 46.96/53.70 for the dual model. Its advantage is therefore not explained only by prompt support. Task-aligned prompts still help: the document prompt produces the best document scores (68.85/74.49), while the transcription prompt produces the best keyword scores (88.79 F1, 89.79 recall).

5.6.3 Rank of Matryoshka Dimensions

We compute the cumulative energy ratio

$$R(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j}, \quad (5.4)$$

where the λ_i are descending eigenvalues of the representation covariance matrix on Kallaama. Figure 5.5 shows that larger embeddings reach full energy using a smaller fraction of coordinates and have lower relative rank. Yet smaller prefix embeddings do not match their accuracy, suggesting slicing discards critical information. Prior work similarly identifies gradient variance and rigid slicing as MRL limitations (Zhang et al., 2025; Wen et al., 2025). Figure 5.6 also shows that documents have higher ranks than speech queries; dimensions 128 and 256 are full-rank.

Observation 8: Information is distributed unevenly across Matryoshka dimensions

Large representations concentrate information in relatively few components, leaving scope for better compression than prefix slicing.

5.6.4 Deployment Costs

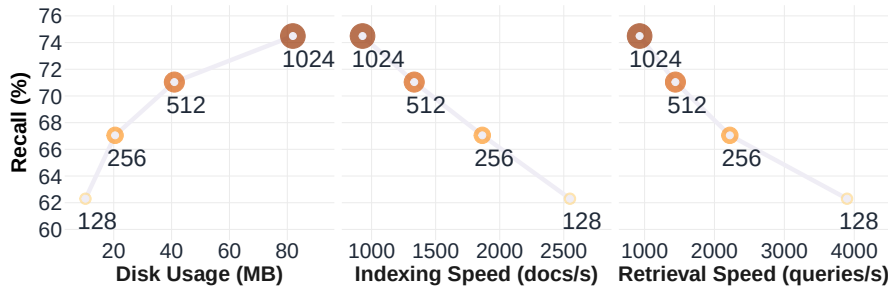


Figure 5.8: Indexing, storage, query-time, and accuracy trade-offs across dimensions.

We index float16 embeddings with chromadb and measure indexing throughput, disk usage, and single-result query latency. Figure 5.8 shows diminishing returns: larger embeddings provide smaller accuracy gains while increasing storage. Smaller embeddings accelerate retrieval substantially, although indexing speed improves more gradually. The best dimension depends on the task and compute budget.

The stronger degradation of small dimensions on FLEURS does not imply that low-dimensional embeddings work only with clean or near-perfect recordings. It is the standard efficiency–accuracy trade-off

becoming steeper under more challenging acoustic conditions: larger vectors preserve more information and are more robust, while smaller vectors retain their storage and speed advantages at the cost of a larger accuracy drop. Deployment should therefore select the embedding dimension in conjunction with expected input quality rather than treating one dimension as universally optimal.

5.7 LIMITATIONS

SYNTHETIC DATA. Most documents are generated from transcribed and translated speech. This enables scale but may not reproduce real-world diversity and noise. It also assumes usable ASR and translation systems, which many languages lack.

BEYOND MATRYOSHKA SLICING. Only a small subset of coordinates represents most information, suggesting prefix slicing is sub-optimal. Sparse-coding alternatives (Wen et al., 2025) or dynamic structured sparsity may offer more flexible compression.

5.8 CONCLUSION

We introduced cross-lingual speech–text Matryoshka embedding models that retrieve French documents directly from Wolof speech. Late modality fusion consistently outperforms dual encoders and an ASR pipeline, while the learned representations transfer to keyword spotting and intent detection.

The dimension and rank analyses expose a central limitation: information is concentrated in a small number of components, yet prefix-truncated embeddings do not preserve enough of it to match the largest representation. This connects adaptive representation learning to activation sparsity and motivates more flexible mechanisms for selecting informative dimensions at inference time.

6.1 INTRODUCTION

Chapter 5 showed that Matryoshka embeddings concentrate much of their information in a low-dimensional subspace. This chapter extends the investigation to autoregressive language models. We study redundancy in the intermediate activations of Transformer MLPs and introduce **ZeroNeuron (ZoN)**, an input-dependent gate that selects which intermediate coordinates to activate for each token. We then return to embedding models to examine adaptive coordinate selection beyond vector slicing as used in MRL.

As stated earlier, increasing LLM size improves performance but also raises training costs. The question is how to train large models efficiently at a lower computational cost. Sparsity, particularly MoEs, has emerged as an effective solution for training large language models economically. In these models, the Transformer’s MLP is replaced by larger matrices, but only parts of them are activated for each input token. MoEs can be viewed as activation sparsity applied to groups of channels: only a subset of these groups is activated for each token. Keeping the total matrix size and sparsity fixed while reducing the group size yields finer-grained sparsity, which has been shown to improve language modeling (Krajewski et al., 2024). Taking this granularity to the extreme leads to an architecture similar to a sparse memory layer (He, 2024; Lample et al., 2019) roughly corresponding to one expert per channel.

A memory layer (Weston et al., 2015; Lample et al., 2019) stores trainable key-value pairs (k_i, v_i) , with $k_i \in \mathbb{R}^{d_k}$ and $v_i \in \mathbb{R}^n$, but only a subset of pairs is used for computation for a given input token. Given an input $x \in \mathbb{R}^d$, a query network first produces $q(x) \in \mathbb{R}^{d_k}$. Each key receives a score $s_i = q(x)^\top k_i$, measuring how well it matches the query. The layer keeps only the k keys with the largest scores. We denote the indices of these selected keys by $\mathcal{I}$ and apply $w = \text{Softmax}((s_i)_{i \in \mathcal{I}})$ to obtain their weights. Finally, the corresponding values are combined into the output $m(x) = \sum_{i \in \mathcal{I}} w_i v_i$. Thus, the query determines which values are retrieved and how much each contributes.

While appealing for achieving finer-grained sparsity, sparse memory layers face several challenges. Selecting keys becomes expensive as the memory grows. Approaches such as PKM (Lample et al., 2019) and UltraSparse (Huang et al., 2025) reduce this cost through decomposition, but TopK selection still adds computational cost as the candidate set grows. When the memory is distributed across GPUs, tokens need to

be sent to the GPUs hosting their selected memory slots. TopK-based sparsity requires all-to-all communication, and these data transfers make sparse memory layers communication-bound, especially when the memory spans many GPUs. Moreover, the discrete selection made by TopK is not differentiable, which can make routing harder to optimize at high levels of fine-grained sparsity. Finally, TopK uses a fixed selection budget: every token activates the same number of key-value pairs, regardless of the difficulty of the task.

We go beyond standard sparse memory designs and TopK with the following contributions:

1. Leveraging the key-value memory interpretation of the MLP (Geva et al., 2021), we propose to make it sparse by learning to activate only the key-value pairs needed for each token.
2. We motivate this approach by observing that MLP intermediate activations mostly lie in a subspace much smaller than the full available space. We show that granular MoE experts compress input representations into narrower spaces that their activations occupy more fully. These observations motivate learning to select which intermediate coordinates to activate.
3. To select the needed key-value pairs for each token, we revisit stretching methods to learn activation sparsity in a way that is differentiable almost everywhere. The sparse gating is adaptive and applied pointwise, which is advantageous in distributed training as it allows each GPU to activate its local memory slots without needing to dispatch token representations as in TopK-based routing. This enables retaining the communication pattern of a dense tensor-parallel MLP in distributed training, which is simply an all-reduce and not the all-to-all in TopK-based activation sparsity.

Section 6.2 presents the key-value memory view and analyzes intermediate activation rank in dense and sparse models. Section 6.3 introduces ZoN, and Section 6.4 evaluates it in autoregressive language models. Section 6.5 examines its extension to embedding models.

6.2 CONTEMPLATING THE MLP

6.2.1 Transformer MLPs as Key-Value Memories

The MLP is defined by two linear projections $\mathbf{W}_k \in \mathbb{R}^{d_{\text{model}} \times d_{\text{inter}}}$, $\mathbf{W}_v \in \mathbb{R}^{d_{\text{inter}} \times d_{\text{model}}}$:

$$\text{MLP}(\mathbf{x}) = \sigma(\mathbf{x}\mathbf{W}_k)\mathbf{W}_v. \quad (6.1)$$

where σ is a non-linear activation function and $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ is the input token representation. Modern architectures often use gated

variants; we use this two-projection form to introduce the memory interpretation. A possible interpretation of the MLP is to consider it as a key-value memory layer (Geva et al., 2021; Dai et al., 2022; Meng et al., 2023a). $\mathbf{xW}_k$ scores how well the input matches each key. The activation function σ transforms these scores into weights, which are then used to aggregate the corresponding value entries in $\mathbf{W}_v$. Usually, the dimension d_{inter} , called the *intermediate dimension*, is much larger than d_{model} , often $4\times$, so it provides a large set of keys. The kind of information stored in this memory remains an open research question. Previous work found that these key-value pairs act as lexical dictionaries (Geva et al., 2021), where some keys are associated with specific lexical tokens. Such associations can also reflect factual knowledge, as when a representation of “The capital of France is” retrieves information associated with “Paris”.

If the MLP can be seen as a memory layer, it is arguable that this memory can be sparse, since a given input may not need to activate all the key-value pairs. It is conceivable that an input needs to retrieve only a subset of the key-value pairs. If so, one could design a sparse activation gate that predicts ahead of time which key-value memory pairs are needed for a given token representation. But to motivate this approach, we first need to examine whether the intermediate activations contain redundancy that a sparse gate could exploit. If so, we then need to design such a sparse gating function.

6.2.2 The Effective Dimensionality of the MLP

We analyze the eigenstructure of the MLP, both in dense and sparse models.

Observation 1: The MLP has low effective intermediate dimension

Most activation energy is concentrated in a subspace with far fewer dimensions than the allocated intermediate space.

MEASURING THE RANK We measure the effective dimensionality of the intermediate activations relative to their allocated width d_{inter} in dense and MoE Transformers. We collect the key-projection activations before the nonlinearity using a calibration dataset $\mathcal{D}$. For any input token $\mathbf{x} \in \mathcal{D}$, we define $\mathcal{K} = \{\mathbf{k} = \mathbf{xW}_k\}_{\mathbf{x} \in \mathcal{D}}$ with $\mathbf{k} \in \mathbb{R}^{d_{\text{inter}}}$. We estimate the covariance matrix from these activations as $\Sigma = \mathbb{E}[\mathbf{k}\mathbf{k}^\top] - \mathbb{E}[\mathbf{k}]\mathbb{E}[\mathbf{k}]^\top$. We then compute the eigenvalues of this covariance matrix and sort them in decreasing order, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$, where $d = d_{\text{inter}}$. We analyze the cumulative energy ratio $R(k)$, which is

the proportion of total activation variance explained by the top- k eigenvalues:

$$R(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^d \lambda_j}, \quad k \in \{1, \dots, d\}.$$

Observation 2: MoEs use smaller subspaces more fully

MoEs maintain a large total intermediate space, but each input token representation is projected into smaller subspaces. Activations occupy a larger fraction of the dimensions within these subspaces.

We report $R@p$, with $p \in [0, 1]$, defined as the smallest fraction k/d such that $R(k) \geq p$. This ratio tells us how the activation energy is spread. If the activations are low rank, $R(k)$ reaches p more quickly, which means only a few dimensions are needed to represent most of the activation energy. For example, if $R@1.0 = 0.4$, then about 40% of the dimensions are needed to represent all the activation energy. In practice, because of numerical precision and small eigenvalue tails, reaching exactly 100% can be unstable. We therefore use $p = 0.99$ in our experiments. Overall, the metric used is related to effective rank and stable rank, but provides more flexibility by varying p .

Table 6.1: We check possible sample size biases for Qwen3-30B-A3B, since MoE layers receive different numbers of tokens. We examine the correlation between the number of routed tokens per expert and the energy ratio $R@0.99$ and find none.

Metric	Value
Mean $R@0.99$ across experts	0.8318
Mean routed tokens per expert	126,988
Median routed tokens per expert	96,432
ρ amount of routed tokens vs. ratio	0.0799

RESULTS. Figure 6.1 shows the percentage of dimensions needed to represent 99% of the activation energy. We can see that all dense models have low effective dimension. This is also true for models with very low dimensions such as Monad. The linear projection already limits the covariance rank to at most $\min(d_{\text{model}}, d_{\text{inter}})$, so expanding to $d_{\text{inter}} > d_{\text{model}}$ necessarily leaves linear dependencies among the intermediate coordinates. Hence, the intermediate activations of dense models contain considerable redundancy. As shown in Figure 6.2, fine-grained sparse MoEs reduce this redundancy by projecting the input representation into different, smaller subspaces. For example, Qwen3-30B-A3B has a total intermediate dimension of 98,304, organized

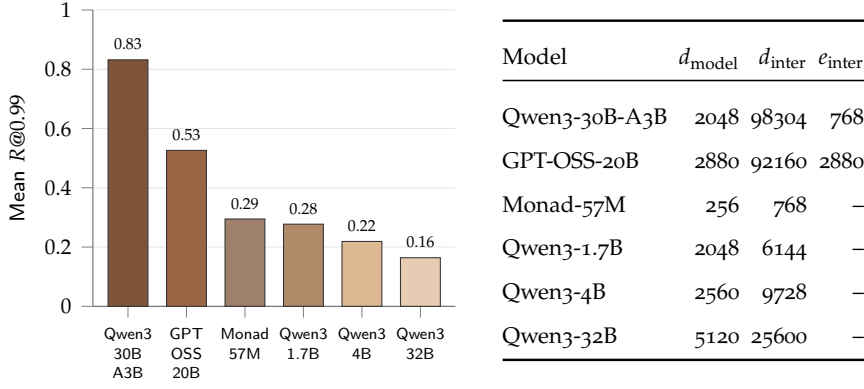


Figure 6.1: Mean $R@0.99$ across models, with model dimensions alongside. Higher $R@0.99$ means that more activation components are needed to represent 99% of the activation energy. For MoEs, e_{inter} is the intermediate dimension per expert and d_{inter} is the total intermediate dimension of the model.

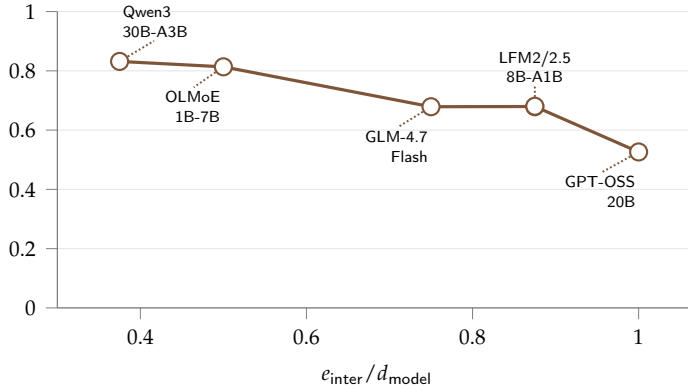


Figure 6.2: Relationship between mean $R@0.99$ and relative expert width across six MoEs. Ratios are measured within each expert, relative to e_{inter} .

into 128 subspaces of 768 dimensions each. Each token is projected into 8 of these subspaces. Hence, the higher energy ratio of MoEs reflects fuller use of their smaller intermediate subspaces. Since experts receive different numbers of tokens, we also check whether this affects the measured ratio. For Qwen3-30B-A3B, Table 6.1 shows negligible linear correlation between the number of routed tokens and the ratio ($\rho = 0.0799$). These observations motivate an approach where the MLP intermediate dimension is large but each token selects only the subspace it needs.

6.3 LEARNING TO SELECT KEY-VALUE PAIRS ADAPTIVELY

We showed that dense MLP activations occupy only part of the available intermediate space. MoEs instead maintain a large total intermediate space, but each input token is projected into smaller subspaces

that are used more fully. This motivates using a large intermediate dimension while selecting only the memory slots needed for each input token. To avoid the fixed selection budget and token dispatch of TopK-based sparse memory layers, we propose a new structured sparse gate that learns to zero out intermediate coordinates for each input token.

6.3.1 ZeroNeuron Sparse Activation Gating

We revisit the hard-concrete distribution (Louizos et al., 2018), originally proposed to learn binary masks $\mathbf{z}$ for pruning. The core idea is to concentrate probability mass at 0 and 1 through a “stretching” mechanism. It starts with a continuous relaxation of Bernoulli sampling, producing values between 0 and 1, stretches them into the interval $[l, r]$, where $l < 0$ and $r > 1$, and finally clips them back to $[0, 1]$:

$$\begin{aligned} \mathbf{u} &\sim \mathcal{U}(0, 1), \quad \mathbf{s} = \text{sigmoid}((\text{logit}(\mathbf{u}) + \log \alpha) / \beta), \quad \bar{\mathbf{s}} = \mathbf{s}(r - l) + l, \\ \mathbf{z} &= \min(1, \max(0, \bar{\mathbf{s}})). \end{aligned} \quad (6.2)$$

Here, $\alpha \in \mathbb{R}_{>0}^{d_{\text{inter}}}$ are learnable parameters and $\beta > 0$ is the temperature. Clipping sets values below 0 to exactly 0 and values above 1 to exactly 1. This makes the decision discrete by allowing some entries to be exactly switched on or off. The reparameterized relaxation provides gradient estimates for learning the gates while allowing computation to be skipped where $\mathbf{z}_i = 0$. This approach has proven effective for learning static pruning masks (Xia et al., 2024; Wang et al., 2020).

In order to learn predictive activation sparsity, we need to adapt this mechanism. First, it was designed to learn a fixed pruning mask, while we want to predict which coordinates to zero out based on the current input token. Second, we want the active gates to remain continuous rather than being clipped to 1. To address the first point, we predict $\log \alpha$ with an input-dependent network. To address the second, we apply stretching only on the left boundary (0). Given an input $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, we define ZeroNeuron ZoN($\mathbf{x}$), a revised hard-concrete distribution, as:

$$\begin{aligned} \mathbf{u} &\sim \mathcal{U}(0, 1), \quad \mathbf{s} = \text{sigmoid}((\text{logit}(\mathbf{u}) + \log \alpha(\mathbf{x})) / \beta), \quad \bar{\mathbf{s}} = \mathbf{s} + l(1 - \mathbf{s}), \\ \mathbf{z} &= \max(0, \bar{\mathbf{s}}). \end{aligned} \quad (6.3)$$

It is possible to encourage sparsity using l_0 regularization. The expected number of active output coordinates has a differentiable closed form:

$$\mathbb{E}[\|\mathbf{z}\|_0] = \sum_{i=1}^{d_{\text{inter}}} \text{sigmoid}(\log \alpha_i(\mathbf{x}) - \beta \log(-l)). \quad (6.4)$$

Adding $\mathbb{E}[\|\mathbf{z}\|_0]$ to the training loss encourages fewer active coordinates. Since $\mathbb{E}[\|\mathbf{z}\|_0]$ directly measures the expected number of active

Listing 6.1: Simplified implementation of the ZoN MLP training forward pass.

```

import math
import torch

def mlp_zon(x, W_k, W_v, act_fn, a, beta=1.0, l=-0.01):
    """
    x: [batch * seq_len, d_model]
    W_k: [d_model, d_inter]
    W_v: [d_inter, d_model]
    a: gate network, d_model -> d_inter
    """
    log_alpha = a(x) # shape: [batch * seq_len, d_inter]

    # Sample gates
    u = torch.empty_like(log_alpha).uniform_(eps, 1 - eps)
    s = torch.sigmoid((torch.logit(u) + log_alpha) / beta)
    # stretch beyond zero and clamp to 0.0
    z = (s + l * (1 - s)).clamp(min=0.0)
    # Expected average of active gates per token
    avg_active = torch.sigmoid(
        log_alpha - beta * math.log(-l)
    ).mean()

    h = act_fn((x @ W_k) * z)
    o = h @ W_v

    return o, avg_active

```

coordinates, we can impose a target active intermediate dimension h through a Lagrangian objective:

$$\mathcal{L}_{\text{sarsity}} = \lambda_1(\mathbb{E}[\|\mathbf{z}\|_0] - h) + \lambda_2(\mathbb{E}[\|\mathbf{z}\|_0] - h)^2 \quad (6.5)$$

where λ_1 and λ_2 are the trainable Lagrangian scalar parameters.

The gate thus retains two useful properties of the hard-concrete distribution: a differentiable objective for the expected number of active coordinates and a pointwise sparsification process. The latter is beneficial for tensor parallelism, as discussed below.

6.3.2 Structured Activation Sparsity of the MLP

Using the definition of the MLP as a key-value memory, we can write its sparsely activated version as:

$$\text{MLP}_{\text{ZoN}}(\mathbf{x}) = \sigma((\mathbf{x}\mathbf{W}_k) \odot \mathbf{z})\mathbf{W}_v \quad (6.6)$$

where $\mathbf{z} \in \mathbb{R}^{d_{\text{inter}}}$ is the output of the sparse activation gate $\mathbf{z} = \text{ZoN}(\mathbf{x})$. Since $\mathbf{z}$ predicts ahead of time which of the d_{inter} coordinates to zero

out for this input token, it decides which key-value pairs to ignore. Hence, the sparsity is structured and can be leveraged to accelerate training provided a GPU kernel can exploit this sparsity. For the predicted sparsity to be used for sparse computation in both $\mathbf{W}_k$ and $\mathbf{W}_v$, the non-linear activation σ must preserve zeros: $\sigma(0) = 0$. This ensures that coordinates zeroed out by the gate remain zero after the activation. Fortunately, this holds for common activations in modern Transformer MLPs, such as SiLU, ReLU, and ReLU². Listing 6.1 provides a dense reference implementation of the gated MLP in PyTorch. It computes both matrix multiplications densely; realizing computational savings requires a kernel that skips the coordinates predicted to be inactive.

6.3.2.1 Local and Global Sparsity Constraints

While the provided implementation is illustrated on a single MLP layer, the Transformer model has L such MLP layers. Each has its own sparse gate. The sparsity constraint can be applied either locally or globally. When applying the sparsity constraint locally, each MLP layer converges to the specified target sparsity. In contrast, applying the constraint globally means that only the average sparsity across layers is constrained. Individual layers can therefore use different sparsity levels, allowing the model to allocate more active neurons to layers that require greater capacity while inducing higher sparsity in less demanding layers. This provides greater flexibility while maintaining the desired overall compute budget. In our setting, we use a global sparsity constraint for greater flexibility. Hence, we define the average expected number of active gates across all L MLP layers as $\mathbb{E}_L[\|\mathbf{Z}\|_0] = \frac{1}{L} \sum_{l=1}^L \mathbb{E}_l[\|\mathbf{z}\|_0]$. Using $\mathbb{E}_L[\|\mathbf{Z}\|_0]$ as the expected sparsity in Equation 6.5 enforces the global sparsity constraint across all layers.

6.3.2.2 Parameterizing the Gate Network

The design of the gate network $\alpha(\mathbf{x})$ is flexible: the network can be complex, non-linear, and deep. However, its parameter count must remain manageable as the intermediate dimension grows. We found that the gating network can be very small without hurting performance. We parameterize the gate logits with a low-rank projection:

$$\log \alpha(\mathbf{x}) = (\mathbf{x}\mathbf{A}_1)\mathbf{A}_2 + \mathbf{b}. \quad (6.7)$$

Here, $\mathbf{A}_1 \in \mathbb{R}^{d_{\text{model}} \times r}$ and $\mathbf{A}_2 \in \mathbb{R}^{r \times d_{\text{inter}}}$. The bias $\mathbf{b} \in \mathbb{R}^{d_{\text{inter}}}$ controls the initial gate activity. We found that a small rank is sufficient, so in our experiments we fix $r = d_{\text{model}}/8$. We do not yet understand why such small gating networks work well. One possible explanation is that the sigmoid is applied independently to each coordinate, so each gating decision can be made without modeling interactions among all intermediate dimensions. In contrast, a softmax produces a probability

distribution jointly over the intermediate dimensions, which may require a larger routing network to model these dependencies.

6.3.2.3 On the efficiency of ZoN

The proposed sparse activation gating offers several opportunities to reduce training and inference costs relative to dense models and other TopK-based sparse models such as MoEs and sparse memory layers.

First, the approach learns structured activation sparsity. The sparsity emerges quickly during training and a sparse GPU kernel can exploit this to skip computation of the zeroed-out coordinates to accelerate training and inference. Given the predicted sparse gates $\mathbf{z} \in \mathbb{R}^{t \times d_{\text{inter}}}$ for t tokens, the corresponding predicted zero gates in W_k and W_v can be skipped during the MLP forward and backward passes. There are many sparse GPU kernels in the literature that can be adapted to accelerate training and inference, such as DeJaVu (Liu et al., 2023b) or TEAL (Liu et al., 2025).

Second, the proposed sparsification approach is pointwise: the sigmoid and stretching apply independently to each coordinate. Compared to TopK-based activation sparsity, this makes the approach less communication-bound in distributed training, especially with tensor-parallel sharding. Suppose the gates and the two MLP projections are sharded along the intermediate dimension across N GPUs. Each GPU predicts its local gates from the same input $\mathbf{x}$ and applies `zon_mlp` using its local weights to compute its contribution $\mathbf{y}_i$ to the output. An all-reduce then sums these contributions across GPUs and gives each GPU the full output $\mathbf{y} = \sum_{i=1}^N \mathbf{y}_i$. This is the same output communication used in a dense tensor-parallel MLP. This approach therefore uses less communication than TopK-based routing that needs to dispatch token representations to the GPUs hosting their selected memory slots.

The main possible disadvantage of the proposed approach is that different tokens can activate different MLP intermediate coordinates, leading to scattered memory accesses for sparse computation. This can make the computation memory-bandwidth-bound: reading the weights becomes the bottleneck rather than performing the computation. One way to reduce this cost is to group tokens that activate similar coordinates into buckets and process them together. The benefit depends on how much their active coordinates overlap and the cost of grouping the tokens.

6.4 EXPERIMENTS

6.4.1 Experimental Setup

IMPLEMENTATION. Our experiments are based on nanochat, a minimalist GPT implementation optimized for dense training. For fair comparisons, we keep the hyperparameter settings unchanged, which

have been optimized for dense models. Transformer matrices, including the sparse gate network projections, are optimized with Muon. The embeddings, the output head, biases, and scalar parameters use AdamW. In each MLP, the gate network uses two linear projections with a bottleneck of $d_{\text{model}}/8$ and a learned output bias. We set $\beta = 1$ and $l = -0.01$. The gates are initialized to keep almost all coordinates active at the beginning of training. The gradient signal from the sparsity loss can be as strong as that from the language modeling loss at the beginning of training. We therefore introduce a sparsity ramp-up in which we leave sparsity unconstrained for the first 1,000 steps, then linearly increase the target sparsity over 4,000 steps. The constraint is applied to the average expected sparsity across layers, with a Lagrange multiplier updated at each training step. Gates are sampled during training and computed deterministically at evaluation.

Table 6.2: Model architecture and training budgets. Dense-600M uses a narrower MLP; the other architecture settings are shared.

Configuration	Dense-600M	Dense-1.2B	ZoN-1.2B-A600M
Total parameters	600,834,122	1,233,125,450	1,233,125,450
Transformer layers		24	
Hidden dimension		1,536	
Query heads		12	
Key/value heads		12	
MLP expansion multiplier	2.417×		8×
MLP intermediate dimension	3,712		12,288
Vocabulary size		32,768	
Context length		2,048	
Sliding-window pattern		3:1	
Training tokens (approx.)		13B	

MODELS. We train three models: Dense-600M, a 600M dense model, Dense-1.2B, a 1.2B dense model, and ZoN-1.2B-A600M, a 1.2B model with 600M active parameters using our proposed approach. All three models are trained on approximately 13B tokens, with their architecture settings summarized in Table 6.2. Dense-1.2B uses the same MLP dimensions as ZoN-1.2B-A600M, while Dense-600M uses a narrower MLP to match its approximate active parameter count. This setup helps us study whether, for the same number of active parameters, increasing the total number of model parameters improves performance.

EVALUATION. We use the DataComp-LM CORE evaluation setup (Li et al., 2025), as implemented in nanochat. The suite contains

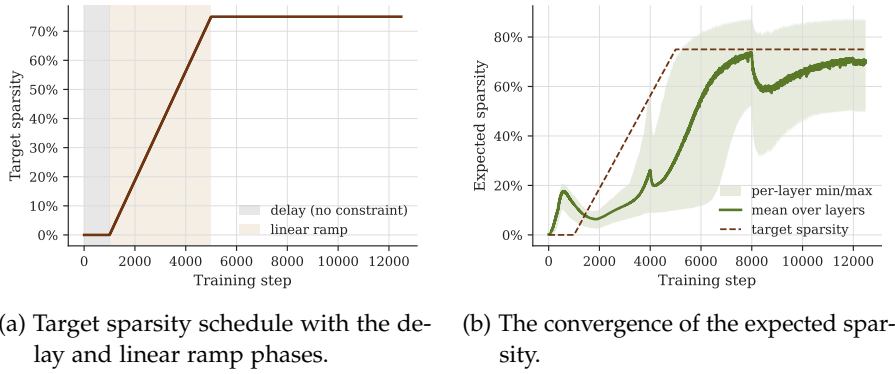


Figure 6.3: Training dynamics of the adaptive ZoN gate.

22 tasks covering commonsense reasoning, reading comprehension, language modeling, knowledge, and symbolic reasoning, with task-specific prompts and few-shot settings. The evaluator scores multiple-choice and schema tasks by selecting the answer with the highest average log-probability over the scored continuation. For completion tasks, every target token must be the model’s most likely prediction given the preceding reference tokens. We report raw task accuracies and the aggregate CORE score. To compute the aggregate CORE score, each task accuracy a is first centered as $(a - b) / (1 - b)$, where b is the task’s random baseline.

6.4.2 Results

Table 6.3 shows that ZoN-1.2B-A600M reaches a CORE score of 29.77, compared with 29.61 for Dense-1.2B and 27.35 for Dense-600M. ZoN therefore achieves similar aggregate performance to the larger dense model while using roughly 600M active parameters per token. Compared with the dense model of similar active size, it improves CORE by 2.42 percentage points. This supports the idea that keeping a larger memory and selecting which parts to activate in the intermediate dimension can be more effective than using a smaller dense memory.

The comparison with Dense-600M is consistent across many tasks. For example, ZoN improves HellaSwag accuracy from 57.24% to 61.24%, CommonsenseQA from 21.38% to 28.67%, and QA Wikidata from 46.44% to 50.51%. Compared with Dense-1.2B, ZoN performs better on Winograd and SQuAD, but worse on PIQA, ARC-Challenge, and CommonsenseQA. Hence, the small CORE difference of 0.16 percentage points between the dense and sparse 1.2B models reflects comparable overall performance rather than a consistent advantage over the larger dense model.

As described above, we gradually increase the target sparsity during training to let the model focus on language modeling at the beginning. Figure 6.3a shows this schedule, while Figure 6.3b shows that the

Table 6.3: Benchmark accuracy (%) for the dense and sparse models. CORE reports the mean centered score (%). BB denotes BIG-bench.

Benchmark (n-shot)	Dense 600M	Dense 1.2B	ZoN 1.2B-A600M
<i>Commonsense reasoning</i>			
PIQA (10)	74.37	76.93	75.35
WinoGrande (o)	56.91	60.30	60.77
ARC-Easy (10)	69.40	72.10	70.67
ARC-Challenge (10)	40.10	42.66	40.96
HellaSwag (o)	57.02	61.79	60.52
HellaSwag (10)	57.24	62.05	61.24
<i>Commonsense reasoning and reading comprehension</i>			
COPA (o)	66.00	64.00	68.00
CommonsenseQA (10)	21.38	32.02	28.67
OpenBookQA (o)	40.40	43.20	41.40
Winograd (o)	69.23	68.13	73.63
BoolQ (10)	61.62	63.79	61.31
SQuAD (10)	45.69	46.34	48.20
<i>Reading comprehension, language modeling, and knowledge</i>			
CoQA (o)	33.30	34.22	35.04
LAMBADA (OpenAI) (o)	43.22	46.81	46.96
Jeopardy (10)	9.40	15.21	15.07
BB: QA Wikidata (10)	46.44	51.30	50.51
BB: Language identification (10)	24.98	24.77	24.81
AGI Eval: LSAT-AR (3)	26.96	22.17	22.17
<i>Symbolic reasoning</i>			
BB: Dyck languages (10)	12.20	13.10	11.60
BB: CS algorithms (10)	44.32	39.39	44.32
BB: Operators (10)	18.57	17.62	19.05
BB: Repeat copy logic (10)	3.12	0.00	0.00
CORE	27.3510	29.6130	29.7722

expected sparsity closely follows it. This suggests that suitable sparse kernels could accelerate training once sparsity emerges. For longer training runs, the same delay and ramp would account for a smaller fraction of the total training steps. The sparsity constraint applies to an average over input tokens, so individual tokens and tasks can have different sparsity levels. The reported sparsity of approximately 70% is an average across evaluation tasks. Table 6.4 reports the predicted sparsity (Zero frac.) for each task, along with the centered accuracies used to compute the CORE metric.

Table 6.4: Evaluation of Dense-1.2B (Dense) and ZoN-1.2B-A600M (Sparse). All entries are percentages; CORE is the mean centered score. %0 measures the fraction of zeros predicted by ZoN MLP gates during inference. BB denotes BIG-bench.

Task	Accuracy		Centered		Predicted %0
	Dense	Sparse	Dense	Sparse	Sparse
HellaSwag (zero-shot)	61.79	60.52	49.05	47.35	69.19
Jeopardy	15.21	15.07	15.21	15.07	65.20
BB: QA Wikidata	51.30	50.51	51.30	50.51	65.30
ARC-Easy	72.10	70.67	62.79	60.89	69.56
ARC-Challenge	42.66	40.96	23.55	21.27	69.40
COPA	64.00	68.00	28.00	36.00	67.41
CommonsenseQA	32.02	28.67	15.03	10.83	71.84
PIQA	76.93	75.35	53.86	50.71	69.14
OpenBookQA	43.20	41.40	24.27	21.87	68.05
LAMBADA (OpenAI)	46.81	46.96	46.81	46.96	66.17
HellaSwag	62.05	61.24	49.40	48.32	68.29
Winograd	68.13	73.63	36.26	47.25	67.09
WinoGrande	60.30	60.77	20.60	21.55	66.98
BB: Dyck languages	13.10	11.60	13.10	11.60	75.95
AGI Eval: LSAT-AR	22.17	22.17	2.72	2.72	72.12
BB: CS algorithms	39.39	44.32	39.39	44.32	73.14
BB: Operators	17.62	19.05	17.62	19.05	73.53
BB: Repeat copy logic	0.00	0.00	0.00	0.00	70.89
SQuAD	46.34	48.20	46.34	48.20	67.87
CoQA	34.22	35.04	34.22	35.04	68.45
BoolQ	63.79	61.31	4.72	-1.80	67.76
BB: Language identification	24.77	24.81	17.24	17.28	67.62
CORE	—	—	29.6130	29.7722	—

6.5 GENERALIZATION TO EMBEDDING LANGUAGE MODELS

We apply the same principle to embedding language models, using ZoN to predict which coordinates of an embedding to keep. Here, we use NV-Embed representations and train the gates without a specific target sparsity constraint. Unlike the MLP setting, we stretch on both the left and right sides and clip the gates to $[0, 1]$, encouraging gates to be either zero or one. Binary masks can reduce the storage cost of the gates in a vector store; storing sparse embeddings also requires identifying their active coordinates. At inference, we keep the k coordinates with the highest sigmoid gate scores, with $k \in \{128, 256\}$ for evaluation, but k can be chosen freely during inference, depending on the budget.

EXPERIMENTAL SETUP. For retrieval, we train a linear projector of shape $d_{\text{model}} \times d_{\text{model}}$ on top of the embeddings, alongside the gate projector. For classification, we instead use a two-layer perceptron, with linear projections from d_{model} to $4d_{\text{model}}$ and back to d_{model} , with a ReLU activation between them. In both settings, the gate network uses a hidden dimension of $r = d_{\text{model}}/4$, with d_{model} being 4096 for NV-Embed. We evaluate retrieval on NFCorpus and SciFact using NDCG@10, and classification on MTOPIntent and Banking77 using top-1 accuracy. We train for 10 epochs on each dataset, except NFCorpus, where we use 2 epochs due to the training budget.

Table 6.5: Performance of NV-Embed representations on classification and retrieval tasks. Scores are percentages. k is the number of coordinates retained at inference; dense representations use all d_{model} coordinates. The dense baseline uses NV-Embed with a projector fine-tuned on each task and retains all coordinates. The best score in each column is bold and the second-highest distinct score is underlined.

			Classification		Retrieval	
			Top-1 Acc. (%) $\uparrow$		NDCG@10 (%) $\uparrow$	
Category	Method	Dim.	MTOPIntent	Banking77	NFCorpus	SciFact
Dense	Fine-tuned	4096	<u>93.96</u>	<u>93.73</u>	55.01	84.87
Sparse	CSR	256	96.85	94.38	56.12	73.20
	CSR	128	96.85	94.38	54.64	73.20
	ZoN (ours)	256	91.68	92.93	<u>55.46</u>	<u>82.06</u>
	ZoN (ours)	128	90.04	91.02	52.14	81.23

RESULTS. We compare with CSR, a sparse-coding approach (Wen et al., 2025), discussed in Chapter 5. Table 6.5 shows that the benefit

depends on the task. On SciFact, ZoN outperforms CSR, but remains below the full fine-tuned representation. On NFCorpus, ZoN with $k = 256$ slightly exceeds the full fine-tuned representation, while CSR is better overall. For classification, ZoN remains below both CSR and the baseline using NV-Embed with a projector fine-tuned on the task and all embedding coordinates retained. Increasing k from 128 to 256 improves ZoN on all four tasks. These results suggest that adaptive coordinate selection can preserve much of the retrieval performance.

6.6 CONCLUSION

This chapter introduced ZeroNeuron, an input-dependent sparse gate motivated by the key-value memory interpretation of Transformer MLPs. The rank analysis showed that dense intermediate activations occupy only a fraction of their available space, whereas fine-grained MoE experts use their smaller subspaces more fully. ZoN learns to select individual intermediate coordinates while constraining the average number of active gates across layers and tokens. At approximately 600M active parameters per token, ZoN-1.2B-A600M achieves a CORE score of 29.77, comparable to 29.61 for Dense-1.2B and above 27.35 for Dense-600M. The embedding experiments further show that adaptive coordinate selection can preserve much of the retrieval performance.

Together with Chapter 5, these results conclude the activation-sparsity part of the thesis: representational capacity need not be used uniformly for every input. Learned adaptive activation sparsity allows the amount of sparsity to depend on the input. Turning this flexibility into reliable runtime gains requires sparse implementations that exploit the learned activation patterns efficiently.

Part IV

CONCLUSION AND FUTURE WORK

OVERALL CONCLUSION AND FUTURE WORK

7.1 OVERALL CONCLUSION

The promises of scaling laws, where scaling up model size gives rise to new capabilities, are colliding with a harsh reality: training these large models and running inference are costly. The central research question is how to scale language models efficiently, both for training and inference. It is fortunate that pre-trained LLMs exhibit redundancy at different levels that can be exploited to improve their efficiency during post-training or pretraining. One example of such redundancy is the low-rank structure of pretrained Transformer activations. Our contributions exploit this observation to improve LLM efficiency in two main directions: **model compression** using activations and **activation sparsity**.

For post-training **model compression**, we propose **Lillama**, a method that locally distills the activations of Transformer layers into low-rank weights. This local optimization makes the approach more efficient than prior work: it enables compressing, on a single GPU, LLMs that would not otherwise fit on one. To achieve speedup, the method does not require specialized GPU kernels, unlike weight-sparsity approaches such as SparseGPT (Frantar and Alistarh, 2023a) or Wanda (Sun et al., 2024a). The approach is also sample-efficient, requiring only 13M tokens, compared to methods like Sheared-Llama (Xia et al., 2024), or Minitron (Muralidharan et al., 2024; Sreenivas et al., 2024b), which use over 50B tokens for compression. Sample efficiency is an important property in post-training compression, as it directly affects the fairness of the approach—for instance, its viability for under-represented languages. This is why we specifically studied Whisper compression in a low-resource setting, using Bambara as a case study. In this work, **BaldWhisper**, we again distill input/output embedding activations into low-rank weights, but push compression further by reducing the number of layers. To preserve the method’s sample efficiency, we do this by merging layers rather than removing them, to stay close to the original model. The resulting model is twice as fast while maintaining most of the performance.

For **activation sparsity**, we first study the eigenstructure of **Matryoshka embedding LLMs** in multilingual and multimodal representation learning. We show that Matryoshka embeddings, viewed as a form of structured activation sparsity, leave room for more effective use of the representation space: most of the activation energy is concentrated in relatively few principal components. The fixed slic-

ing operator applied to the embedding vectors does not preserve all the information needed for downstream tasks while introducing redundancy. We make the same observation for dense MLPs in autoregressive LLMs: their intermediate activations occupy a subspace substantially smaller than the allocated dimension. Sparse MoEs, however, project each token into smaller subspaces of the available intermediate space, and we show that these smaller subspaces are used more fully. These observations motivate an approach where the MLP has a large intermediate dimension but each token adaptively selects only the intermediate coordinates it needs rather than activating the whole available space. We introduce **ZeroNeuron (ZoN)**, an input-dependent gating mechanism that learns which coordinates to activate while constraining the average active dimension. Compared to fine-grained sparse mixture-of-experts and sparse memory layers, ZoN gets rid of TopK and predicts sparsity pointwise, which makes it not only more compute-adaptive but also more communication-efficient in distributed training, compared to TopK-based sparse mixture-of-experts and sparse memory layers that require an all-to-all token dispatch and all-to-all return. With approximately 600M active parameters per token, the 1.2B ZoN model matches the aggregate performance of its dense counterpart and outperforms a dense model of a comparable number of active parameters. These results show that model capacity can be preserved while reducing the parameters activated for each input, and that such sparsity can be exploited by GPU kernels to accelerate training and inference.

7.2 FUTURE WORK

We believe in sparsity, and we expect the most performant machine learning systems of the future to be sparsely activated. The remaining question is how this sparsity should be designed. We do not believe that TopK-based mixtures of experts or sparse memory layers are the right long-term answer: there is no reason to impose the same activation budget on every token, regardless of the task and its difficulty. Moreover, under current hardware constraints, distributed TopK routing incurs substantial cross-GPU expert communication. This is one of the biggest challenges in scaling fine-grained sparsity, and approaches such as LatentMoE (Elango et al., 2026) are trying to address it. Maybe this will no longer be a problem in a few years, if the next generation of hardware offers individual GPUs with terabytes of VRAM and the capacity to train LLMs with trillions of parameters on a single device. But beyond the engineering issues, TopK-based routing also faces the stability problems discussed earlier, including dead experts and expert collapse. Rather than putting patches everywhere to keep TopK-based sparsity working, we believe it can be beneficial to fundamentally revisit how sparsity is learned. We believe that such approaches must

be adaptive, communication-efficient, and stable by design. ZoN is our first step in this direction. We plan to investigate its scalability through larger models and longer training runs, while developing GPU kernels that translate its learned sparsity into actual computational savings.

Part V

APPENDIX

APPENDIX TEST

Lorem ipsum at nusquam appellantur his, ut eos erant homero concludaturque. Albucius appellantur deterruisset id eam, vivendum partiendo dissentiet ei ius. Vis melius facilisis ea, sea id convenire referrentur, takimata adolescens ex duo. Ei harum argumentum per. Eam vidit exerci appetere ad, ut vel zzril intellegam interpretaris.

More dummy text.

A.1 APPENDIX SECTION TEST

Test: [Table A.1](#) (This reference should have a lowercase, small caps A if the option floatperchapter is activated, just as in the table itself → however, this does not work at the moment.)

LABITUR BONORUM PRI NO	QUE VISTA	HUMAN
fastidii ea ius	germano	demonstratea
suscipit instructor	titulo	personas
quaestio philosophia	facto	demonstrated

Table A.1: Autem usu id.

A.2 ANOTHER APPENDIX SECTION TEST

Equidem detraxit cu nam, vix eu delenit periculis. Eos ut vero constituto, no vidit propriae complectitur sea. Diceret nonummy in has, no qui eligendi recteque consetetur. Mel eu dictas suscipiantur, et sed placerat oporteat. At ipsum electram mei, ad aequae atomorum mea. There is also a useless Pascal listing below: [Listing A.1](#).

Listing A.1: A floating example (listings manual)

```
for i:=maxint downto 0 do
begin
{ do nothing }
end;
```


BIBLIOGRAPHY

- Abdaoui, Amine, Camille Pradel, and Grégoire Sigel (2020). “Load What You Need: Smaller Versions of Multilingual BERT.” In: *Proceedings of SustaiNLP: Workshop on Simple and Efficient Natural Language Processing*. Association for Computational Linguistics, pp. 119–123.
- Adelani, David Ifeoluwa et al. (2024). *Irokobench: A new benchmark for african languages in the age of large language models*. Preprint, arXiv:2406.03368.
- Agarwal, Rishabh, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem (2024). “On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes.” In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=3zKtaqxLhW>.
- Aghajanyan, Armen, Luke Zettlemoyer, and Sonal Gupta (2020). *Intrinsic dimensionality explains the effectiveness of language model fine-tuning*. Preprint, arXiv:2012.13255.
- Alabi, Jesujoba O., Xuechen Liu, Dietrich Klakow, and Junichi Yamagishi (2025). “AfriHuBERT: A self-supervised speech representation model for African languages.” In: *Interspeech 2025*, pp. 4023–4027. DOI: [10.21437/Interspeech.2025-1437](https://doi.org/10.21437/Interspeech.2025-1437).
- Anshumann, Mohd Abbas Zaidi, Akhil Kedia, Jinwoo Ahn, Taehwak Kwon, Kangwook Lee, Haejun Lee, and Joohyung Lee (2025). “Sparse Logit Sampling: Accelerating Knowledge Distillation in LLMs.” In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, pp. 18085–18108. DOI: [10.18653/v1/2025.acl-long.885](https://doi.org/10.18653/v1/2025.acl-long.885). URL: <https://aclanthology.org/2025.acl-long.885/>.
- Arun Babu and Changhan Wang and Andros Tjandra and Kushal Lakhotia and Qiantong Xu and Naman Goyal and Kritika Singh and Patrick von Platen and Yatharth Saraf and Juan Pino and Alexei Baevski and Alexis Conneau and Michael Auli (2022). “XLS-R: Self-supervised Cross-lingual Speech Representation Learning at Scale.” In: *Interspeech 2022*, 2278–2282. DOI: [10.21437/Interspeech.2022-143](https://doi.org/10.21437/Interspeech.2022-143).
- Ashkboos, Saleh, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman (2024). *SliceGPT: Compress Large Language Models by Deleting Rows and Columns*. arXiv preprint; ArXiv:2401.15024 [cs].
- Baevski, Alexei, Arun Babu, Wei-Ning Hsu, and Michael Auli (2023). *Efficient Self-supervised Learning with Contextualized Target Represen-*

- tations for Vision, Speech and Language. arXiv: 2212.07525 [cs.LG]. URL: <https://arxiv.org/abs/2212.07525>.
- Baevski, Alexei, Henry Zhou, Abdelrahman Mohamed, and Michael Auli (2020a). *wav2vec 2.0: A framework for self-supervised learning of speech representations*. Preprint, arXiv:2006.11477.
- Baevski, Alexei, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli (2020b). “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations.” In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., pp. 12449–12460. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/92d1e1eb1cd6f9fba3227870bb6d7f07-Paper.pdf.
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio (2015). “Neural Machine Translation by Jointly Learning to Align and Translate.” In: *International Conference on Learning Representations*.
- Bisk, Yonatan, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi (2020). *Piqa: Reasoning about physical commonsense in natural language*. In Thirty-Fourth AAAI Conference on Artificial Intelligence.
- Bonifacio, Luiz Henrique, Israel Campiotti, Roberto Lotufo, and Rodrigo Nogueira (2021). “mMARCO: A Multilingual Version of MS MARCO Passage Ranking Dataset.” In: *arXiv:2108.13897*.
- Cai, Mu, Jianwei Yang, Jianfeng Gao, and Yong Jae Lee (2024). *Matryoshka Multimodal Models*. arXiv: 2405.17430 [cs.CV]. URL: <https://arxiv.org/abs/2405.17430>.
- Cappellazzo, Umberto, Minsu Kim, and Stavros Petridis (2025). *Adaptive Audio-Visual Speech Recognition via Matryoshka-Based Multimodal LLMs*. arXiv: 2503.06362 [cs.CV]. URL: <https://arxiv.org/abs/2503.06362>.
- Caubrière, Antoine and Elodie Gauthier (2024). *Africa-Centric Self-Supervised Pre-Training for Multilingual Speech Representation in a Sub-Saharan Context*. arXiv: 2404.02000 [cs.CL]. URL: <https://arxiv.org/abs/2404.02000>.
- Chang, Heng-Jui, Shu wen Yang, and Hung yi Lee (2022). *Distilhubert: Speech representation learning by layer-wise distillation of hidden-unit bert*. Preprint, arXiv:2110.01900.
- Chang, Xuankai et al. (2021). *An exploration of self-supervised pretrained representations for end-to-end speech recognition*. Preprint, arXiv:2110.04590.
- Chen, Patrick, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh (2021). *Drone: Data-aware low-rank compression for large nlp models*. In *Advances in Neural Information Processing Systems*, volume 34, pages 29321–29334. Curran Associates, Inc.
- Chomsky, Noam (1956). “Three Models for the Description of Language.” In: *IRE Transactions on Information Theory* 2.3, pp. 113–124. doi: 10.1109/TIT.1956.1056813.
- Chomsky, Noam (1965). *Aspects of the Theory of Syntax*. Cambridge, MA: MIT Press.

- Clark, Christopher, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova (2019a). *Boolq: Exploring the surprising difficulty of natural yes/no questions*. ArXiv, abs/1905.10044.
- Clark, Kevin, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning (2019b). “What Does BERT Look At? An Analysis of BERT’s Attention.” In: *Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Association for Computational Linguistics, pp. 276–286. DOI: [10.18653/v1/W19-4828](https://doi.org/10.18653/v1/W19-4828). URL: <https://aclanthology.org/W19-4828/>.
- Clark, Peter, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord (2018). *Think you have solved question answering? try arc, the ai2 reasoning challenge*. ArXiv, abs/1803.05457.
- Computer, Together (2023). *Redpajama: An open source recipe to reproduce llama training dataset*.
- Conneau, Alexis, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov (2019). “Unsupervised Cross-lingual Representation Learning at Scale.” In: *CoRR* abs/1911.02116. arXiv: [1911.02116](https://arxiv.org/abs/1911.02116). URL: <http://arxiv.org/abs/1911.02116>.
- Conneau, Alexis, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna (2022). “FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.” In: *arXiv preprint arXiv:2205.12446*. URL: <https://arxiv.org/abs/2205.12446>.
- DIOP, Thierno Ibrahima (Feb. 2021). *WOLOF ASR data on urban transport*. DOI: [10.5281/zenodo.4525892](https://doi.org/10.5281/zenodo.4525892). URL: <https://doi.org/10.5281/zenodo.4525892>.
- Dai, Damai, Li Dong, Yaru Hao, Zhifang Sui, Baobao Chang, and Furu Wei (2022). *Knowledge Neurons in Pretrained Transformers*. arXiv: [2104.08696](https://arxiv.org/abs/2104.08696) [cs.CL]. URL: <https://arxiv.org/abs/2104.08696>.
- DeepSeek-AI (2024a). *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. arXiv: [2405.04434](https://arxiv.org/abs/2405.04434) [cs.CL]. URL: <https://arxiv.org/abs/2405.04434>.
- DeepSeek-AI (2024b). *DeepSeek-V3 Technical Report*. arXiv: [2412.19437](https://arxiv.org/abs/2412.19437) [cs.CL]. URL: <https://arxiv.org/abs/2412.19437>.
- DeepSeek-AI (2026). *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. arXiv: [2606.19348](https://arxiv.org/abs/2606.19348) [cs.CL]. URL: <https://arxiv.org/abs/2606.19348>.
- DeepSeek-AI, Aixin Liu, et al. (2025). *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models*. arXiv: [2512.02556](https://arxiv.org/abs/2512.02556) [cs.CL]. URL: <https://arxiv.org/abs/2512.02556>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova (2018). “BERT: Pre-training of Deep Bidirectional Transformers for

- Language Understanding.” In: *CoRR* abs/1810.04805. arXiv: [1810.04805](https://arxiv.org/abs/1810.04805). URL: <http://arxiv.org/abs/1810.04805>.
- Devvrit et al. (2024). “MatFormer: Nested Transformer for Elastic Inference.” In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 140535–140564. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/fe96a0a1399c4a2c11d27a3a46092e5d-Abstract-Conference.html.
- Do, Truong, Minh-Phuong Nguyen, and Le-Minh Nguyen (2025). “PruneSLU: Efficient On-device Spoken Language Understanding through Vocabulary and Structural Pruning.” In: *Interspeech 2025*, pp. 1738–1742.
- Elango, Venmugil et al. (2026). *LatentMoE: Toward Optimal Accuracy per FLOP and Parameter in Mixture of Experts*. arXiv: [2601.18089](https://arxiv.org/abs/2601.18089) [cs.LG]. URL: <https://arxiv.org/abs/2601.18089>.
- Elizalde, Benjamin, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang (2022). *CLAP: Learning Audio Concepts From Natural Language Supervision*. arXiv: [2206.04769](https://arxiv.org/abs/2206.04769) [cs.SD]. URL: <https://arxiv.org/abs/2206.04769>.
- Fang, Junfeng, Houcheng Jiang, Kun Wang, Yunshan Ma, Shi Jie, Xiang Wang, Xiangnan He, and Tat seng Chua (2025). *AlphaEdit: Null-Space Constrained Knowledge Editing for Language Models*. arXiv: [2410.02355](https://arxiv.org/abs/2410.02355) [cs.CL]. URL: <https://arxiv.org/abs/2410.02355>.
- Fedus, William, Barret Zoph, and Noam Shazeer (Jan. 2022). “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.” In: *Journal of Machine Learning Research* 23.1. ISSN: 1532-4435.
- Frankle, Jonathan and Michael Carbin (2019). *The lottery ticket hypothesis: Finding sparse, trainable neural networks*. Preprint, arXiv:1803.03635.
- Frantar, Elias and Dan Alistarh (2023a). “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot.” In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, pp. 10323–10337. URL: <https://proceedings.mlr.press/v202/frantar23a.html>.
- Frantar, Elias and Dan Alistarh (2023b). *Sparsegpt: Massive language models can be accurately pruned in one-shot*. Preprint, arXiv:2301.00774.
- Gandhi, Sanchit, Patrick von Platen, and Alexander M. Rush (2023). *Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling*. Preprint, arXiv:2311.00430.
- Gao, Leo et al. (2023). *A framework for few-shot language model evaluation*.
- Gauthier, Elodie, Laurent Besacier, Sylvie Voisin, Michael Melese, and Uriel Pascal Elingui (2016). “Collecting Resources in Sub-Saharan African Languages for Automatic Speech Recognition: a Case Study of Wolof.” In: *LREC*.
- Gauthier, Elodie, Aminata Ndiaye, and Abdoulaye Guissé (May 2024). “Kallaama: A Transcribed Speech Dataset about Agriculture in the Three Most Widely Spoken Languages in Senegal.” In: *Proceedings*

- of the Fifth Workshop on Resources for African Indigenous Languages @ LREC-COLING 2024. Ed. by Rooweither Mabuya, Muzi Matfunjwa, Mmasibidi Setaka, and Menno van Zaanen. Torino, Italia: ELRA and ICCL, pp. 10–19. URL: <https://aclanthology.org/2024.rail-1.2/>.
- Ge, Tao, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu (2024). “Scaling Synthetic Data Creation with 1,000,000,000 Personas.” In: *arXiv preprint arXiv:2406.20094*. DOI: [10.48550/arXiv.2406.20094](https://arxiv.org/abs/2406.20094). URL: <https://arxiv.org/abs/2406.20094>.
- Gemma Team (2024). “Gemma 2: Improving Open Language Models at a Practical Size.” In: *arXiv preprint arXiv:2408.00118*. URL: <https://arxiv.org/abs/2408.00118>.
- Geva, Mor, Roei Schuster, Jonathan Berant, and Omer Levy (Nov. 2021). “Transformer Feed-Forward Layers Are Key-Value Memories.” In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Ed. by Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen tau Yih. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, pp. 5484–5495. DOI: [10.18653/v1/2021.emnlp-main.446](https://aclanthology.org/2021.emnlp-main.446/). URL: <https://aclanthology.org/2021.emnlp-main.446/>.
- Godey, Nathan and Yoav Artzi (2026). *Lost in Backpropagation: The LM Head is a Gradient Bottleneck*. arXiv: [2603.10145 \[cs.CL\]](https://arxiv.org/abs/2603.10145). URL: <https://arxiv.org/abs/2603.10145>.
- Godey, Nathan, Éric de la Clergerie, and Benoît Sagot (2024). *Why do Small Language Models Underperform? Studying Language Model Saturation via the Softmax Bottleneck*. arXiv: [2404.07647 \[cs.CL\]](https://arxiv.org/abs/2404.07647). URL: <https://arxiv.org/abs/2404.07647>.
- Grattafiori, Aaron et al. (2024). *The Llama 3 Herd of Models*. arXiv: [2407.21783 \[cs.AI\]](https://arxiv.org/abs/2407.21783). URL: <https://arxiv.org/abs/2407.21783>.
- Gritsch, Nikolas, Qizhen Zhang, Acyr Locatelli, Sara Hooker, and Ahmet Üstün (Nov. 2025). “Nexus: Adaptive Upcycling to Efficiently Pretrain Mixture of Experts.” In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. Ed. by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, pp. 24364–24381. DOI: [10.18653/v1/2025.findings-emnlp.1323](https://aclanthology.org/2025.findings-emnlp.1323/). URL: <https://aclanthology.org/2025.findings-emnlp.1323/>.
- Gu, Albert and Tri Dao (2024). *Mamba: Linear-time sequence modeling with selective state spaces*. Preprint, arXiv:2312.00752.
- He, Haorui et al. (2024). “Emilia: An Extensive, Multilingual, and Diverse Speech Dataset for Large-Scale Speech Generation.” In: *Proc. of SLT*.
- He, Haorui et al. (2025). “Emilia: A Large-Scale, Extensive, Multilingual, and Diverse Dataset for Speech Generation.” In: *arXiv:2501.15907*.

- He, Junhui, Zhihui Fu, Jun Wang, and Qingan Li (2026). *POP: Prefill-Only Pruning for Efficient Large Model Inference*. arXiv: 2602.03295 [cs.CL]. URL: <https://arxiv.org/abs/2602.03295>.
- He, Xu Owen (2024). *Mixture of A Million Experts*. arXiv: 2407.04153 [cs.LG]. URL: <https://arxiv.org/abs/2407.04153>.
- Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean (2015). *Distilling the Knowledge in a Neural Network*. Preprint, arXiv:1503.02531.
- Hoffmann, Jordan et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv: 2203.15556 [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- Hsieh, Cheng-Yu, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister (2023). “Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes.” In: *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, pp. 8003–8017. DOI: 10.18653/v1/2023.findings-acl.507. URL: <https://aclanthology.org/2023.findings-acl.507>.
- Hsu, Wei-Ning, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed (Oct. 2021). “HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units.” In: *IEEE/ACM Trans. Audio, Speech and Lang. Proc.* 29, 3451–3460. ISSN: 2329-9290. DOI: 10.1109/TASLP.2021.3122291. URL: <https://doi.org/10.1109/TASLP.2021.3122291>.
- Hu, Wenbo, Zi-Yi Dou, Liunian Harold Li, Amita Kamath, Nanyun Peng, and Kai-Wei Chang (2024). *Matryoshka Query Transformer for Large Vision-Language Models*. arXiv: 2405.19315 [cs.CV]. URL: <https://arxiv.org/abs/2405.19315>.
- Huang, Quzhe et al. (2024). *Harder Tasks Need More Experts: Dynamic Routing in MoE Models*. arXiv: 2403.07652 [cs.LG]. URL: <https://arxiv.org/abs/2403.07652>.
- Huang, Zihao, Qiyang Min, Hongzhi Huang, Defa Zhu, Yutao Zeng, Ran Guo, and Xun Zhou (2025). *Ultra-Sparse Memory Network*. arXiv: 2411.12364 [cs.LG]. URL: <https://arxiv.org/abs/2411.12364>.
- Jiang, Albert Q. et al. (2024). *Mixtral of Experts*. arXiv preprint; ArXiv:2401.04088 [cs]. URL: <https://arxiv.org/abs/2401.04088>.
- Jin, Hongye, Linwei Li, Xiaotian Han, Xin Liu, Haoyang Wen, Tuo Zhao, Qingyu Yin, and Binxuan Huang (July 2026). *Mitigate Silent Expert Death In Ultra-Sparse MoE*. URL: <https://alltoall.notion.site/save-lower-layer-moe-experts-llal>.
- Kandji, Abdou Karim, Frédéric Precioso, Cheikh Ba, Samba Ndiaye, and Augustin Ndione (2025). *WolBanking77: Wolof Banking Speech Intent Classification Dataset*. arXiv: 2509.19271 [cs.CL]. URL: <https://arxiv.org/abs/2509.19271>.
- Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu,

- and Dario Amodei (2020). *Scaling Laws for Neural Language Models*. arXiv: 2001.08361 [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.
- Kasai, Jungo, Nikolaos Pappas, Hao Peng, James Cross, and Noah A. Smith (2021). *Deep encoder, shallow decoder: Reevaluating non-autoregressive machine translation*. Preprint, arXiv:2006.10369.
- Kim, Yoon and Alexander M. Rush (2016). “Sequence-Level Knowledge Distillation.” In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 1317–1327. DOI: 10.18653/v1/D16-1139. URL: <https://aclanthology.org/D16-1139>.
- Kimi Team (2026). *Kimi K3: Open Frontier Intelligence*. arXiv: 2607.24653 [cs.CL]. URL: <https://arxiv.org/abs/2607.24653>.
- Knuth, Donald E. (1974). “Computer Programming as an Art.” In: *Communications of the ACM* 17.12, pp. 667–673.
- Kovaleva, Olga, Alexey Romanov, Anna Rogers, and Anna Rumshisky (2019). “Revealing the Dark Secrets of BERT.” In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*. Association for Computational Linguistics, pp. 4365–4374. DOI: 10.18653/v1/D19-1445. URL: <https://aclanthology.org/D19-1445/>.
- Krajewski, Jakub et al. (2024). *Scaling Laws for Fine-Grained Mixture of Experts*. arXiv: 2402.07871 [cs.LG]. URL: <https://arxiv.org/abs/2402.07871>.
- Kusupati, Aditya et al. (2022). “Matryoshka Representation Learning.” In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc., pp. 30233–30249. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/c32319f4868da7613d78af9993100e42-Paper-Conference.pdf.
- Lam, Tsz Kin, Marco Gaido, Sara Papi, Luisa Bentivogli, and Barry Haddow (Apr. 2025). “Prepending or Cross-Attention for Speech-to-Text? An Empirical Comparison.” In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Albuquerque, New Mexico: Association for Computational Linguistics, pp. 2994–3006. ISBN: 979-8-89176-189-6. DOI: 10.18653/v1/2025.naacl-long.153. URL: <https://aclanthology.org/2025.naacl-long.153/>.
- Lample, Guillaume, Alexandre Sablayrolles, Marc’Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou (2019). *Large Memory Layers with Product Keys*. arXiv: 1907.05242 [cs.CL]. URL: <https://arxiv.org/abs/1907.05242>.
- Lan, Zhenzhong, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut (2020). *ALBERT: A Lite BERT for*

- Self-supervised Learning of Language Representations*. arXiv: 1909.11942 [cs.CL]. URL: <https://arxiv.org/abs/1909.11942>.
- Lee, Donghyun, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini (2024). *CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models*. arXiv: 2404.08763 [cs.LG]. URL: <https://arxiv.org/abs/2404.08763>.
- Li, Chunyuan, Heerad Farkhoor, Rosanne Liu, and Jason Yosinski (2018). *Measuring the intrinsic dimension of objective landscapes*. Preprint, arXiv:1804.08838.
- Li, Jeffrey et al. (2025). *DataComp-LM: In search of the next generation of training sets for language models*. arXiv: 2406.11794 [cs.LG]. URL: <https://arxiv.org/abs/2406.11794>.
- Li, Zhuohan, Eric Wallace, Sheng Shen, Kevin Lin, Kurt Keutzer, Dan Klein, and Joseph E. Gonzalez (2020). *Train Large, Then Compress: Rethinking Model Size for Efficient Training and Inference of Transformers*. arXiv: 2002.11794 [cs.CL]. URL: <https://arxiv.org/abs/2002.11794>.
- Lian, Wing, Guan Wang, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknium" (2023). *Slimorca: An open dataset of gpt-4 augmented flan reasoning traces, with verification*.
- Lin, Stephanie, Jacob Hilton, and Owain Evans (2022). *TruthfulQA: Measuring how models mimic human falsehoods*. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, Dublin, Ireland. Association for Computational Linguistics.
- Liu, Alexander H. et al. (2026a). *Ministral 3*. arXiv: 2601.08584 [cs.CL]. URL: <https://arxiv.org/abs/2601.08584>.
- Liu, Deyuan et al. (2024). "Pruning via Merging: Compressing LLMs via Manifold Alignment Based Layer Merging." In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, pp. 17817–17829.
- Liu, Haotian, Chunyuan Li, Qingyang Wu, and Yong Jae Lee (2023a). "Visual Instruction Tuning." In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., pp. 34892–34916. URL: https://proceedings.neurips.cc/paper_files/paper/2023/file/6dcf277ea32ce3288914faf369fe6de0-Paper-Conference.pdf.
- Liu, Hong et al. (2026b). *Scaling Embeddings Outperforms Scaling Experts in Language Models*. arXiv: 2601.21204 [cs.CL]. URL: <https://arxiv.org/abs/2601.21204>.
- Liu, James, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun (2025). *Training-Free Activation Sparsity in Large Language Models*. arXiv: 2408.14690 [cs.CL]. URL: <https://arxiv.org/abs/2408.14690>.

- Liu, Jian, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang (2020). *Logiqa: A challenge dataset for machine reading comprehension with logical reasoning*. Preprint, arXiv:2007.08124.
- Liu, Xin, Simin Ma, Shujian Liu, Song Wang, Sathish Reddy Indurthi, Haoyun Deng, Lu Wang, and Kaiqiang Song (2026c). “Understanding Knowledge Distillation in Post-Training: When It Helps and When It Fails.” In: *arXiv preprint arXiv:2606.22942*. URL: <https://arxiv.org/abs/2606.22942>.
- Liu, Zichang et al. (2023b). *Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time*. arXiv: 2310.17157 [cs.LG]. URL: <https://arxiv.org/abs/2310.17157>.
- Louizos, Christos, Max Welling, and Diederik P. Kingma (2018). “Learning Sparse Neural Networks through L_0 Regularization.” In: *International Conference on Learning Representations*. URL: <https://arxiv.org/abs/1712.01312>.
- Ma, Xinyin, Gongfan Fang, and Xinchao Wang (2023). “LLM-Pruner: On the Structural Pruning of Large Language Models.” In: *Advances in Neural Information Processing Systems*. Vol. 36, pp. 21702–21720. URL: <https://openreview.net/forum?id=J8Ajf9WfXP>.
- Markov, A. (Jan. 2007). “An Example of Statistical Investigation of the Text Eugene Onegin Concerning the Connection of Samples in Chains.” In: *Science in Context* 19, pp. 591–600. DOI: [10.1017/S0269889706001074](https://doi.org/10.1017/S0269889706001074).
- Meituan LongCat Team et al. (2025). *LongCat-Flash Technical Report*. arXiv: 2509.01322 [cs.CL]. URL: <https://arxiv.org/abs/2509.01322>.
- Men, Xin, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen (2024). “ShortGPT: Layers in Large Language Models are More Redundant Than You Expect.” In: *arXiv preprint arXiv:2403.03853*. URL: <https://arxiv.org/abs/2403.03853>.
- Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov (2023a). *Locating and Editing Factual Associations in GPT*. arXiv: 2202.05262 [cs.CL]. URL: <https://arxiv.org/abs/2202.05262>.
- Meng, Kevin, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau (2023b). *Mass-Editing Memory in a Transformer*. arXiv: 2210.07229 [cs.CL]. URL: <https://arxiv.org/abs/2210.07229>.
- Michel, Paul, Omer Levy, and Graham Neubig (2019). “Are Sixteen Heads Really Better than One?” In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc. URL: <https://proceedings.neurips.cc/paper/2019/hash/2c601ad9d2ff9bc8b282670cdd54f69f-Abstract.html>.
- Mihaylov, Todor, Peter Clark, Tushar Khot, and Ashish Sabharwal (2018). *Can a suit of armor conduct electricity? a new dataset for open book question answering*. In EMNLP.

- Mirzadeh, Iman, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar (2023). *ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models*. arXiv: 2310.04564 [cs.LG]. URL: <https://arxiv.org/abs/2310.04564>.
- Muennighoff, Niklas et al. (2025). *OLMoE: Open Mixture-of-Experts Language Models*. arXiv: 2409.02060 [cs.CL]. URL: <https://arxiv.org/abs/2409.02060>.
- Mukherjee, Subhabrata, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah (2023). *Orca: Progressive Learning from Complex Explanation Traces of GPT-4*. DOI: 10.48550/arXiv.2306.02707. URL: <https://arxiv.org/abs/2306.02707>.
- Muralidharan, Saurav, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov (2024). “Compact Language Models via Pruning and Knowledge Distillation.” In: *arXiv preprint arXiv:2407.14679*. URL: <https://arxiv.org/abs/2407.14679>.
- NLLB Team et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation*. arXiv: 2207.04672 [cs.CL]. URL: <https://arxiv.org/abs/2207.04672>.
- Oord, Aaron van den, Yazhe Li, and Oriol Vinyals (2019). *Representation Learning with Contrastive Predictive Coding*. arXiv: 1807.03748 [cs.LG]. URL: <https://arxiv.org/abs/1807.03748>.
- Panayotov, Vassil, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur (2015). “Librispeech: An ASR corpus based on public domain audio books.” In: *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5206–5210. DOI: 10.1109/ICASSP.2015.7178964.
- Pareras, Oriol, Gerard I. Gállego, Federico Costa, Cristina España-Bonet, and Javier Hernando (2025). *Revisiting Direct Speech-to-Text Translation with Speech LLMs: Better Scaling than CoT Prompting?* arXiv: 2510.03093 [cs.CL]. URL: <https://arxiv.org/abs/2510.03093>.
- Pasad, Ankita, Ju-Chieh Chou, and Karen Livescu (2022). *Layer-wise analysis of a self-supervised speech representation model*. Preprint, arXiv:2107.04734.
- Peng, Letian, Yuwei Zhang, Zilong Wang, Jayanth Srinivasa, Gaowen Liu, Zihan Wang, and Jingbo Shang (Aug. 2024). “Answer is All You Need: Instruction-following Text Embedding via Answering the Question.” In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, pp. 459–477. DOI: 10.18653/v1/2024.acl-long.27. URL: <https://aclanthology.org/2024.acl-long.27/>.
- Press, Ofir and Lior Wolf (Apr. 2017). “Using the Output Embedding to Improve Language Models.” In: *Proceedings of the 15th Conference*

- of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers. Ed. by Mirella Lapata, Phil Blunsom, and Alexander Koller. Valencia, Spain: Association for Computational Linguistics, pp. 157–163. URL: <https://aclanthology.org/E17-2025/>.
- Qwen Team (Mar. 2024). *Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters*. <https://qwenlm.github.io/blog/qwen-moe/>.
- Qwen Team (2026a). *On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability*. arXiv: 2608.30320 [cs.CL]. URL: <https://arxiv.org/abs/2608.30320>.
- Qwen Team (2026b). *Qwen3.8-2.4T-A95B*. <https://huggingface.co/Qwen/Qwen3.8-2.4T-A95B>.
- Radford, Alec, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever (2022a). *Robust Speech Recognition via Large-Scale Weak Supervision*. Preprint, arXiv:2212.04356.
- Radford, Alec, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever (2022b). *Robust speech recognition via large-scale weak supervision*. Preprint, arXiv:2212.04356.
- Radford, Alec et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. arXiv: 2103.00020 [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- Reddy, Chandan K A, Vishak Gopal, and Ross Cutler (2021). *DNSMOS: A Non-Intrusive Perceptual Objective Speech Quality metric to evaluate Noise Suppressors*. arXiv: 2010.15258 [cs.SD]. URL: <https://arxiv.org/abs/2010.15258>.
- Roy, Aurko et al. (2022). *N-Grammer: Augmenting Transformers with Latent n-grams*. arXiv: 2207.06366 [cs.CL]. URL: <https://arxiv.org/abs/2207.06366>.
- Sakaguchi, Keisuke, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi (2019). *Winogrande: An adversarial winograd schema challenge at scale*. arXiv preprint arXiv:1907.10641.
- Sap, Maarten, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi (2019). *Social iqa: Commonsense reasoning about social interactions*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 4463–4473.
- Schmidt, Fabian David, Philipp Borchert, Ivan Vulić, and Goran Glavaš (Nov. 2024). “Self-Distillation for Model Stacking Unlocks Cross-Lingual NLU in 200+ Languages.” In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, pp. 6724–6743. DOI: 10.18653/v1/2024.findings-emnlp.394. URL: <https://aclanthology.org/2024.findings-emnlp.394/>.

- Schmidt, Fabian David, Ivan Vulić, Goran Glavaš, and David Ifeoluwa Adelani (2025). *Fleurs-SLU: A Massively Multilingual Benchmark for Spoken Language Understanding*. arXiv: 2501.06117 [cs.CL]. URL: <https://arxiv.org/abs/2501.06117>.
- Schneider, Steffen, Alexei Baevski, Ronan Collobert, and Michael Auli (2019). *wav2vec: Unsupervised Pre-training for Speech Recognition*. arXiv: 1904.05862 [cs.CL]. URL: <https://arxiv.org/abs/1904.05862>.
- Shannon, Claude E. (1948). "A Mathematical Theory of Communication." In: *Bell System Technical Journal* 27.3, pp. 379–423.
- Shazeer, Noam (2020). *GLU Variants Improve Transformer*. Preprint, arXiv:2002.05202.
- Singh, Shivalika et al. (2024). *Aya dataset: An open-access collection for multilingual instruction tuning*. Preprint, arXiv:2402.06619.
- Sinha, Akshit, Arvinth Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping (2026). *The Illusion of Diminishing Returns: Measuring Long Horizon Execution in LLMs*. arXiv: 2509.09677 [cs.AI]. URL: <https://arxiv.org/abs/2509.09677>.
- Sreenivas, Sharath Turuvekere, Saurav Muralidharan, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov (2024a). *Llm pruning and distillation in practice: The minitron approach*. Preprint, arXiv:2408.11796.
- Sreenivas, Sharath Turuvekere et al. (2024b). "LLM Pruning and Distillation in Practice: The Minitron Approach." In: *arXiv preprint arXiv:2408.11796*. URL: <https://arxiv.org/abs/2408.11796>.
- StepFun Team (2026). *Step 3.5 Flash: Open Frontier-Level Intelligence with 11B Active Parameters*. arXiv: 2602.10604 [cs.CL]. URL: <https://arxiv.org/abs/2602.10604>.
- Sun, Mingjie, Zhuang Liu, Anna Bair, and J. Zico Kolter (2024a). "A Simple and Effective Pruning Approach for Large Language Models." In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=PxoFut3dWW>.
- Sun, Mingjie, Zhuang Liu, Anna Bair, and J. Zico Kolter (2024b). *A simple and effective pruning approach for large language models*. Preprint, arXiv:2306.11695.
- Sy, Yaya, Dioula Doucouré, Christophe Cerisara, and Irina Illina (2025). *Speech Language Models for Under-Represented Languages: Insights from Wolof*. arXiv: 2509.15362 [cs.CL]. URL: <https://arxiv.org/abs/2509.15362>.
- Tang, Yuqing, Chau Tran, Xian Li, Peng-Jen Chen, Naman Goyal, Vishrav Chaudhary, Jiatao Gu, and Angela Fan (2020). "Multilingual Translation with Extensible Multilingual Pretraining and Finetuning." In: arXiv: 2008.00401 [cs.CL].
- Taori, Rohan, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto (2023). *Stanford Alpaca: An Instruction-Following LLaMA Model*. URL: https://github.com/tatsu-lab/stanford_alpaca.

- Team, Qwen (2024). *Qwen2.5: A Party of Foundation Models*. URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- Thawakar, Omkar, Ashmal Vayani, Salman Khan, Hisham Cholakkal, Rao Muhammad Anwer, Michael Felsberg, Timothy Baldwin, Eric P. Xing, and Fahad Shahbaz Khan (2024). *Mobillama: Towards accurate and lightweight fully transparent gpt*. Preprint, arXiv:2402.16840.
- Tonja, Atnafu Lambebo et al. (2024). *Inkubalm: A small language model for low-resource african languages*. arXiv preprint arXiv:2408.17024.
- Touvron, Hugo et al. (2023a). *LLaMA: Open and Efficient Foundation Language Models*. arXiv: 2302.13971 [cs.CL]. URL: <https://arxiv.org/abs/2302.13971>.
- Touvron, Hugo et al. (2023b). *Llama 2: Open Foundation and Fine-Tuned Chat Models*. arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288>.
- Tunstall, Lewis, Nils Reimers, Unso Eun Seo Jo, Luke Bates, Daniel Korat, Moshe Wasserblat, and Oren Pereg (2022). *Efficient Few-Shot Learning Without Prompts*. DOI: 10.48550/ARXIV.2209.11055. URL: <https://arxiv.org/abs/2209.11055>.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin (2017). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Voita, Elena, David Talbot, Fedor Moiseev, Rico Sennrich, and Ivan Titov (2019). “Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned.” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 5797–5808. DOI: 10.18653/v1/P19-1580. URL: <https://aclanthology.org/P19-1580/>.
- Wang, Keyu, Tian Lyu, Guinan Su, Jonas Geiping, Lu Yin, Marco Canini, and Shiwei Liu (2025a). *When Fewer Layers Break More Chains: Layer Pruning Harms Test-Time Scaling in LLMs*. arXiv: 2510.22228 [cs.CL]. URL: <https://arxiv.org/abs/2510.22228>.
- Wang, Lean, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai (2024a). *Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts*. arXiv: 2408.15664 [cs.LG]. URL: <https://arxiv.org/abs/2408.15664>.
- Wang, Xin, Yu Zheng, Zhongwei Wan, and Mi Zhang (2024b). *SVD-LLM: Truncation-aware Singular Value Decomposition for Large Language Model Compression*. arXiv preprint arXiv:2403.07378.
- Wang, Ziheng, Jeremy Wohlwend, and Tao Lei (Nov. 2020). “Structured Pruning of Large Language Models.” In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu. Online: Association for Computational Linguistics, pp. 6151–

6162. DOI: [10.18653/v1/2020.emnlp-main.496](https://doi.org/10.18653/v1/2020.emnlp-main.496). URL: <https://aclanthology.org/2020.emnlp-main.496/>.
- Wang, Ziteng, Jun Zhu, and Jianfei Chen (2025b). *ReMoE: Fully Differentiable Mixture-of-Experts with ReLU Routing*. arXiv: [2412.14711](https://arxiv.org/abs/2412.14711) [cs.LG]. URL: <https://arxiv.org/abs/2412.14711>.
- Wei, Jason et al. (2022). *Emergent Abilities of Large Language Models*. arXiv: [2206.07682](https://arxiv.org/abs/2206.07682) [cs.CL]. URL: <https://arxiv.org/abs/2206.07682>.
- Wen, Tiansheng, Yifei Wang, Zequn Zeng, Zhong Peng, Yudi Su, Xinyang Liu, Bo Chen, Hongwei Liu, Stefanie Jegelka, and Chenyu You (2025). *Beyond Matryoshka: Revisiting Sparse Coding for Adaptive Representation*. arXiv: [2503.01776](https://arxiv.org/abs/2503.01776) [cs.LG]. URL: <https://arxiv.org/abs/2503.01776>.
- Weston, Jason, Sumit Chopra, and Antoine Bordes (2015). *Memory Networks*. arXiv: [1410.3916](https://arxiv.org/abs/1410.3916) [cs.AI]. URL: <https://arxiv.org/abs/1410.3916>.
- Wolf, Thomas et al. (2020). *Huggingface’s transformers: State-of-the-art natural language processing*. Preprint, arXiv:1910.03771.
- Wortsman, Mitchell et al. (2022). “Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time.” In: *Proceedings of the 39th International Conference on Machine Learning*. Vol. 162. PMLR, pp. 23965–23998.
- Wu, Jian et al. (2023). *On decoder-only architecture for speech-to-text and large language model integration*. arXiv: [2307.03917](https://arxiv.org/abs/2307.03917) [eess.AS]. URL: <https://arxiv.org/abs/2307.03917>.
- Xia, Mengzhou, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen (2023). *Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning*. arXiv preprint; ArXiv:2310.06694 [cs].
- Xia, Mengzhou, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen (2024). “Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning.” In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=09i0dae0zp>.
- Xu, Zhangchen, Fengqing Jiang, Luyao Niu, Yuntian Deng, Radha Poovendran, Yejin Choi, and Bill Yuchen Lin (2025). “Magpie: Alignment Data Synthesis from Scratch by Prompting Aligned Large Language Models with Nothing.” In: *The Thirteenth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=G7sN4c7Sw0>.
- Yadav, Prateek, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal (2023). “TIES-Merging: Resolving Interference When Merging Models.” In: *Advances in Neural Information Processing Systems*.
- Yang, An et al. (2024a). “Qwen2 Technical Report.” In: *arXiv preprint arXiv:2407.10671*.

- Yang, Yifei, Zouying Cao, and Hai Zhao (2024b). “LaCo: Large Language Model Pruning via Layer Collapse.” In: *arXiv preprint arXiv:2402.11187*. URL: <https://arxiv.org/abs/2402.11187>.
- Yang, Zhilin, Zihang Dai, Ruslan Salakhutdinov, and William W. Cohen (2018). *Breaking the Softmax Bottleneck: A High-Rank RNN Language Model*. arXiv: 1711.03953 [cs.CL]. URL: <https://arxiv.org/abs/1711.03953>.
- Yin, Lu et al. (2024). “Outlier Weighed Layerwise Sparsity (OWL): A Missing Secret Sauce for Pruning LLMs to High Sparsity.” In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, pp. 57101–57115. URL: <https://proceedings.mlr.press/v235/yin24e.html>.
- Yu, Hao and Jianxin Wu (2023). *Compressing Transformers: Features Are Low-Rank, but Weights Are Not!* Proceedings of the AAAI Conference on Artificial Intelligence, 37(9):11007–11015.
- Yu, Le, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li (2024). “Language Models Are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch.” In: *Proceedings of the 41st International Conference on Machine Learning*. JMLR.org.
- Yuan, Zheng, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou (2023). “Scaling Relationship on Learning Mathematical Reasoning with Large Language Models.” In: *arXiv preprint arXiv:2308.01825*. DOI: 10.48550/arXiv.2308.01825. URL: <https://arxiv.org/abs/2308.01825>.
- Zanon Boito, Marceley, Vivek Iyer, Nikolaos Lagos, Laurent Besacier, and Ioan Calapodescu (2024). “mHuBERT-147: A Compact Multilingual HuBERT Model.” In: *Interspeech 2024*, pp. 3939–3943. DOI: 10.21437/Interspeech.2024-938.
- Zhang, Biao, Lixin Chen, Tong Liu, and Bo Zheng (Nov. 2025). “SMEC: Rethinking Matryoshka Representation Learning for Retrieval Embedding Compression.” In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Ed. by Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng. Suzhou, China: Association for Computational Linguistics, pp. 26220–26233. ISBN: 979-8-89176-332-6. DOI: 10.18653/v1/2025.emnlp-main.1332. URL: <https://aclanthology.org/2025.emnlp-main.1332/>.
- Zhang, Qizhen et al. (2024a). “BAM! Just Like That: Simple and Efficient Parameter Upcycling for Mixture of Experts.” In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 56304–56321. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/68179e00db78ec1c708f6352c0f1724a-Abstract-Conference.html.
- Zhang, Xueyao et al. (2024b). “Amphion: An Open-Source Audio, Music and Speech Generation Toolkit.” In: *Proc. of SLT*.
- Zhang, Yingtao, Haoli Bai, Haokun Lin, Jialin Zhao, Lu Hou, and Carlo Vittorio Cannistraci (2024c). “Plug-and-Play: An Efficient Post-

- training Pruning Method for Large Language Models.” In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=Tr0lPx9woF>.
- Zhang, Yuxin, Lirui Zhao, Mingbao Lin, Yunyun Sun, Yiwu Yao, Xingjia Han, Jared Tanner, Shiwei Liu, and Rongrong Ji (2024d). “Dynamic Sparse No Training: Training-Free Fine-Tuning for Sparse LLMs.” In: *The Twelfth International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=1ndDmZdT4g>.
- Zhao, Siyan, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover (2026). *Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models*. arXiv: 2601.18734 [cs.LG]. URL: <https://arxiv.org/abs/2601.18734>.
- Zheng, Haizhong, Xiaoyan Bai, Xueshen Liu, Z. Morley Mao, Beidi Chen, Fan Lai, and Atul Prakash (2024). “Learn-To-be-Efficient: Build Structured Sparsity in Large Language Models.” In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 101969–101991. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/b8f10193cab43d45df9bb810637333fd-Paper-Conference.pdf.
- al., Gemma Team et (2025). *Gemma 3 Technical Report*. Preprint, arXiv:2503.19786.

DECLARATION

Put your declaration here.

Saarbrücken, February 2025

André Miede & Ivo Pletikosić

COLOPHON

This document was typeset using the typographical style `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst’s seminal book on typography “*The Elements of Typographic Style*”. `classicthesis` is available for both $\text{\LaTeX}$ and $\text{\LyX}$:

<https://bitbucket.org/amiede/classicthesis/>

Happy users of `classicthesis` usually send a real postcard to the author. A collection of postcards received so far is featured here:

<http://postcards.miede.de/>

Thank you very much for your feedback and contribution.